

Python Perfectionnement



Frédéric Gaurat

Formateur et développeur

Horaires

- 9h00
- pause
- 12h30
- 13h30
- pause
- 17h00

Plan de cours

Types et structures avancés

Introspection et métaprogrammation

Programmation fonctionnelle avancée

Programmation fonctionnelle

Générateurs et itérateurs

Concepts POO avancés

Design patterns

Métaprogrammation avancée

Context managers avancés

Écosystème Python avancé

Optimisation de performance

Programmation parallèle

Applications distribuées

Chapitre 1

Types et structures avancés

Python Perfectionnement — Jour 1

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Comprendre** le modèle d'objets de Python : tout est objet, typage dynamique *fort*
- **Distinguer** mutabilité et immutabilité, et leurs conséquences (pièges inclus)
- **Maîtriser** le slicing avancé sur les séquences
- **Exploiter** les structures de `collections` : `namedtuple`, `deque`, `Counter`, `defaultdict`, `ChainMap`
- **Choisir** la bonne structure de données selon le besoin (lisibilité **et** performance)
- **Optimiser** la mémoire : `__slots__`, `array`, `memoryview`, `generators`

Travaux pratiques : manipulation des types, slicing avancé, optimisation de structures.

Le modèle d'objets Python : tout est objet

En Python, **absolument tout** est un objet : les entiers, les fonctions, les classes, les modules...

```
>>> isinstance(42, object)
True
>>> isinstance(print, object)
True
>>> isinstance(int, object)      # même les types sont des objets !
True
```

Chaque objet possède trois caractéristiques fondamentales :

Caractéristique	Accès	Description
Identité	<code>id(obj)</code>	Adresse mémoire (en CPython), unique pendant la vie de l'objet
Type	<code>type(obj)</code>	Détermine les opérations possibles, ne change jamais
Valeur	l'objet lui-même	Modifiable ou non selon la mutabilité du type

Typage dynamique... mais fort

Dynamique

Le type est porté par l'**objet**, pas par la variable. Une variable est une simple **étiquette** (référence).

```
x = 42      # x référence un int
x = "hello" # x référence maintenant un str
x = [1, 2]  # puis une liste – aucun souci
```

 ne « contient » rien : elle **pointe** vers un objet.

Fort

Pas de conversion implicite hasardeuse entre types incompatibles (contrairement à JavaScript ou PHP) :

```
>>> "3" + 4
TypeError: can only concatenate str
(not "int") to str

>>> "3" * 4      # défini explicitement
'3333'
>>> int("3") + 4 # conversion explicite
7
```

À retenir : Python ne devine jamais — il faut convertir explicitement.

Variables = références : conséquences

```
a = [1, 2, 3]
b = a          # b référence LE MÊME objet (pas de copie !)
b.append(4)
print(a)       # [1, 2, 3, 4] ← a est modifié aussi
print(a is b)  # True – même identité
```

is vs ==

- `is` → compare les **identités** (même objet ?)
- `==` → compare les **valeurs** (`__eq__`)

```
x = [1, 2]; y = [1, 2]
x == y      # True  (mêmes valeurs)
x is y      # False (objets distincts)
```

Copies

```
import copy
b = a.copy()      # copie superficielle
b = a[:]          # idem (séquences)
b = copy.deepcopy(a) # copie profonde
```

⚠ La copie superficielle **partage** les objets internes (listes imbriquées !)

Mutabilité vs immutabilité

Immutable	Mutable
<code>int</code> , <code>float</code> , <code>complex</code> , <code>bool</code>	<code>list</code> , <code>dict</code> , <code>set</code>
<code>str</code> , <code>bytes</code> , <code>tuple</code> , <code>frozenset</code>	<code>bytearray</code>
<code>range</code> , <code>NoneType</code>	instances de classes (par défaut)

```
s = "hello"
s.upper()      # crée un NOUVEL objet - s inchangé !
s = s.upper()  # il faut réaffecter

t = (1, 2, [3, 4])
t[2].append(5) # ✅ légal ! le tuple est immutable,
               # mais la liste qu'il contient ne l'est pas
print(t)       # (1, 2, [3, 4, 5])
```

Subtilité : l'immuabilité d'un conteneur porte sur ses **références**, pas sur les objets référencés.

Pourquoi l'immuabilité compte

1. Clés de dict / éléments de set

Seuls les objets **hachables** (donc immutables, en général) sont acceptés :

```
d = {(1, 2): "ok"}      # ✅ tuple
d = {[1, 2]: "ko"}      # ❌ TypeError:
                        # unhashable type: 'list'
fs = frozenset({1, 2})
nested = {fs: "un set comme clé !"}
```

2. Sécurité du partage

Un objet immuable peut être partagé entre threads / fonctions **sans risque** de modification surprise.

3. Le piège classique : argument par défaut mutable

```
def append_to(item, target=[]): # ❌ PIÈGE
    target.append(item)
    return target

append_to(1) # [1]
append_to(2) # [1, 2] ← surprise !
```

La liste par défaut est créée **une seule fois**, à la définition de la fonction.

```
def append_to(item, target=None): # ✅
    if target is None:
        target = []
    target.append(item)
    return target
```

Types numériques : au-delà des bases

int — précision arbitraire

Pas de dépassement de capacité en Python :

```
>>> 2 ** 1000
10715086071862673209484250490600018105614048...
>>> import sys
>>> sys.getsizeof(0), sys.getsizeof(2**1000)
(28, 160) # la taille mémoire s'adapte
```

Séparateurs de lisibilité et bases :

```
budget = 1_000_000 # lisible !
mask    = 0b1010_1010 # binaire
color   = 0xFF_A0_00 # hexa
perms   = 0o755 # octal
```

float — IEEE 754 double précision

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> import math
>>> math.isclose(0.1 + 0.2, 0.3)
True # la bonne pratique
```

decimal.Decimal — précision exacte

Indispensable pour la finance :

```
from decimal import Decimal, getcontext
getcontext().prec = 28 # précision réglable

Decimal("0.1") + Decimal("0.2") # Decimal('0.3') ✓
# ⚠ toujours construire depuis un str :
Decimal(0.1) # Decimal('0.1000000000000000055511...')
```

fractions.Fraction — rationnels exacts

```
from fractions import Fraction
Fraction(1, 3) + Fraction(1, 6) # Fraction(1, 2)
Fraction("0.25") # Fraction(1, 4)
```

En résumé

Besoin	Type
Calcul général	float (+ math.isclose)
Argent, comptabilité	Decimal
Rationnels exacts	Fraction
Calcul scientifique massif	NumPy (chap. 10)

Les séquences : panorama

Une **séquence** est un conteneur ordonné, indexable, itérable, avec une longueur.

Type	Mutable	Contenu	Usage typique
<code>list</code>	✓	hétérogène	collection générale, modifiable
<code>tuple</code>	✗	hétérogène	enregistrement, retour multiple, clé de dict
<code>str</code>	✗	caractères Unicode	texte
<code>bytes</code>	✗	octets (0–255)	données binaires, I/O, réseau
<code>bytearray</code>	✓	octets	buffer binaire modifiable
<code>range</code>	✗	entiers calculés	itération, mémoire O(1)

Toutes partagent le même **protocole** : `len()` , `in` , indexation `[i]` , slicing `[a:b:c]` , concaténation `+` , répétition `*` , itération...

```
"abc"[1]      # 'b'
(1, 2, 3)[-1] # 3
b"abc"[0]     # 97  ⚠ un bytes indexé renvoie un int !
```

tuple vs list : bien choisir

tuple — données hétérogènes figées

Un tuple est un **enregistrement** : chaque position a un sens.

```
point = (48.85, 2.35)      # (lat, lon)
person = ("Alice", 30, "Lyon") # nom, âge, ville

# Retour multiple = tuple implicite
def min_max(data):
    return min(data), max(data)

lo, hi = min_max([3, 1, 4]) # unpacking
```

Avantages :

- Hachable → clé de dict, élément de set
- Plus **léger** et plus rapide à créer qu'une liste
- Intention claire : « ne sera pas modifié »

list — collection homogène évolutive

Une liste contient des éléments *de même nature* qu'on parcourt, filtre, trie...

```
temperatures = [21.5, 22.0, 19.8]
temperatures.append(23.1)
temperatures.sort()
```

Performance (à taille égale)

```
import sys
sys.getsizeof((1, 2, 3)) # 64 octets
sys.getsizeof([1, 2, 3]) # 88 octets
```

La liste sur-alloue pour amortir les `append` (croissance amortie **O(1)**).

Règle simple : structure figée et positionnelle → `tuple` ; collection qui évolue → `list` .

Slicing : les fondamentaux

Syntaxe : `sequence[start:stop:step]` — `stop` est **exclu** (intervalle semi-ouvert).

```
letters = ['a', 'b', 'c', 'd', 'e', 'f']
# index:  0   1   2   3   4   5
# négatif:-6  -5  -4  -3  -2  -1

letters[1:4]    # ['b', 'c', 'd']    (indices 1, 2, 3)
letters[:3]     # ['a', 'b', 'c']    (début → 2)
letters[3:]     # ['d', 'e', 'f']    (3 → fin)
letters[-2:]    # ['e', 'f']        (les 2 derniers)
letters[:]      # copie superficielle de toute la liste
```

Pourquoi `stop` **exclu** ? Les intervalles semi-ouverts se composent sans chevauchement :

```
letters[:k] + letters[k:] == letters    # toujours vrai
len(letters[a:b]) == b - a              # longueur évidente
```

Le slicing ne lève **jamais** `IndexError` : `letters[10:20]` renvoie simplement `[]`.

Slicing avancé : le pas (**step**)

```
nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

nums[::2]      # [0, 2, 4, 6, 8]    un élément sur deux
nums[1::2]     # [1, 3, 5, 7, 9]    les impairs
nums[::-1]     # [9, 8, ..., 0]     ← inversion idiomatique
nums[8:2:-2]   # [8, 6, 4]         à rebours, pas de 2
```

Applications classiques

```
# Palindrome ?
s == s[::-1]

# Inverser une chaîne
"Python"[::-1]      # 'nohtyP'

# Décimer un signal (1 point sur 10)
samples[::-10]
```

⚠ Pièges du pas négatif

Avec `step < 0`, `start` doit être après `stop` :

```
nums[2:8:-1]      # [] - vide !
nums[8:2:-1]      # [8, 7, 6, 5, 4, 3]
```

Préférez la lisibilité : `reversed(nums[3:9])` est souvent plus clair.

Slicing avancé : assignation et suppression

Sur les séquences **mutables**, un slice est aussi une **cible d'affectation** :

```
nums = [0, 1, 2, 3, 4, 5]

nums[1:3] = [10, 20, 30] # remplacement (tailles différentes OK)
# [0, 10, 20, 30, 3, 4, 5]

nums[::2] = [-1, -1, -1, -1] # avec un pas : tailles DOIVENT correspondre
del nums[1:3]                # suppression d'une tranche

nums[2:2] = [99, 98]         # insertion sans rien remplacer
nums[:] = [7, 8, 9]         # remplace LE CONTENU (garde l'identité !)
```

`nums[:] = ...` **VS** `nums = ...` — différence cruciale :

```
def reset(data):
    data = [] # ✗ réaffecte la variable locale, l'appelant ne voit rien
def reset(data):
    data[:] = [] # ✔ vide l'objet partagé, visible par l'appelant
```

Slicing avancé : objets `slice` nommés

`seq[a:b:c]` est du sucre syntaxique pour `seq[slice(a, b, c)]`. On peut donc nommer ses slices :

```
# Parsing d'un fichier à colonnes fixes (format legacy)
record = "20260604ALICE      00012999EUR"

DATE      = slice(0, 8)
NAME      = slice(8, 18)
AMOUNT    = slice(18, 26)
CURRENCY  = slice(26, 29)

record[DATE]      # '20260604'
record[NAME]      # 'ALICE      '
int(record[AMOUNT]) / 100 # 129.99
```

✓ **Clean code** : fini les indices magiques dispersés — la structure du format est documentée en un seul endroit.

```
s = slice(1, 10, 2)
s.start, s.stop, s.step # (1, 10, 2)
s.indices(6)           # (1, 6, 2) — bornes normalisées pour une longueur 6
```

`__getitem__` reçoit l'objet `slice` : vos propres classes peuvent supporter le slicing (chap. 6).

Unpacking : au-delà du simple

a, b = b, a

```
# L'étoile capture « le reste » (PEP 3132)
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2, 3, 4], last=5

head, *tail = range(5)          # tête / queue
*init, final = "abcd"           # init=['a','b','c'], final='d'

# Imbriqué
(a, b), (c, d) = (1, 2), (3, 4)

# Dans les boucles
points = [(1, 2, "rouge"), (3, 4, "bleu")]
for x, y, color in points:
    ...
for x, *rest in points:          # étoile aussi en boucle
    ...
```

```
# Unpacking dans les littéraux (PEP 448)
merged_list = [*range(3), *"ab"]      # [0, 1, 2, 'a', 'b']
merged_tuple = (*t1, *t2)
merged_dict = {**defaults, **overrides} # le dernier gagne
print(*[1, 2, 3], sep=" | ")          # 1 | 2 | 3
```

str vs **bytes** : la frontière texte / binaire

str — texte Unicode

Suite de points de code Unicode (pas d'octets !).

```
s = "café ☺"  
len(s)      # 6 caractères  
ord("é")    # 233  
"\N{SNOWMAN}" # '☺'
```

bytes — données brutes

Suite d'octets (0–255), immutable.

```
b = b"caf\xc3\xa9"  
len(b)      # 5 octets  
b[0]        # 99 (un int !)
```

La conversion est toujours explicite

```
"café".encode("utf-8")    # b'caf\xc3\xa9'  
b"caf\xc3\xa9".decode("utf-8") # 'café'  
"café".encode("latin-1") # b'caf\xe9'
```

Règle du sandwich : décoder à l'entrée, travailler en **str**, encoder à la sortie.

bytearray — buffer mutable

```
buf = bytearray(b"hello")  
buf[0] = ord("H")  
buf.extend(b" world")  
bytes(buf)    # b'Hello world'
```

Idéal pour construire des trames réseau ou modifier des données binaires sans copie.

memoryview : manipuler sans copier

Un `memoryview` expose la mémoire d'un objet supportant le *buffer protocol* (`bytes`, `bytearray`, `array`, tableaux NumPy...) **sans duplication**.

```
data = bytearray(10_000_000)    # 10 Mo
chunk = data[:1_000_000]       # ✗ COPIE 1 Mo
view = memoryview(data)
chunk = view[:1_000_000]       # ✓ aucune copie – fenêtre sur data

chunk[0] = 255                 # modifie data directement !
data[0]                         # 255
```

Cas d'usage : découper de gros fichiers binaires, parser des protocoles réseau, passer des sous-tranches à `socket.send()` sans dupliquer les buffers.

```
# Envoi par blocs sans copie (pattern classique)
with memoryview(payload) as view:
    while view:
        sent = sock.send(view[:4096])
        view = view[sent:]      # avance sans copier
```

⚠ Tant qu'un `memoryview` existe, l'objet source ne peut pas être redimensionné.

dict : le couteau suisse, version moderne

Depuis Python 3.7, le dict **préserve l'ordre d'insertion** (garanti par le langage).

```
config = {"host": "localhost", "port": 8080}

# Accès sécurisé
config.get("timeout", 30) # 30 - pas de KeyError
config.setdefault("retries", 3) # insère si absent, renvoie la valeur

# Fusion (Python 3.9+, PEP 584)
defaults = {"port": 80, "debug": False}
final = defaults | config # le dict de DROITE gagne
config |= {"debug": True} # fusion en place
```

```
# Les vues sont DYNAMIQUES (pas des copies)
keys = config.keys()
config["new"] = 1
"new" in keys # True - la vue suit le dict

# Et se comportent comme des ensembles :
common = d1.keys() & d2.keys() # clés communes
diff = d1.keys() - d2.keys()
```

Compréhensions de dict et de set

```
# Dict comprehension
squares = {n: n**2 for n in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

# Inverser un dict (si les valeurs sont uniques et hachables)
inverted = {v: k for k, v in squares.items()}

# Filtrer
actifs = {k: v for k, v in users.items() if v.active}

# Transformer des clés
normalized = {k.lower().strip(): v for k, v in raw.items()}

# Set comprehension
extensions = {f.suffix for f in path.iterdir()} # unicité gratuite
```

✓ Bonnes pratiques

- Une compréhension = **une** transformation lisible
- Nommer le résultat avec un nom métier

✗ À éviter

- Compréhensions imbriquées illisibles → boucle classique
- Effets de bord dans une compréhension

`collections.defaultdict` : la valeur par défaut automatique

Problème : grouper des éléments oblige à tester l'existence de la clé.

Sans `defaultdict`

```
groups = {}
for word in words:
    key = word[0]
    if key not in groups:      # boilerplate
        groups[key] = []
    groups[key].append(word)
```

Avec `defaultdict`

```
from collections import defaultdict

groups = defaultdict(list)
for word in words:
    groups[word[0]].append(word)
```

La *factory* (`list`, `int`, `set`, ou toute callable sans argument) est appelée à chaque **clé manquante** :

```
counter = defaultdict(int)      # compteur : int() == 0
index   = defaultdict(set)      # index inversé
tree    = lambda: defaultdict(tree) # arbre auto-vivifiant 🌳
```

⚠ Une simple **lecture** `d[clé_absente]` crée l'entrée ! Utilisez `.get()` ou `in` pour tester sans créer.

`collections.Counter` : compter en une ligne

```
from collections import Counter

votes = ["python", "rust", "python", "go", "python", "rust"]
c = Counter(votes)
# Counter({'python': 3, 'rust': 2, 'go': 1})

c.most_common(2)      # [('python', 3), ('rust', 2)]
c["python"]           # 3
c["java"]             # 0 - pas de KeyError pour une clé absente
c.total()             # 6 (Python 3.10+)
```

`Counter` est un dict spécialisé qui supporte l'arithmétique multi-ensembles :

```
stock = Counter(pommes=10, poires=5)
commande = Counter(pommes=3, poires=7)

stock - commande      # Counter({'pommes': 7}) - jamais de négatif avec -
stock + commande      # addition des comptes
stock & commande      # minimum : Counter({'pommes': 3, 'poires': 5})

# Mise à jour incrémentale
c = Counter()
for line in log_file:
    c.update(extract_ips(line))
```

ChainMap et OrderedDict

ChainMap — recherche en cascade

Empile plusieurs dicts **sans les fusionner** : la recherche parcourt les couches dans l'ordre.

```
from collections import ChainMap

cli_args = {"port": 9000}
env_vars = {"host": "0.0.0.0"}
defaults = {"host": "localhost",
            "port": 8080, "debug": False}

config = ChainMap(cli_args, env_vars, defaults)
config["port"] # 9000 (couche CLI)
config["host"] # '0.0.0.0' (couche env)
config["debug"] # False (couche defaults)
```

-  **Vue dynamique** : modifier une couche se reflète immédiatement
-  Les écritures vont dans la **première** couche
- Cas d'usage : configuration en couches, portées de variables (interpréteurs)
- Différence avec `{**defaults, **env}` : pas de copie, et la liaison reste **vivante**

OrderedDict — encore utile ?

Depuis 3.7, `dict` préserve l'ordre... mais `OrderedDict` garde des atouts :

```
from collections import OrderedDict

od = OrderedDict(a=1, b=2, c=3)
od.move_to_end("a") # déplace en fin
od.move_to_end("c", last=False) # ou en tête
od.popitem(last=False) # pop FIFO
```

- L'égalité `OrderedDict == OrderedDict` est **sensible à l'ordre** (pas celle de `dict`)
- `move_to_end` + `popitem(last=False)` = brique idéale pour un **cache LRU** maison

```
class LRUCache(OrderedDict):
    def __init__(self, maxsize=128):
        super().__init__()
        self.maxsize = maxsize
    def __setitem__(self, key, value):
        super().__setitem__(key, value)
        self.move_to_end(key)
        if len(self) > self.maxsize:
            self.popitem(last=False)
```

(En production : `functools.lru_cache` — chap. 3)

set et frozenset : l'algèbre des ensembles

Collections **non ordonnées** d'éléments **uniques** et **hachables**. Test d'appartenance en $O(1)$.

```
admins = {"alice", "bob"}
devs   = {"bob", "carol", "dan"}

admins | devs      # union : tous
admins & devs      # intersection : {'bob'}
admins - devs      # différence : {'alice'}
admins ^ devs      # diff. symétrique : dans un seul des deux

admins <= devs     # sous-ensemble ? False
{"bob"} < devs     # sous-ensemble strict ? True
```

Idiomes courants

```
# Dédupliquer (ordre perdu)
unique = set(items)
# Dédupliquer EN GARDANT l'ordre
unique = list(dict.fromkeys(items))

# Validation de champs
missing = required_fields - data.keys()
if missing:
    raise ValueError(f"Champs manquants : {missing}")
```

frozenset — le set immutable

```
ROLES = frozenset({"admin", "dev", "ops"})

# Hachable → utilisable comme clé !
matrix = {
    frozenset({"a", "b"}): "paire AB",
}
```

⚠️ {} crée un **dict** vide. Un set vide s'écrit `set()`.

Le test d'appartenance : `list` vs `set`

Le réflexe de performance n°1 sur les structures :

```
import timeit

haystack_list = list(range(100_000))
haystack_set = set(haystack_list)

timeit.timeit(lambda: 99_999 in haystack_list, number=1000) # ~0.9 s
timeit.timeit(lambda: 99_999 in haystack_set, number=1000) # ~0.00005 s
```

Opération	<code>list</code>	<code>set</code> / <code>dict</code>
<code>x in collection</code>	$O(n)$ — parcours linéaire	$O(1)$ — table de hachage

```
# ❌ Anti-pattern fréquent : recherche répétée dans une liste
valid_ids = load_ids_as_list() # 50 000 éléments
results = [r for r in records if r.id in valid_ids] #  $O(n \times m)$  !

# ✅ Convertir UNE FOIS en set
valid_ids = set(load_ids_as_list())
results = [r for r in records if r.id in valid_ids] #  $O(n)$ 
```

Règle : dès qu'on teste l'appartenance **plus d'une fois**, on convertit en `set`.

`collections.namedtuple` : des tuples lisibles

Problème : `row[2]` ne dit rien. **Solution :** nommer les champs.

```
from collections import namedtuple

Point = namedtuple("Point", ["x", "y"])
p = Point(3, 4)

p.x, p.y          # 3, 4 – accès par NOM
x, y = p          # reste un tuple : unpacking OK
p[0]              # 3 – indexation OK
p.x = 10          # ❌ AttributeError : immutable
```

```
# Méthodes utilitaires
p._asdict()        # {'x': 3, 'y': 4}
p._replace(x=10)   # Point(x=10, y=4) – nouvelle instance
Point._fields      # ('x', 'y')
Point(**{"x": 1, "y": 2}) # depuis un dict

# Cas d'usage typique : lignes de résultats SQL / CSV
Employee = namedtuple("Employee", "name dept salary")
rows = [Employee(*row) for row in cursor.fetchall()]
top = [e.name for e in rows if e.salary > 50_000] # lisible !
```

Même coût mémoire qu'un tuple — bien plus léger qu'un dict ou qu'une instance classique.

typing.NamedTuple et dataclasses : la version moderne

typing.NamedTuple

namedtuple + annotations de types + valeurs par défaut + méthodes :

```
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float
    label: str = ""

    def norm(self) -> float:
        return (self.x**2 + self.y**2) ** 0.5

p = Point(3.0, 4.0)
p.norm()      # 5.0
```

- ✓ Immutable, léger, compatible unpacking
- ✓ Vérifiable par mypy / pyright

@dataclass

La classe « porteuse de données » sans boilerplate :

```
from dataclasses import dataclass, field

@dataclass
class Order:
    ref: str
    items: list[str] = field(default_factory=list)
    total: float = 0.0

o = Order("CMD-001")
# __init__, __repr__, __eq__ générés !
```

- ✓ **Mutable** par défaut (contrairement à NamedTuple)
- ✓ `field(default_factory=list)` évite le piège du mutable partagé

@dataclass : les options qui changent tout

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)    # slots= : Python 3.10+
class Config:
    host: str
    port: int = 8080

c = Config("localhost")
c.port = 9999          # ❌ FrozenInstanceError - immutable
hash(c)                # ✅ frozen=True rend hachable
```

Option	Effet
<code>frozen=True</code>	Instances immutables + hachables (utilisables en clés/sets)
<code>slots=True</code>	Pas de <code>__dict__</code> → mémoire réduite , accès plus rapides
<code>order=True</code>	Génère <code><</code> , <code><=</code> , <code>></code> , <code>>=</code> (comparaison champ à champ)
<code>kw_only=True</code>	Force les arguments nommés à la construction

```
@dataclass
class Measure:
    value: float
    unit: str = "m"
    def __post_init__(self):          # validation après __init__
        if self.value < 0:
            raise ValueError("valeur négative")
```

Quel « porteur de données » choisir ?

Critère	tuple	namedtuple / NamedTuple	@dataclass	dict	TypedDict
Accès par nom	✗	✓	✓	d["k"]	d["k"]
Mutable	✗	✗	✓ (option frozen)	✓	✓
Typage statique	partiel	✓	✓	✗	✓
Méthodes métier	✗	✓	✓	✗	✗
Coût mémoire	minimal	minimal	moyen (slots=True → faible)	élevé	élevé
Unpacking a, b = x	✓	✓	✗	✗	✗

```
from typing import TypedDict

class UserPayload(TypedDict):
    name: str
    age: int
    email: str

def handle(payload: UserPayload) -> None:
    payload["name"]
```

Guide de décision :

- Données **externes** (JSON, API) → TypedDict • Valeur **immutable** légère → NamedTuple
- Objet **métier** avec comportement → @dataclass • Besoin de validation forte → Pydantic (chap. 10)

collections.deque : la file à deux bouts

`list.pop(0)` et `list.insert(0, x)` sont $O(n)$ (décalage de tous les éléments). `deque` fait les deux extrémités en $O(1)$.

```
from collections import deque

d = deque([1, 2, 3])
d.appendleft(0)    # O(1) → deque([0, 1, 2, 3])
d.popleft()        # O(1) → 0
d.append(4)         # O(1)
d.rotate(1)         # décale circulairement
```

File FIFO

```
queue = deque()
queue.append(task)    # producteur
task = queue.popleft() # consommateur
```

(Inter-threads → `queue.Queue`, chap. 12)

Buffer circulaire avec maxlen

```
last_logs = deque(maxlen=100)
for line in stream:
    last_logs.append(line)
# garde automatiquement les 100
# dernières lignes – les anciennes
# sont évincées à gauche
```

⚠ En revanche, l'accès indexé `d[i]` au **milieu** est $O(n)$ — la liste reste meilleure pour ça.

array et les structures spécialisées

array.array — tableaux compacts

Une liste de 10M de `float` stocke 10M d'objets Python. `array` stocke les valeurs brutes :

```
from array import array
import sys

floats_list = [0.0] * 1_000_000
floats_arr = array("d", [0.0]) * 1_000_000

sys.getsizeof(floats_list) # ~8 Mo
sys.getsizeof(floats_arr) # ~8 Mo de doubles BRUTS
# mais la liste référence des objets float
# de 24 octets chacun → ~32 Mo au total !
```

Typecodes : `"b"` int8, `"i"` int32, `"d"` double...

Compatible `memoryview` et écriture binaire directe.

Pour du **calcul**, passez à NumPy (chap. 10) : vectorisation + mémoire compacte.

bisect — recherche dichotomique

Sur une liste triée, $O(\log n)$:

```
import bisect
grades = [60, 70, 80, 90]
bisect.bisect(grades, 75) # 2
bisect.insort(grades, 75) # insertion triée
```

heapq — file de priorité

```
import heapq
heap = []
heapq.heappush(heap, (2, "normal"))
heapq.heappush(heap, (1, "urgent"))
heapq.heappop(heap) # (1, 'urgent')
heapq.nlargest(3, scores) # top 3 en  $O(n \log k)$ 
```

Le plus petit élément sort toujours en premier — base des schedulers et du pattern *Worker Pool* (chap. 12).

`__slots__` : alléger ses objets

Par défaut, chaque instance porte un `__dict__` : flexible (attributs dynamiques) mais coûteux.

```
class PointDict:
    def __init__(self, x, y):
        self.x, self.y = x, y

class PointSlots:
    __slots__ = ("x", "y")          # structure FIGÉE
    def __init__(self, x, y):
        self.x, self.y = x, y
```

```
import sys
p1, p2 = PointDict(1, 2), PointSlots(1, 2)
sys.getsizeof(p1) + sys.getsizeof(p1.__dict__)  # ~48 + 296 octets
sys.getsizeof(p2)                             # ~48 octets, pas de __dict__

p2.z = 5    # ✗ AttributeError – attribut non déclaré
```

Gains mesurés sur des millions d'instances : **3x à 5x moins de mémoire**, accès aux attributs ~20 % plus rapides.

Quand l'utiliser : classes instanciées en masse (points, événements, lignes de données).

Moderne : `@dataclass(slots=True)` fait tout le travail proprement.

Mesurer avant d'optimiser

« *Premature optimization is the root of all evil* » — D. Knuth. On **measure**, puis on optimise (méthodologie complète au chap. 11).

Mémoire : `sys.getsizeof`

```
import sys
sys.getsizeof([])      # 56 (liste vide)
sys.getsizeof([1, 2, 3]) # 88
sys.getsizeof([1, 2, 3]) # 64
sys.getsizeof([1, 2, 3]) # 216
sys.getsizeof({"a": 1}) # 184
```

⚠ Taille du **conteneur seul** — pas des objets référencés ! Pour le total :

```
from pympler import asizeof # pip install pympler
sizeof, asizeof([[[1]*100]*100]) # taille récursive réelle
```

Vue d'ensemble : `tracemalloc` (stdlib)

```
import tracemalloc
tracemalloc.start()
data = load_everything()
snapshot = tracemalloc.take_snapshot()
for stat in snapshot.statistics("lineno")[:5]:
    print(stat) # top 5 des allocations
```

Temps : `timeit`

```
import timeit

timeit.timeit("x in c",
    setup="c = list(range(10_000)); x = 9_999",
    number=10_000) # liste : lent

timeit.timeit("x in c",
    setup="c = set(range(10_000)); x = 9_999",
    number=10_000) # set : ~100x plus rapide
```

En ligne de commande / notebook :

```
python -m timeit -s "c = set(range(10000))" "9999 in c"
```

```
%timeit 9999 in c # magic IPython / Jupyter
```

Le détail qui surprend : l'interning

```
a = 256; b = 256
a is b # True - petits ints mis en cache (-5..256)
a = 257; b = 257
a is b # False (selon le contexte d'exécution)
```

D'où la règle : `==` pour les valeurs, `is` uniquement pour `None` (et les sentinelles).

Complexité des opérations : l'antisèche

Opération	<code>list</code>	<code>deque</code>	<code>set</code>	<code>dict</code>
<code>x in c</code>	$O(n)$	$O(n)$	$O(1)$	$O(1)$
<code>append / add</code>	$O(1)^*$	$O(1)$	$O(1)$	$O(1)$
<code>pop()</code> fin	$O(1)$	$O(1)$	—	—
<code>pop(0) / popleft</code>	$O(n)$ ⚠	$O(1)$ ✅	—	—
<code>insert(0, x)</code>	$O(n)$ ⚠	$O(1)$	—	—
Accès <code>c[i]</code>	$O(1)$	$O(n)$ milieu	—	$O(1)$ par clé
<code>sort</code>	$O(n \log n)$	—	—	—
<code>min / max</code>	$O(n)$	$O(n)$	$O(n)$	$O(n)$

* amorti — la liste sur-alloue. Référence complète : wiki.python.org/moin/TimeComplexity

Les 3 erreurs les plus fréquentes en revue de code :

1. `in` sur une grosse liste dans une boucle → `set`
2. `pop(0) / insert(0, ...)` sur une liste → `deque`
3. Concaténation de chaînes en boucle `s += x` → `"".join(parts)`

Choisir sa structure : arbre de décision

```
Besoin ?
├── Paires clé → valeur ?
│   ├── valeurs par défaut automatiques ..... defaultdict
│   ├── compter des occurrences ..... Counter
│   ├── config en couches / cascade ..... ChainMap
│   └── cas général ..... dict
├── Éléments uniques / tests d'appartenance ?
│   ├── doit être hachable (clé, constante) .... frozenset
│   └── cas général ..... set
├── Séquence ordonnée ?
│   ├── ajouts/retraits aux DEUX bouts ..... deque
│   ├── figée, positionnelle ..... tuple
│   ├── numérique massive ..... array / NumPy
│   ├── toujours triée ..... list + bisect
│   ├── file de priorité ..... heapq
│   └── cas général ..... list
└── Enregistrement structuré ?
    ├── immutable, léger, unpacking ..... NamedTuple
    ├── objet métier, méthodes, mutable ..... @dataclass
    ├── JSON / dict externe à typer ..... TypedDict
    └── millions d'instances ..... slots=True
```

Bonnes pratiques — synthèse du chapitre

Clean code appliqué aux structures :

- Préférez les structures **immuables** par défaut (`tuple` , `frozenset` , `frozen=True`) : moins d'états possibles = moins de bugs
- Donnez du **sens** aux données : `namedtuple` / `dataclass` plutôt que tuples anonymes et indices magiques
- Une compréhension doit rester **lisible en une ligne de pensée** — sinon, boucle explicite
- `is` pour `None` , `==` pour les valeurs — toujours

Performance :

- Le choix de structure prime sur la micro-optimisation : **O(1) vs O(n)** se joue à la conception
- Test d'appartenance répété → `set` ; files → `deque` ; comptage → `Counter`
- Millions d'instances → `slots=True` ; gros buffers binaires → `memoryview`
- **Mesurez** (`timeit` , `tracemalloc`) avant d'optimiser

À ne pas oublier :

- Argument par défaut mutable = piège n°1 de Python
- Copie superficielle ≠ copie profonde
- `{}` est un dict vide, pas un set !

Travaux pratiques

TP 1.1 — Manipulation des types (30 min)

1. Écrivez une fonction qui met en évidence le piège de l'argument par défaut mutable, puis corrigez-la
2. Comparez `Decimal` et `float` sur un calcul de TVA cumulée (1000 lignes de facture à 19,6 %) — quantifiez la dérive
3. Démontrez avec `id()` et `is` le comportement de l'interning des petits entiers et des chaînes

TP 1.2 — Slicing avancé (30 min)

1. Parsez un fichier à colonnes fixes (fourni) avec des objets `slice` nommés
2. Implémentez `rotate(seq, n)` qui décale une séquence de n positions... en une ligne de slicing
3. À partir d'une liste de mesures, extrayez : les valeurs paires d'index, la liste inversée, une décimation 1/10
4. Remplacez « in place » tous les éléments d'index pair par leur opposé (assignation par slice étendu)

TP 1.3 — Optimisation des structures (45 min)

1. Un script (fourni) déduplique 500 000 enregistrements avec `record in seen_list` : mesurez avec `timeit`, optimisez avec la bonne structure, mesurez le gain
2. Le même script utilise `list.pop(0)` comme file de traitement : corrigez et mesurez
3. Modélisez un carnet d'ordres : choisissez et justifiez les structures (recherche par référence, file de priorité par prix, historique borné des 1000 derniers ordres)
4. Convertissez la classe `Order` en `@dataclass(slots=True)` et mesurez la mémoire avec `tracemalloc` sur 1 000 000 d'instances

Livrables : code + petites mesures chiffrées (avant/après) commentées en une phrase chacune.

Chapitre 2

Introspection et métaprogrammation

Python Perfectionnement — Jour 1

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Définir** introspection et métaprogrammation, et savoir quand y recourir
- **Interroger** n'importe quel objet à l'exécution : type, attributs, signature, source
- **Manipuler** dynamiquement les attributs : `getattr`, `setattr`, `hasattr`, `delattr`
- **Exploiter** le module `inspect` pour analyser fonctions, classes et piles d'appels
- **Construire** des outils dynamiques : plugins, dispatchers, validateurs
- **Évaluer** les risques : `eval` / `exec`, monkey patching, et leurs alternatives

Travaux pratiques : exploration de l'introspection, création d'outils dynamiques, tests et validation.

Les **métaclasses**, descripteurs avancés et imports personnalisés seront traités au **chapitre 8** — ici, nous posons les fondations.

Introspection et métaprogrammation : définitions

Introspection

Capacité d'un programme à **examiner** ses propres objets à l'exécution :

- Quel est le type de cet objet ?
- Quels attributs / méthodes possède-t-il ?
- Quelle est la signature de cette fonction ?
- D'où vient ce module ?

Lecture seule — on observe.

Qui s'en sert ? Tous les frameworks que vous utilisez : Django (ORM → classes depuis le schéma), pytest (découverte des `test_*`), FastAPI (signatures → validation), `dataclasses` (génération de `__init__`), pickle, debuggers, ORM...

C'est ce qui rend Python si adapté aux **frameworks** : le langage s'examine et se modifie lui-même.

Métaprogrammation

Capacité d'un programme à **créer ou modifier** du code à l'exécution :

- Ajouter des attributs/méthodes dynamiquement
- Créer des classes à la volée
- Générer des fonctions
- Intercepter les accès aux attributs

Écriture — on transforme.

type(), isinstance(), issubclass()

```

type(42)           # <class 'int'>
type(42) is int    # True - comparaison exacte

instance(42, int)  # True
instance(True, int) # True ⚠ bool hérite de int !
instance(42, (int, float)) # tuple = "ou logique"

issubclass(bool, int) # True
issubclass(int, object) # True - tout hérite d'object

```

type() vs isinstance()

```

class Animal: pass
class Chien(Animal): pass

c = Chien()
type(c) is Animal    # False - type exact
instance(c, Animal)  # True - héritage pris
#                    # en compte ✓

```

Règle : `instance()` respecte le polymorphisme — c'est presque toujours lui qu'il faut.

Mais d'abord... le duck typing

```

# ✗ Style défensif non pythonique
def process(data):
    if not isinstance(data, list):
        raise TypeError("liste attendue")

# ✓ EAFP : on essaie, on gère l'échec
def process(data):
    try:
        return [transform(x) for x in data]
    except TypeError:
        raise ValueError(f"itérable attendu")

```

`instance` se justifie aux **frontières** : entrées externes, dispatch, API publiques.

Explorer un objet : `dir()`, `vars()`, `__dict__`

```
class User:
    role = "guest"           # attribut de CLASSE
    def __init__(self, name):
        self.name = name    # attribut d'INSTANCE

u = User("alice")
```

```
dir(u)                # TOUT ce qui est accessible : instance + classe + héritage
# ['__class__', ..., 'name', 'role']

vars(u)               # équivalent de u.__dict__ : attributs d'INSTANCE seulement
# {'name': 'alice'}

vars(User)            # l'espace de noms de la classe
# {..., 'role': 'guest', '__init__': <function ...>}

u.__class__           # <class 'User'> – remonter à la classe
User.__mro__          # (User, object) – ordre de résolution des méthodes
User.__subclasses__() # qui hérite de moi ?
```

La chaîne de recherche d'attribut : `u.role` → cherche dans `u.__dict__`, puis `User.__dict__`, puis les bases (MRO). C'est ce mécanisme que la métaprogrammation exploite.

Accès dynamique aux attributs : la famille `*attr` 1/2

Le nom de l'attribut devient une **chaîne** — donc une donnée manipulable :

```
getattr(obj, "name")           # équivaut à obj.name
getattr(obj, "size", 0)        # valeur par défaut si absent
setattr(obj, "name", "bob")    # obj.name = "bob"
hasattr(obj, "close")          # l'attribut existe-t-il ?
delattr(obj, "cache")          # del obj.cache
```

Accès dynamique aux attributs : la famille `*attr` 2/2

Cas d'usage réel : hydrater un objet depuis un dict / une ligne SQL

```
class Product:
    FIELDS = ("ref", "label", "price")

    @classmethod
    def from_dict(cls, data: dict) -> "Product":
        obj = cls()
        for field in cls.FIELDS:
            setattr(obj, field, data[field])
        return obj

p = Product.from_dict({"ref": "A42", "label": "Clavier", "price": 49.9})
```

C'est — en miniature — ce que font tous les ORM et désérialiseurs.

Dispatch dynamique : remplacer les `if/elif` par `getattr`

Problème : router une commande vers la bonne méthode.

❌ La cascade qui enfle

```
class CLI:
    def run(self, cmd, *args):
        if cmd == "start":
            return self.do_start(*args)
        elif cmd == "stop":
            return self.do_stop(*args)
        elif cmd == "status":
            return self.do_status(*args)
        # ... à modifier à CHAQUE ajout
```

Violation d'*open/closed* : ajouter une commande oblige à modifier `run`.

Pattern utilisé par `cmd.Cmd` (stdlib), `unittest (test_*)`, `pytest`, `Django...`

✅ Dispatch par convention

```
class CLI:
    def run(self, cmd, *args):
        handler = getattr(self, f"do_{cmd}", None)
        if handler is None or not callable(handler):
            raise ValueError(f"Commande inconnue : {cmd}")
        return handler(*args)

    def do_start(self): ...
    def do_stop(self): ...
    def do_status(self): ...
    # Ajouter une commande = ajouter UNE méthode
```

Le préfixe `do_` sert de **liste blanche** : impossible d'invoquer `run` ou `__init__` par injection.

callable() et le protocole d'appel

```
callable(print)      # True - fonction
callable(int)        # True - une classe s'appelle (→ instance)
callable(42)         # False
callable(lambda: 1)  # True
```

Tout objet définissant `__call__` est callable — y compris **vos instances** :

```
class Multiplier:
    """Une 'fonction' avec état et configuration."""
    def __init__(self, factor):
        self.factor = factor
        self.calls = 0

    def __call__(self, x):
        self.calls += 1
        return x * self.factor

double = Multiplier(2)
double(21)      # 42 - l'instance s'utilise comme une fonction
double.calls    # 1 - mais elle garde un état !
```

Cas d'usage : décorateurs avec état (chap. 3), stratégies configurables, callbacks instrumentés. C'est la passerelle entre POO et programmation fonctionnelle.

Le module `inspect` : la boîte à outils officielle

```
import inspect

def transfer(src: str, dst: str, amount: float = 0.0, *, dry_run: bool = False) -> bool:
    """Transfère un montant entre deux comptes."""
    ...
```

```
sig = inspect.signature(transfer)
str(sig)          # "(src: str, dst: str, amount: float = 0.0, *, dry_run: bool = False) -> bool"

for name, param in sig.parameters.items():
    print(name, param.kind, param.default, param.annotation)
# src      POSITIONAL_OR_KEYWORD  <empty>    <class 'str'>
# amount   POSITIONAL_OR_KEYWORD  0.0       <class 'float'>
# dry_run  KEYWORD_ONLY          False     <class 'bool'>

sig.return_annotation # <class 'bool'>
```

```
# Valider des arguments AVANT d'appeler (ce que fait FastAPI !)
bound = sig.bind("livret", "courant", amount=100.0)
bound.apply_defaults()
bound.arguments      # {'src': 'livret', 'dst': 'courant', 'amount': 100.0, 'dry_run': False}
```

`sig.bind(...)` lève `TypeError` si les arguments ne correspondent pas — **sans exécuter** la fonction.

inspect : sources, membres, pile d'appels

Lire le code source

```
import inspect, json

inspect.getsource(json.dumps) # le code de la fonction !
inspect.getsourcefile(json)  # /usr/lib/python3.12/json/__init__.py
inspect.getdoc(json.dumps)   # docstring nettoyée
inspect.getcomments(my_func) # commentaires juste avant
```

⚠ Indisponible pour le code C (builtins) : `TypeError` .

Lister les membres avec filtre

```
inspect.getmembers(obj, inspect.ismethod)
inspect.getmembers(module, inspect.isclass)
inspect.getmembers(module, inspect.isfunction)
```

Prédicats : `isclass` , `isfunction` , `ismethod` , `isgenerator` ,
`iscoroutine` , `isbuiltin` ...

```
# Découvrir tous les tests d'un module (comme pytest)
tests = [f for name, f in inspect.getmembers(mod, inspect.isfunction)
         if name.startswith("test_")]
```

Inspecter la pile d'appels

```
def who_called_me():
    frame = inspect.currentframe().f_back
    info = inspect.getframeinfo(frame)
    return f"{info.filename}:{info.lineno} dans {info.function}"

def business_code():
    log(f"appelé depuis {who_called_me()}")
```

Usage : loggers enrichis, assertions de debug, profilers maison (chap. 11).

⚠ Coûteux — jamais dans une boucle chaude.

Attributs spéciaux des fonctions

```
f.__name__      # nom
f.__module__    # module d'origine
f.__doc__       # docstring
f.__defaults__  # valeurs par défaut
f.__annotations__ # annotations brutes
f.__code__.co_varnames # variables locales
f.__code__.co_argcount # nb d'arguments
```

Ces attributs sont ce que `functools.wraps` **préserve** sur un décorateur (chap. 3).

Exploiter les annotations à l'exécution 1/2

Les annotations de type ne sont **pas vérifiées** par Python... mais elles sont **stockées** et exploitables :

```
from typing import get_type_hints

class Config:
    host: str
    port: int
    debug: bool

get_type_hints(Config)
# {'host': <class 'str'>, 'port': <class 'int'>, 'debug': <class 'bool'>}
```

Exploiter les annotations à l'exécution 2/2

Outil dynamique : un validateur générique basé sur les annotations

```
def validate(obj) -> list[str]:
    """Vérifie que les attributs respectent les annotations de la classe."""
    errors = []
    for attr, expected in get_type_hints(type(obj)).items():
        value = getattr(obj, attr, None)
        if not isinstance(value, expected):
            errors.append(f"{attr}: attendu {expected.__name__}, "
                          f"reçu {type(value).__name__}")
    return errors
```

Une seule fonction valide *toutes* vos classes. C'est le principe de **Pydantic** et des `dataclasses`.

💡 Préférez `get_type_hints()` à `__annotations__` brut : il résout les annotations en chaînes (`"User"`).

Créer des classes à la volée : `type()` à 3 arguments 1/2

`type` a une double personnalité : interrogateur... et fabrique de classes.

```
# Ces deux écritures sont STRICTEMENT équivalentes :  
  
class Point:  
    dim = 2  
    def norm(self):  
        return (self.x**2  
                + self.y**2) ** 0.5  
  
Point = type(  
    "Point",          # __name__  
    (object,),        # bases  
    {                 # espace de noms  
        "dim": 2,  
        "norm": lambda self: (self.x**2 + self.y**2) ** 0.5,  
    },  
)
```

Créer des classes à la volée : `type()` à 3 arguments 2/2

L'instruction `class` n'est que du **sucre syntaxique** autour de `type(name, bases, namespace)`.

```
# Cas d'usage : générer des classes d'exception par domaine
def make_errors(domain: str, names: list[str]) -> dict[str, type]:
    base = type(f"{domain}Error", (Exception,), {})
    return {n: type(n, (base,), {}) for n in names}

errors = make_errors("Payment", ["CardDeclined", "InsufficientFunds"])
raise errors["CardDeclined"]("carte refusée")
```

C'est la porte d'entrée des **métaclasses** (chapitre 8) : si `type` crée les classes... on peut le sous-classer.

Import dynamique : `importlib` 1/2

Problème : charger un module dont le nom n'est connu qu'à l'exécution (config, plugin, driver...).

```
import importlib

# ❌ Surtout pas : exec(f"import {name}")
# ✅ L'outil officiel :
module = importlib.import_module("json")
module = importlib.import_module(f"drivers.{driver_name}")

# Recharger un module modifié (dev interactif, hot-reload)
importlib.reload(module)
```


Import dynamique : `importlib` 2/2

Architecture à plugins — le pattern complet

```
import importlib, pkgutil
import plugins # un package ordinaire

def load_plugins() -> dict[str, "module"]:
    """Découvre et charge tous les modules du package plugins/."""
    registry = {}
    for info in pkgutil.iter_modules(plugins.__path__):
        mod = importlib.import_module(f"plugins.{info.name}")
        if hasattr(mod, "register"): # contrat du plugin
            registry[info.name] = mod.register()
    return registry
```

Déposer un fichier dans `plugins/` suffit à étendre l'application — **zéro modification** du cœur.

`eval()` **et** `exec()` : puissance et danger 1/2

Ce qu'ils font

```
eval("2 + 3 * 4")      # 14 - UNE expression
exec("x = 10\nprint(x)") # exécute des INSTRUCTIONS

# Espaces de noms contrôlables
env = {}
exec("y = 2 ** 8", {}, env)
env["y"]                # 256
```

`globals()` / `locals()` donnent accès aux espaces de noms courants.

Usage légitime (rare)

La stdlib elle-même : `namedtuple` et `dataclasses` génèrent le code de `__init__` puis l'`exec` utent — c'est pour ça qu'ils sont aussi rapides qu'un code écrit à la main.

`eval()` **et** `exec()` : puissance et danger 2/2

Le danger : l'injection de code

```
user_input = "__import__('os').system('rm -rf /')"  
eval(user_input) # ⚠️ exécution arbitraire
```

Jamais d'`eval` / `exec` sur une donnée externe (formulaire, fichier, réseau, BDD) — même « filtrée ».

Les alternatives sûres

```
import ast  
ast.literal_eval("[1, 2, {'a': 3}])"  
# littéraux Python uniquement – refuse tout appel  
  
int("42"), float("3.14") # conversions  
json.loads(payload) # données structurées  
getattr(obj, name) # accès dynamique  
operator_map = {"+": operator.add, ...} # mini-DSL
```

Règle d'or : si vous pensez avoir besoin d'`eval`, il existe presque toujours une conception plus sûre.

Monkey patching : modifier le code des autres

Remplacer à chaud un attribut/une fonction d'un module ou d'une classe **existante** :

```
import payment_gateway

def fake_charge(card, amount):
    print(f"[TEST] charge simulée : {amount} €")
    return {"status": "ok", "id": "fake-123"}

payment_gateway.charge = fake_charge      # patché pour tout le process !
```

Usages légitimes

- **Tests** : isoler le code des dépendances (réseau, BDD, horloge)
- Hotfix temporaire d'une lib tierce boguée
- Instrumentation / mesure non intrusive

Les risques

- Effet **global** et invisible au lecteur
- Fragile aux mises à jour de la lib
- Ordre d'import significatif → bugs erratiques

Règle : en production → ultime recours documenté ; en test → `mock.patch` , jamais d'affectation brute.

✓ En test : `unittest.mock.patch`

Patch **délimité** et auto-nettoyé :

```
from unittest.mock import patch

def test_payment():
    with patch("payment_gateway.charge") as mock:
        mock.return_value = {"status": "ok"}
        result = process_order(order)
        mock.assert_called_once_with(
            order.card, 49.90)
```

Le patch disparaît à la sortie du `with` — même en cas d'exception (context manager, chap. 9).

Intercepter les accès : `__getattr__` 1/2

`__getattr__` n'est appelé **que si** la recherche normale échoue — le filet de sécurité idéal :

```
class APIClient:
    """client.get_users() → GET /users, client.get_orders() → GET /orders..."""
    def __init__(self, base_url):
        self.base_url = base_url

    def __getattr__(self, name):
        # uniquement si attribut introuvable
        if name.startswith("get_"):
            endpoint = name.removeprefix("get_")
            def caller(**params):
                return http_get(f"{self.base_url}/{endpoint}", params)
            return caller
        raise AttributeError(
            f"type(self).__name__!r} n'a pas d'attribut {name!r}"
        )

api = APIClient("https://api.example.com")
api.get_users(active=True)  # méthode qui n'existe... nulle part !
```

Intercepter les accès : `__getattr__` 2/2

	Appelé quand ?	Risque
<code>__getattr__</code>	attribut introuvable seulement	faible — le cas nominal reste normal
<code>__getattribute__</code>	chaque accès, sans exception	réursion infinie, lenteur — chap. 8

Pattern des proxies, lazy-loading, wrappers d'API (boto3 fonctionne ainsi).

Fil rouge : un mini-framework de sérialisation

Assemblons les briques du chapitre — introspection + annotations + accès dynamique :

```
from typing import get_type_hints
import json

class Serializable:
    """Toute classe héritant de Serializable gagne to_json / from_json."""

    def to_dict(self) -> dict:
        return {attr: getattr(self, attr)          # lecture dynamique
                for attr in get_type_hints(type(self))} # pilotée par les annotations

    def to_json(self) -> str:
        return json.dumps(self.to_dict())

    @classmethod
    def from_dict(cls, data: dict):
        hints = get_type_hints(cls)
        obj = cls.__new__(cls) # instance sans __init__
        for attr, expected in hints.items():
            if attr not in data:
                raise ValueError(f"champ manquant : {attr}")
            value = data[attr]
            if not isinstance(value, expected): # validation par annotation
                raise TypeError(f"{attr}: attendu {expected.__name__}, reçu {type(value).__name__}")
            setattr(obj, attr, value) # écriture dynamique
        return obj

class User(Serializable):
    name: str
    age: int

u = User.from_dict({"name": "Alice", "age": 30}) # ✅ validé
u.to_json() # '{"name": "Alice", "age": 30}'
User.from_dict({"name": "Bob", "age": "trente"}) # ❌ TypeError: age: attendu int, reçu str
```

~25 lignes : c'est le cœur de ce que font **Pydantic**, **marshmallow**, **attrs**. La version industrielle ajoute : types imbriqués, conversions, messages d'erreur riches... mais le **principe** est celui-ci.

Bonnes pratiques — synthèse du chapitre

Quand utiliser la métaprogrammation :

-  Éliminer de la **répétition structurelle** (boilerplate) qu'aucune fonction ne peut factoriser
-  Construire des **frameworks** / outils internes : plugins, validation, sérialisation, dispatch
-  Jamais pour « faire malin » — *Readability counts* : si une boucle `if/elif` de 3 cas suffit, gardez-la

Réflexes de sécurité :

- `eval` / `exec` sur données externes : **jamais** → `ast.literal_eval`, `json`, dispatch par dict
- Dispatch par `getattr` : toujours une **liste blanche** (préfixe `do_`, registre explicite)
- `__getattr__` doit lever `AttributeError` — sinon `hasattr` et `copy` se dérèglent
- Monkey patching : en test via `mock.patch`, en production presque jamais

Outillage à retenir :

Besoin	Outil
Type, héritage	<code>isinstance</code> , <code>type</code> , <code>__mro__</code>

Travaux pratiques

TP 2.1 — Exploration de l'introspection (30 min)

1. Écrivez `explore(obj)` qui affiche un rapport : type, module, docstring, attributs publics (séparés données / méthodes via `inspect`), signature si callable, MRO si c'est une classe
2. Testez-la sur : un `int`, une fonction de `json`, la classe `collections.Counter`, une instance de vos classes du chapitre 1
3. Bonus : faites-lui détecter les générateurs et coroutines (`inspect.isgeneratorfunction`, `inspect.iscoroutinefunction`)

TP 2.2 — Création d'outils dynamiques (45 min)

1. **Dispatcher CLI** : une classe `Shell` qui route `shell.run("user add alice")` vers `cmd_user_add(self, name)` par convention de nommage — avec messages d'erreur listant les commandes disponibles (découvertes par introspection)
2. **Architecture à plugins** : un package `plugins/` dont chaque module expose `register()` ; le cœur les découvre avec `pkgutil` + `importlib` et les charge sans aucun import en dur
3. Bonus : un client REST « magique » à base de `__getattr__` (`api.get_users()`, `api.get_products(limit=10)`)

TP 2.3 — Tests et validation (45 min)

1. Implémentez `@validate_call` : un décorateur qui vérifie à l'appel que les arguments respectent les annotations de la fonction (utilisez `inspect.signature` et `sig.bind`) — il vous servira de transition vers le chapitre 3
2. Étendez le mini-framework `Serializable` du cours : gérez `list[int]` et les objets imbriqués (un champ annoté avec une classe elle-même `Serializable`)
3. Écrivez les tests pytest de votre validateur : cas nominal, champ manquant, mauvais type — et utilisez `mock.patch` pour tester le client REST du TP 2.2 sans réseau

Livrables : code + tests pytest verts. Gardez le décorateur du TP 2.3 — nous le réutiliserons au chapitre 3.

Chapitre 3

Programmation fonctionnelle avancée

Python Perfectionnement — Jour 1

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Exploiter** les fonctions comme objets de première classe et les **closures**
- **Maîtriser** les décorateurs : simples, paramétrés, avec état, empilés
- **Utiliser** les décorateurs de la stdlib : `lru_cache`, `cached_property`, `singledispatch`
- **Implémenter** des patterns fonctionnels : composition, application partielle, pipelines, registres
- **Découvrir** la programmation réactive avec RxPY : observables et opérateurs

Travaux pratiques : maîtrise des décorateurs, implémentation de patterns fonctionnels, programmation réactive.

Les fondements (pureté, `map` / `filter` / `reduce`, `itertools`) seront consolidés au **chapitre 4** ; les générateurs au **chapitre 5**.

Les fonctions, citoyennes de première classe

En Python, une fonction est un **objet comme un autre** : on peut l'affecter, la passer, la retourner, la stocker.

```
def shout(text):
    return text.upper() + " !"

speak = shout                # affectation — pas d'appel (pas de parenthèses)
speak("bonjour")             # 'BONJOUR !'

actions = {"crier": shout, "chuchoter": str.lower}    # stockage
actions["crier"]("hé")      # 'HÉ !'

def apply_twice(func, value): # fonction en ARGUMENT
    return func(func(value))

def make_greeter(greeting):   # fonction en RETOUR
    def greeter(name):
        return f"{greeting}, {name} !"
    return greeter

hello = make_greeter("Bonjour")
hello("Alice")                 # 'Bonjour, Alice !'
```

Ces quatre capacités — affecter, passer, retourner, stocker — sont le **socle** de tout ce chapitre.

Les closures : des fonctions avec mémoire

Une **closure** est une fonction qui **capture** des variables de sa portée englobante — elles survivent à la fin de l'appel parent :

```
def make_counter():
    count = 0                # variable "capturée"
    def counter():
        nonlocal count      # nécessaire pour MODIFIER la capture
        count += 1
        return count
    return counter

c1 = make_counter()
c2 = make_counter()        # chaque closure a SA propre cellule
c1(), c1(), c1()           # 1, 2, 3
c2()                      # 1 - indépendant !
```

Sous le capot

```
c1.__closure__
# (<cell at 0x...: int object at ...>,)
c1.__closure__[0].cell_contents    # 3
c1.__code__.co_freevars            # ('count',)
```

La capture est **par référence** (cellule), pas par copie.

Règle LEGB (rappel)

Résolution des noms : Local → Enclosing → Global → Builtins

- lire une variable englobante : rien à faire
- la **réaffecter** : `nonlocal`
- réaffecter une globale : `global` (à éviter)

Le piège des closures : late binding

La closure capture la **variable**, pas sa **valeur** au moment de la définition :

✗ Le bug classique

```
multipliers = []
for i in range(3):
    multipliers.append(lambda x: x * i)

[m(10) for m in multipliers]
# [20, 20, 20] 🐞 attendu : [0, 10, 20]
```

Les 3 lambdas partagent la **même** variable `i`, lue à l'appel — elle vaut alors `2`.

Même piège avec les callbacks d'UI, les threads, les tâches asyncio créées en boucle.

À retenir : une portée n'est créée que par un **appel de fonction** — pas par une boucle.

✓ Les corrections

```
# 1. Argument par défaut : évalué À LA DÉFINITION
for i in range(3):
    multipliers.append(lambda x, i=i: x * i)

# 2. Fabrique : un appel = une portée = une cellule
def make_mult(factor):
    return lambda x: x * factor
multipliers = [make_mult(i) for i in range(3)]

# 3. functools.partial (vu plus loin)
from functools import partial
from operator import mul
multipliers = [partial(mul, i) for i in range(3)]
```

Décorateurs : le principe

Un décorateur est une fonction qui **prend une fonction et en retourne une autre**. Le `@` n'est que du sucre :

```
@timer
def compute():
    ...

⇨

def compute():
    ...
    compute = timer(compute)
```

```
import time, functools

def timer(func):
    @functools.wraps(func)           # ← préserve nom, doc, signature
    def wrapper(*args, **kwargs):    # accepte TOUT type d'arguments
        start = time.perf_counter()
        result = func(*args, **kwargs) # appel de la fonction décorée
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} : {elapsed:.4f}s")
        return result                # ⚠ ne pas oublier le return !
    return wrapper

@timer
def slow_sum(n):
    return sum(range(n))

slow_sum(10_000_000)                # slow_sum : 0.2153s → 49999995000000
```

L'anatomie à mémoriser : **capture** (`func`), **wrapper** (`*args, **kwargs`), **appel + return**, `@wraps` .

`functools.wraps` : pourquoi c'est obligatoire

Sans `@wraps`, la fonction décorée perd son identité :

❌ Sans `@wraps`

```
def timer(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@timer
def fetch_data():
    """Charge les données."""

fetch_data.__name__    # 'wrapper' 🚫
fetch_data.__doc__     # None
help(fetch_data)       # inutilisable
```

Conséquences réelles : logs illisibles, `pickle` cassé, doc Sphinx vide, debugging pénible, introspection (chap. 2) faussée.

✅ Avec `@wraps`

```
import functools

def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@timer
def fetch_data():
    """Charge les données."""

fetch_data.__name__    # 'fetch_data' ✅
fetch_data.__doc__     # 'Charge les données.'
fetch_data.__wrapped__ # accès à l'originale !
```

`inspect.signature` suit `__wrapped__` : votre `@validate_call` du TP 2.3 fonctionne même sur des fonctions décorées.

Décorateurs paramétrés : trois niveaux

Pour écrire `@retry(attempts=3)`, il faut une **fabrique de décorateurs** — une fonction de plus :

```
import functools, time

def retry(attempts=3, delay=1.0, exceptions=(Exception,)):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, attempts + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as exc:
                    if attempt == attempts:
                        raise
                    print(f"{func.__name__} échec {attempt}/{attempts} : {exc}")
                    time.sleep(delay * attempt)
            return wrapper
        return decorator

    @retry(attempts=5, delay=0.5, exceptions=(ConnectionError, TimeoutError))
    def fetch(url):
        ...
```

Lecture mentale : `@retry(...)` s'évalue d'abord → renvoie `decorator` → qui décore `fetch`.

Chaque niveau capture le précédent par closure : params → fonction → arguments.

Décorateurs avec état : la classe décorateur

Quand le wrapper doit **mémoriser** des choses entre les appels, une classe avec `__call__` (chap. 2) est souvent plus lisible qu'une closure :

```
import functools

class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)    # équivalent de @wraps pour les classes
        self.func = func
        self.count = 0

    def __call__(self, *args, **kwargs):
        self.count += 1
        return self.func(*args, **kwargs)

@CountCalls
def process(item):
    ...

process("a"); process("b")
process.count    # 2 - l'état est INSPECTABLE de l'extérieur ✓
```

Closure : concis, idiomatique, état privé

→ décorateurs simples et sans réglage externe

Classe : état exposé, méthodes auxiliaires (`reset()` ...), héritage possible

→ instrumentation, rate-limiting, circuit breaker

Empiler les décorateurs : l'ordre compte

```
@timer                # appliqué EN DERNIER (extérieur)
@retry(attempts=3)     # appliqué en 2e
@validate_call         # appliqué EN PREMIER (intérieur, le plus proche de func)
def sync_inventory(source: str, dry_run: bool = False):
    ...

# Équivalent : sync_inventory = timer(retry(attempts=3)(validate_call(sync_inventory)))
```

Se lit de bas en haut (application) — s'exécute de haut en bas (appel) :

```
appel → timer ➡ retry ➡ validate_call ➡ sync_inventory
retour ➡ timer ➡ retry ➡ validate_call ➡ résultat
```

L'ordre change la sémantique :

Empilement	Effet
@timer puis @retry	chronomètre tous les essais cumulés
@retry puis @timer	chronomètre chaque essai individuellement
@lru_cache au-dessus de @retry	met en cache le résultat final (après retries) ✓
@retry au-dessus de @lru_cache	re-tente... un cache qui ne peut pas échouer 🤔

Les décorateurs de la stdlib : `functools.lru_cache`

La **mémoïsation** industrialisée — cache les résultats par arguments :

```
import functools

@functools.lru_cache(maxsize=256)      # LRU : évince les moins récemment utilisés
def fib(n):
    return n if n < 2 else fib(n - 1) + fib(n - 2)

fib(35)      # instantané – sans cache : ~5 s (récursion exponentielle)
fib.cache_info() # CacheInfo(hits=33, misses=36, maxsize=256, cursize=36)
fib.cache_clear() # vider (tests !)

@functools.cache      # 3.9+ : cache illimité, plus rapide
def load_config(env: str) -> dict: ...
```

Conditions d'usage

- Fonction **pure** (même entrée → même sortie)
- Arguments **hachables** (chap. 1 — pas de `list / dict` !)
- Attention aux **méthodes** : le cache retient `self` → fuite mémoire (utiliser `cached_property` ou un cache externe)

`functools.cached_property`

```
class Dataset:
    def __init__(self, path):
        self.path = path

    @functools.cached_property
    def stats(self):
        return expensive(self.path) # calculé au 1er accès,
                                     # puis stocké sur
                                     # l'INSTANCE

ds.stats      # calcul
ds.stats      # lecture directe
del ds.stats  # invalide le cache
```

functools.singledispatch : le polymorphisme fonctionnel

Une fonction générique qui **dispatche** sur le type du premier argument — l'alternative propre aux cascades `isinstance` :

```
from functools import singledispatch

@singledispatch
def render(value) -> str:
    return str(value) # implémentation par DÉFAUT

@render.register
def _(value: list) -> str:
    return "<ul>" + "".join(f"<li>{render(v)}</li>" for v in value) + "</ul>" # dispatch via l'ANNOTATION

@render.register
def _(value: dict) -> str:
    return "<dl>" + "".join(f"<dt>{k}</dt><dd>{render(v)}</dd>" for k, v in value.items()) + "</dl>"

@render.register
def _(value: None) -> str:
    return "<em>vide</em>" # même None est gérable

render([1, {"a": 2}]) # '<ul><li>1</li><li><dl><dt>a</dt><dd>2</dd></dl></li></ul>'
```

-  **Ouvert à l'extension** : un module tiers peut `register` ses propres types sans toucher au code existant
- Le dispatch respecte l'**héritage** (MRO) ; `singledispatchmethod` existe pour les méthodes
- Pattern utilisé par `pprint`, les sérialiseurs, les moteurs de rendu

functools.partial : l'application partielle

Figurer certains arguments d'une fonction pour en créer une version spécialisée :

```
from functools import partial

def log(level: str, component: str, message: str):
    print(f"[{level}] {component}: {message}")

log_db_error = partial(log, "ERROR", "database")    # 2 arguments figés
log_db_error("connexion perdue")                  # [ERROR] database: connexion perdue

# Figurer par mot-clé
int_base2 = partial(int, base=2)
int_base2("1010")                                # 10

# partial est INSPECTABLE (contrairement à une lambda)
log_db_error.func, log_db_error.args              # (<function log>, ('ERROR', 'database'))
```

Là où partial brille : adapter des signatures

```
# Un callback qui n'accepte qu'un argument ? On adapte :
button.on_click(partial(save_document, doc, format="pdf"))

# Avec map / executors (chap. 12) :
results = executor.map(partial(resize, width=800), images)

# Corriger le late binding des boucles (vu plus tôt) :
handlers = [partial(handle, event_id) for event_id in events]
```

`partialmethod` : la même chose pour définir des méthodes dérivées dans une classe.

Pattern : composition de fonctions

Construire des traitements complexes en **assemblant** des fonctions simples :

```
import functools

def compose(*funcs):
    """compose(f, g, h)(x) == f(g(h(x))) - comme en math, de droite à gauche."""
    return functools.reduce(lambda f, g: lambda x: f(g(x)), funcs)

def pipe(*funcs):
    """pipe(f, g, h)(x) == h(g(f(x))) - ordre de LECTURE, plus naturel."""
    def piped(value):
        for func in funcs:
            value = func(value)
        return value
    return piped
```

```
# Pipeline de nettoyage de texte - chaque étape est PURE et TESTABLE isolément
clean = pipe(
    str.strip,
    str.lower,
    partial(re.sub, r"\s+", " "),
    partial(re.sub, r"[\^w ]", ""),
)

clean("  Bonjour,   le MONDE !  ")    # 'bonjour le monde'
```

✓ **Clean code** : des petites fonctions à responsabilité unique + une composition **déclarative** qui se lit comme une recette.

Ajouter/retirer/réordonner une étape = une ligne.

Pattern : le registre par décorateur

Le décorateur comme **mécanisme d'enregistrement** — il ne modifie pas la fonction, il la **référence** :

```
HANDLERS: dict[str, callable] = {}

def handles(event_type: str):
    def decorator(func):
        HANDLERS[event_type] = func      # enregistrement au moment de l'import
        return func                     # ← fonction rendue INTACTE
    return decorator

@handles("order.created")
def send_confirmation(event): ...

@handles("order.cancelled")
def refund_payment(event): ...

def dispatch(event):
    handler = HANDLERS.get(event["type"])
    if handler is None:
        raise ValueError(f"Aucun handler pour {event['type']}!")
    return handler(event)
```

- C'est **exactement** le mécanisme de `@app.route("/users")` (Flask), `@pytest.fixture`, `@celery.task`
- Combiné à la découverte de plugins du chapitre 2 (`pkgutil` + `importlib`) → architecture extensible complète
- Variante : registre de **stratégies** métier (`@pricing_rule("black_friday")`)

Programmation réactive : le concept

Changer de paradigme : au lieu de **demander** les données (*pull*), on **réagit** à des flux d'événements (*push*).

Pull — itération classique

```
for item in collection:
    process(item)
```

- Le consommateur **contrôle** le rythme
- Données finies, disponibles
- chap. 5 : les générateurs

Push — flux réactif

```
stream.subscribe(process)
```

- Le **producteur** émet quand il veut
- Flux potentiellement infinis : clics, capteurs, cours de bourse, logs, websockets
- Le consommateur **déclare** sa réaction

Les trois acteurs (pattern Observer généralisé — chap. 7)

Acteur	Rôle	Méthodes
Observable	la source qui émet	<code>subscribe()</code>
Observer	le consommateur	<code>on_next()</code> , <code>on_error()</code> , <code>on_completed()</code>
Opérateurs	transforment le flux entre les deux	<code>map</code> , <code>filter</code> , <code>debounce</code> , <code>merge</code> ...

ReactiveX (Rx) : standard multi-langages — RxJS, RxJava... et **RxPY** (`pip install reactivex`).

RxPY : premiers pas

```
import reactivex as rx
from reactivex import operators as ops

# Une source qui émet 1, 2, 3, 4, 5 puis se termine
source = rx.of(1, 2, 3, 4, 5)

source.pipe(
    ops.map(lambda x: x * 10),      # le pipeline d'opérateurs
    ops.filter(lambda x: x > 15),  #   transforme : 10, 20, 30, 40, 50
    ops.filter(lambda x: x > 15),  #   filtre      : 20, 30, 40, 50
).subscribe(
    on_next=lambda v: print(f"reçu : {v}"),
    on_error=lambda e: print(f"erreur : {e}"),
    on_completed=lambda: print("terminé"),
)
# reçu : 20 ... reçu : 50
# terminé
```

Lecture : `pipe()` est la composition de fonctions vue plus tôt — appliquée à des **flux**. Rien ne s'exécute avant `subscribe()` (évaluation paresseuse, comme les générateurs du chap. 5).

```
# D'autres façons de créer des observables
rx.from_iterable(open("app.log"))      # depuis tout itérable
rx.interval(1.0)                       # tick toutes les secondes
rx.from_callable(fetch_price)         # depuis un appel
```

RxPY : les opérateurs qui changent la donne

Là où le réactif surpasse la boucle : les opérateurs **temporels** et de **combinaison** :

```
import reactivex as rx
from reactivex import operators as ops

# Recherche "instantanée" : la barre de recherche idéale
search_input.pipe(
    ops.debounce(0.3),          # attend 300 ms de silence (l'utilisateur a fini de taper)
    ops.distinct_until_changed(), # ignore si la valeur n'a pas changé
    ops.flat_map(lambda q: rx.from_callable(lambda: search_api(q))),
).subscribe(display_results)
```

```
# Supervision de capteurs : moyenne glissante + alerte
sensor_stream.pipe(
    ops.buffer_with_time(5.0), # paquets de 5 secondes
    ops.map(lambda window: sum(window) / len(window)), # moyenne de la fenêtre
    ops.filter(lambda avg: avg > THRESHOLD),
).subscribe(trigger_alert)
```

Famille	Opérateurs	Usage
Transformation	map , flat_map , scan	transformer, aplatir, accumuler
Filtrage	filter , take , skip , distinct	sélectionner
Temporels	debounce , throttle , delay , buffer_with_time	dompter le temps
Combinaison	merge , zip , combine_latest	fusionner des flux

Réactif : quand (et quand ne pas) l'utiliser

✓ Bons cas d'usage

- Événements asynchrones multiples : UI, websockets, MQTT/IoT
- Logique temporelle : debounce, fenêtres glissantes, timeout
- Combinaison de sources : fusionner capteurs, flux marchés
- Chaînes de transformation **déclaratives** sur des flux infinis

Le code exprime le **quoi**, la bibliothèque gère le **quand**.

Alternatives dans l'écosystème : `asyncio` + files (chap. 12) couvre beaucoup de cas ; les *async generators* (chap. 5) offrent un style pull sur flux asynchrones ; Kafka/RabbitMQ pour le réactif **distribué** (chap. 13).

✗ Mauvais cas d'usage

- Traitement **batch** d'une collection finie → compréhensions, générateurs (chap. 5)
- Une seule source, pas de logique temporelle → une simple boucle suffit
- Équipe non formée : la **dette de lisibilité** est réelle (debugging des pipelines, stacktraces...)

Simple is better than complex — le réactif est un outil de niche, puissant au bon endroit.

Bonnes pratiques — synthèse du chapitre

Décorateurs :

- Toujours `@functools.wraps` (ou `update_wrapper`) — non négociable
- Wrapper en `*args, **kwargs` + `return` du résultat — les deux oublis classiques
- Décorateur paramétré = 3 niveaux ; mémorisez le squelette `fabrique` → `decorator` → `wrapper`
- L'**ordre d'empilement** est sémantique : documentez-le quand il compte
- Un décorateur doit faire **une chose** (timing OU retry OU validation) — composez-les

Patterns fonctionnels :

- Petites fonctions **pures** + composition déclarative = testabilité maximale
- `partial` plutôt que `lambda` pour figer des arguments (inspectable, picklable)
- `singledispatch` plutôt que cascades `isinstance` ; registre par décorateur plutôt que `if/elif`
- Cache : `lru_cache` exige pureté + arguments hachables — vérifiez les deux

Réactif :

- Réservez RxPY aux flux **événementiels et temporels** ; pour le batch, restez sur les outils natifs
- Rien ne part avant `subscribe()` — pensez « plomberie déclarée, puis ouverture du robinet »

Travaux pratiques 1/2

TP 3.1 — Maîtrise des décorateurs (45 min)

1. `@timer` enrichi : paramétrable (`@timer(unit="ms", threshold=0.1)`) — ne logue que si la durée dépasse le seuil
2. `@retry(attempts, delay, exceptions)` avec backoff exponentiel — testez-le sur une fonction qui échoue aléatoirement (`random.random() < 0.7`)
3. `@rate_limit(calls=5, period=60)` en classe décorateur : conserve l'horodatage des appels (`deque(maxlen=calls)` du chap. 1 !) et lève `RateLimitError` en cas d'excès
4. Reprenez votre `@validate_call` du TP 2.3 : vérifiez avec `inspect` qu'il préserve bien signature et docstring, corrigez si besoin
5. Empilez les quatre sur une même fonction et **justifiez l'ordre** choisi en commentaire

TP 3.2 — Patterns fonctionnels (45 min)

1. Implémentez `pipe(*funcs)` et construisez un pipeline de nettoyage de données : lignes CSV brutes → strip → normalisation des décimaux (`"1234,56"` → `1234.56`) → filtrage des lignes invalides
2. Moteur de tarification par `registre` : `@pricing_rule("standard")`, `@pricing_rule("premium")` ... et une fonction `compute_price(customer_type, amount)` qui dispatche — ajoutez une règle sans modifier le moteur
3. `render()` multi-formats avec `singledispatch` : `str`, `int` / `float`, `list`, `dict`, `datetime`, `None` → sortie texte alignée
4. Mesurez avec `timeit` (chap. 1) l'effet de `@lru_cache` sur une fonction récursive (`fib`) puis sur un appel d'I/O simulé (`time.sleep(0.1)`)

Travaux pratiques 2/2

TP 3.3 — Programmation réactive (45 min)

1. `pip install reactivex` — flux de température simulé (`rx.interval(0.5)` + bruit aléatoire) : affichez la moyenne glissante sur 10 valeurs (`buffer_with_count` ou `scan`)
2. Ajoutez une **alerte** si 3 mesures consécutives dépassent un seuil (`buffer_with_count(3, 1)` + `filter`)
3. Fusionnez deux capteurs avec `merge` puis comparez-les avec `combine_latest`
4. **Discussion** : réécrivez l'exercice 1 avec une simple boucle `while` + `deque(maxlen=10)` . Quelle version préférez-vous, et pourquoi ?

Livrables : code + tests pytest pour les TP 3.1 et 3.2 (testez vos décorateurs avec `mock.patch` sur `time.sleep` !).

Chapitre 4

Programmation fonctionnelle

Python Perfectionnement — Jour 2

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Appliquer** les principes fonctionnels : pureté, immutabilité, transparence référentielle
- **Identifier et isoler** les effets de bord (*functional core, imperative shell*)
- **Utiliser** à bon escient `lambda`, `map`, `filter`, `reduce` — et savoir quand préférer les compréhensions
- **Exploiter** les modules `operator` et `itertools`
- **Consolider** l'écriture de décorateurs (acquis du chapitre 3)
- **Implémenter** des patterns fonctionnels de traitement de données

Travaux pratiques : création de décorateurs, implémentation de patterns.

Ce chapitre consolide les **fondements** du paradigme ; le chapitre 3 en a montré les techniques avancées.

Le paradigme fonctionnel : trois piliers

Pilier	Principe	En Python
Fonctions pures	même entrée → même sortie, sans effet de bord	discipline (non imposé)
Immutabilité	on ne modifie pas, on crée de nouvelles valeurs	tuple , frozenset , frozen=True
Composition	des petites fonctions assemblées	pipe , décorateurs, générateurs

Python est **multi-paradigme** : il n'impose pas le fonctionnel (contrairement à Haskell), il le **permet**.

```
# Style impératif : COMMENT faire, pas à pas
total = 0
for order in orders:
    if order.status == "paid":
        total += order.amount

# Style fonctionnel : QUOI calculer, déclaratif
total = sum(o.amount for o in orders if o.status == "paid")
```

L'objectif n'est pas le dogme — c'est un code plus **prévisible**, plus **testable**, plus **parallélisable** (chap. 12).

Fonctions pures : la définition

Une fonction est **pure** si :

1. Sa sortie ne dépend **que** de ses arguments (pas d'état global, d'horloge, de fichier...)
2. Elle ne produit **aucun effet de bord** (pas de mutation, d'I/O, de print...)

✗ Impure

```
discount_rate = 0.1          # état global  
  
def total(cart):  
    cart.append("audit")      # mutation de l'argument !  
    return sum(cart.prices) * (1 - discount_rate)
```

Résultat **imprévisible** : dépend de `discount_rate`, et l'appel **modifie**

`cart`.

✓ Pure

```
def total(prices: tuple[float, ...],  
          discount_rate: float) -> float:  
    return sum(prices) * (1 - discount_rate)
```

Tout est **explicite** dans la signature. Appeler 1000 fois → 1000 fois le même résultat.

Bénéfices immédiats : testable sans setup ni mock, cachable (`lru_cache`, chap. 3), parallélisable sans verrou (chap. 12), raisonnable par simple lecture.

Effets de bord : les identifier, les isoler 1/2

Un **effet de bord** = toute interaction avec l'extérieur de la fonction : I/O, mutation, horloge, aléatoire, réseau, globales...

Les effets de bord ne sont pas le mal — **un programme sans effet de bord ne fait rien**. La question est : **où** les placer.

Effets de bord : les identifier, les isoler 2/2

Le pattern *functional core, imperative shell*

```
# — CŒUR FONCTIONNEL : pur, testable, dense en logique —
def compute_payroll(employees: tuple, rates: dict) -> tuple:
    return tuple(
        (e.id, e.hours * rates[e.grade] * (1.5 if e.overtime else 1.0))
        for e in employees
    )

# — COQUILLE IMPÉRATIVE : fine, triviale, tous les effets de bord —
def main():
    employees = load_from_db()      # I/O
    rates = load_rates("rates.json") # I/O
    payroll = compute_payroll(employees, rates) # ← TOUTE la logique, pure
    save_to_db(payroll)            # I/O
    notify_accounting(payroll)     # I/O
```

Règle d'architecture : la logique métier au centre (pure), les I/O aux bords (fines).

C'est l'intuition derrière l'architecture hexagonale / clean architecture.

Lambda : l'usage juste 1/2

Une `lambda` est une fonction **anonyme** limitée à **une expression** :

```
key=lambda o: o.priority          # ✓ l'usage idéal : un argument key/callback  
sorted(orders, key=lambda o: (o.date, -o.amount)) # tri multi-critères
```

✓ Bon usage

- Argument `key=` de `sorted` / `min` / `max`
- Callback court et évident
- Jetable, lisible d'un coup d'œil

✗ Mauvais usage

```
# PEP 8 : jamais de lambda NOMMÉE  
f = lambda x: x + 1          # ✗  
def f(x): return x + 1       # ✓  
  
# Lambda illisible → vraie fonction  
lambda d: d["a"][0] if d.get("a") \  
    else d.get("b", [None])[0] # ✗
```

Lambda : l'usage juste 2/2

Alternatives souvent meilleures :

```
from operator import itemgetter, attrgetter
sorted(rows, key=itemgetter(2))      # plutôt que lambda r: r[2]
sorted(users, key=attrgetter("name")) # plutôt que lambda u: u.name
partial(round, ndigits=2)            # plutôt que lambda x: round(x, 2)
```

map et filter vs compréhensions

```
names = ["alice", "BOB", "Carol"]

# map / filter : renvoient des ITÉRATEURS paresseux (chap. 5)
map(str.title, names)      # <map object> – rien n'est calculé !
filter(str.isupper, names) # <filter object>

list(map(str.title, names)) # ['Alice', 'Bob', 'Carol']
```

Compréhension — le défaut idiomatique

```
[n.title() for n in names]
[n for n in names if n.isupper()]
[n.title() for n in names if len(n) > 3]
```

✓ Lisible, combine map+filter, expression arbitraire sans lambda

map / filter gardent l'avantage quand...

```
# La fonction EXISTE déjà (pas de lambda)
map(str.strip, lines)
map(int, tokens)
filter(None, items)    # retire les falsy

# Paresse voulue sur GROS volume
total = sum(map(len, huge_file))
```

Règle pratique : compréhension par défaut ; `map / filter` si la fonction existe déjà **et** que la paresse compte. ⚠ Un `map` non consommé ne fait rien — piège classique.

functools.reduce : plier une séquence

`reduce(f, seq, init)` replie la séquence en appliquant `f` cumulativement :

```
from functools import reduce
reduce(lambda acc, x: acc + x, [1, 2, 3, 4], 0) # (((0+1)+2)+3)+4 = 10
```

Mais Python fournit déjà les réductions courantes — ne les réinventez pas :

Besoin	 <code>reduce</code>	 Builtin
somme	<code>reduce(add, xs, 0)</code>	<code>sum(xs)</code>
produit	<code>reduce(mul, xs, 1)</code>	<code>math.prod(xs)</code>
min / max	<code>reduce(lambda a,b: ..., ...)</code>	<code>min(xs)</code> , <code>max(xs)</code>
concaténation	<code>reduce(add, strs)</code>  $O(n^2)$	<code>"".join(strs)</code>
tous / au moins un	—	<code>all(xs)</code> , <code>any(xs)</code>

`reduce` reste **le bon outil** pour les réductions sans builtin :

```
from operator import or_
merged = reduce(or_, dicts) # fusionner N dicts (| du chap. 1)
deep_get = reduce(lambda d, k: d[k], "a.b.c".split("."), config) # accès par chemin
composed = reduce(lambda f, g: lambda x: f(g(x)), funcs) # compose du chap. 3 !
```

Le module `operator` : les opérateurs en fonctions

Chaque opérateur Python existe en version **fonction nommée** — finies les micro-lambdas :

```
from operator import add, mul, itemgetter, attrgetter, methodcaller

reduce(mul, [1, 2, 3, 4])           # 24 — plutôt que lambda a, b: a * b
```

```
rows = [("alice", 30, "Lyon"), ("bob", 25, "Paris")]

# itemgetter : accès par index ou clé
sorted(rows, key=itemgetter(1))      # tri par âge
get_name_city = itemgetter(0, 2)     # plusieurs ! → tuple
get_name_city(rows[0])               # ('alice', 'Lyon')

# attrgetter : accès par attribut (gère les chemins !)
sorted(rows, key=attrgetter("customer.country", "amount"))

# methodcaller : appel de méthode
list(map(methodcaller("strip", "<>"), tags)) # list(map(methodcaller("split", ";"), lines))
```

Bonus performance : `itemgetter` / `attrgetter` sont implémentés en C — plus rapides qu'une lambda équivalente (~20-30 % sur un gros tri).

Les builtins fonctionnels : l'arsenal quotidien

```
scores = {"alice": 85, "bob": 92, "carol": 78}

# any / all : court-circuitent dès que possible
all(s >= 50 for s in scores.values()) # True - tous reçus ?
any(s >= 90 for s in scores.values()) # True - au moins un excellent ?

# sorted / min / max avec key
max(scores.items(), key=itemgetter(1)) # ('bob', 92)
sorted(scores, key=scores.get, reverse=True) # ['bob', 'alice', 'carol']

# zip : appairer des séquences
names, values = zip(*scores.items()) # "dézipper" : 2 tuples
dict(zip(keys, values)) # construire un dict
for prev, curr in zip(prices, prices[1:]): # fenêtre glissante de 2
    yield curr - prev

# enumerate : index + valeur (jamais range(len(...)) !)
for i, line in enumerate(lines, start=1):
    print(f"{i:>4}: {line}")
```

⚠ `zip` s'arrête au plus **court** ; `zip(a, b, strict=True)` (3.10+) lève si les longueurs diffèrent.

itertools 1/2 : chaîner, découper, conditionner

Des itérateurs combinables, paresseux, écrits en C — le langage des flux :

```
from itertools import chain, islice, takewhile, dropwhile, pairwise

# chain : concaténer sans copier
for item in chain(list_a, tuple_b, gen_c):
    ...
chain.from_iterable(list_of_lists)      # aplatir UN niveau

# islice : le slicing des itérateurs (chap. 1, version paresseuse)
first_100 = islice(huge_log, 100)        # head
sample    = islice(huge_log, 1000, 2000)  # tranche sans tout charger

# takewhile / dropwhile : couper sur condition
header = takewhile(lambda l: l.startswith("#"), lines)
body    = dropwhile(lambda l: l.startswith("#"), lines)

# pairwise (3.10+) : fenêtre glissante de 2
deltas = [b - a for a, b in pairwise(prices)]
```

Tout est **paresseux** : rien n'est calculé avant consommation — idéal sur fichiers et flux (chap. 5).

itertools 2/2 : grouper, combiner, accumuler

```
from itertools import groupby, product, combinations, permutations, accumulate

# groupby : regrouper les éléments CONSÉCUTIFS égaux
sales = sorted(sales, key=attrgetter("region")) # ⚠ TRIER d'abord !
for region, group in groupby(sales, key=attrgetter("region")):
    print(region, sum(s.amount for s in group))

# product : boucles imbriquées aplaties
for env, py in product(["dev", "prod"], ["3.11", "3.12"]):
    run_tests(env, py) # 4 combinaisons

# combinations / permutations
combinations("ABCD", 2) # AB AC AD BC BD CD - sans ordre
permutations("ABC", 2) # AB AC BA BC CA CB - avec ordre

# accumulate : reduce qui garde les étapes
list(accumulate([100, -30, 50, -20])) # [100, 70, 120, 100] - solde courant !
list(accumulate(rates, mul)) # produits cumulés
```

⚠ **Le piège** `groupby` : il ne groupe que les éléments **adjacents** — sans tri préalable, les groupes sont fragmentés. Pour grouper sans trier : `defaultdict(list)` (chap. 1).

Consolidation : créer ses décorateurs — le réflexe

Le squelette du chapitre 3, devenu automatisme — **trois questions** avant d'écrire :

1. Paramétré ou non ?

```
# Non → 2 niveaux
def log_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        print(f"→ {func.__name__}{args}")
        return func(*args, **kwargs)
    return wrapper

# Oui → 3 niveaux (fabrique)
def log_calls(logger=print):
    def decorator(func): ...
    return decorator
```

2. Avec état ?

Closure + `nonlocal`, ou classe `__call__` si l'état doit être inspectable (chap. 3).

3. Avant, après, autour, à la place ?

```
def wrapper(*args, **kwargs):
    ...                               # AVANT : valider, logger
    try:
        result = func(*args, **kwargs)
    except SomeError:
        ...                           # À LA PLACE : fallback
        result = default
    finally:
        ...                           # APRÈS : nettoyer, mesurer
    return transform(result)          # AUTOUR : enrichir
```

Un décorateur bien conçu = un **aspect** transversal (log, cache, retry, auth) sorti du code métier.

Pattern : le pipeline de données fonctionnel

Assembler les briques du chapitre — déclaratif, paresseux, testable :

```
from itertools import islice
from operator import attrgetter

def parse(lines):
    # str → Order (pur)
    return map(Order.from_csv, lines)

def only_valid(orders):
    # filtre (pur)
    return filter(lambda o: o.amount > 0 and o.status != "void", orders)

def enrich(orders, rates):
    # transformation (pure)
    return map(lambda o: o.with_tax(rates[o.country]), orders)

# La coquille impérative assemble et consomme :
def main():
    with open("orders.csv") as f:
        # effet de bord : I/O
        pipeline = enrich(only_valid(parse(f)), load_rates())
        top10 = sorted(pipeline, key=attrgetter("total"), reverse=True)[:10]
        report(top10)
        # effet de bord : sortie
```

- Chaque étape : **pure**, testable avec une simple liste en entrée
- Rien n'est chargé en mémoire avant `sorted` — le fichier coule **ligne à ligne** à travers les `map / filter`
- Au chapitre 5, les **générateurs** rendront ce style encore plus naturel

Récursion en Python : possible, mais encadrée 1/2

Le bon cas : structures récursives

```
def walk(node) -> int:
    """Taille totale d'une arborescence."""
    if node.is_file():
        return node.size          # cas de base
    return sum(walk(child)
               for child in node.children)
```

Arbres, JSON imbriqué, AST, parcours de graphes : la récursion épouse la structure.

Les limites de Python

```
import sys
sys.getrecursionlimit()          # 1000 par défaut

def count(n):
    return 0 if n == 0 else 1 + count(n - 1)
count(10_000)                    # ⚡ RecursionError
```

- **Pas d'optimisation d'appel terminal** (TCO) — choix assumé de Guido (stacktraces lisibles)
- Profondeur > ~1000 → réécrire en **itératif** avec une pile explicite (`deque`, chap. 1)

Récursion en Python : possible, mais encadrée 2/2

```
def walk_iterative(root) -> int:          # même algo, sans limite de profondeur
    total, stack = 0, [root]
    while stack:
        node = stack.pop()
        total += node.size if node.is_file() else 0
        stack.extend(node.children)
    return total
```

Règle : récursion pour la **lisibilité** sur données peu profondes ; itératif pour les données **non bornées**.

Bonnes pratiques — synthèse du chapitre

Le style fonctionnel pythonique :

- Fonctions **pures** par défaut ; effets de bord regroupés dans une coquille fine (*functional core, imperative shell*)
- Compréhensions d'abord ; `map` / `filter` quand la fonction existe déjà ; `reduce` en dernier recours
- `operator.itemgetter` / `attrgetter` / `methodcaller` plutôt que des micro-lambdas
- `itertools` pour les flux : paresseux, composable, écrit en C
- `groupby` exige un tri préalable ; `zip(strict=True)` pour détecter les désalignements

Ce que le fonctionnel apporte à la suite de la formation :

Acquis	Réutilisé pour
Paresse (<code>map</code> , <code>itertools</code>)	Générateurs — chap. 5
Pureté	Parallélisme sans verrous — chap. 12

Simple is better than complex : le fonctionnel en Python est un **style**, pas une religion — mélangez les paradigmes là où chacun excelle.

Travaux pratiques

TP 4.1 — Création de décorateurs (45 min)

1. `@log_calls(logger)` : logue entrée (nom, arguments via `inspect.signature.bind` — chap. 2) et sortie (résultat ou exception) — testez-le avec un logger factice injecté
2. `@memoize` maison : votre propre `lru_cache` simplifié (dict en closure, clé = `(args, frozenset(kwargs.items()))`) — comparez son comportement avec `functools.cache` sur arguments non hachables
3. `@deprecated(reason, since)` : émet un `DeprecationWarning` (module `warnings`) au premier appel **seulement** (état dans la closure)
4. `@ensure(predicate, message)` : vérifie une post-condition sur le résultat — `@ensure(lambda r: r >= 0, "montant négatif")` — et chaînez-en plusieurs

TP 4.2 — Implémentation de patterns (60 min)

À partir d'un fichier de ventes (CSV fourni, ~50 000 lignes : date, région, produit, quantité, prix) :

1. **Pipeline pur** : parse → validation (filtrage des lignes corrompues) → enrichissement (montant = quantité × prix) — chaque étape une fonction pure testée, composition avec votre `pipe` du chap. 3
2. **Reporting fonctionnel** : CA par région (`groupby` après tri), top 5 produits (`sorted` + `itemgetter`), CA cumulé par mois (`accumulate`), tout en style déclaratif — **aucune boucle for d'accumulation**
3. **Fenêtre glissante** : moyenne mobile sur 7 jours des ventes quotidiennes avec `pairwise` / `islice` (ou une `deque(maxlen=7)`)
4. **Refactoring** : on vous fournit une fonction impérative de 60 lignes truffée d'effets de bord — découpez-la en cœur fonctionnel pur + coquille impérative, et écrivez les tests du cœur **sans aucun mock**

Livrables : code + tests pytest. Critère de réussite du 4.2.4 : la logique métier testée sans `mock.patch` .

Chapitre 5

Générateurs et itérateurs

Python Perfectionnement — Jour 2

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Expliquer** le protocole d'itération : `__iter__`, `__next__`, `StopIteration`
- **Distinguer** itérable et itérateur — et pourquoi cette distinction change tout
- **Développer** vos propres itérateurs (classes) et générateurs (`yield`)
- **Composer** des pipelines de générateurs paresseux ; déléguer avec `yield from`
- **Mesurer** les gains mémoire et les compromis de performance
- **Éviter** les pièges : épuisement, réutilisation, fermeture

Travaux pratiques : développement d'itérateurs, création de générateurs, tests de performance.

Ce chapitre explique la **paresse** entrevue aux chapitres 3 (Rx) et 4 (`map`, `itertools`).

Ce que cache une boucle `for`

```
for item in collection:
    process(item)
```

Le `for` n'est que du sucre. Voici ce que Python exécute **réellement** :

```
iterator = iter(collection)      # 1. appelle collection.__iter__()
while True:
    try:
        item = next(iterator)    # 2. appelle iterator.__next__()
    except StopIteration:        # 3. fin du flux – signal NORMAL, pas une erreur
        break
    process(item)
```

Deux méthodes, une exception — c'est tout le protocole d'itération :

Élément	Rôle
<code>__iter__()</code>	renvoie un itérateur (souvent un objet distinct)
<code>__next__()</code>	renvoie l'élément suivant, ou lève <code>StopIteration</code>
<code>StopIteration</code>	signale la fin — consommée silencieusement par <code>for</code>

Ce protocole irrigue **tout** Python : `for`, compréhensions, `sum`, `zip`, `list()`, unpacking `*`, `in ...`

Itérable vs itérateur : la distinction clé

Itérable

« On peut itérer **dessus** » — a `__iter__()` qui renvoie un itérateur.

```
nums = [1, 2, 3]          # list = itérable
iter(nums)                # <list_iterator>
iter(nums) is iter(nums)  # False — un NOUVEL
                           # itérateur à chaque fois
```

→ **réutilisable** : chaque `for` repart de zéro.

```
# Le test pratique :
from collections.abc import Iterable, Iterator
isinstance([1, 2], Iterable) # True      isinstance([1, 2], Iterator) # False
isinstance(iter([1]), Iterator) # True    # tout Iterator est aussi Iterable (il renvoie self)
```

Itérateur

« Le curseur qui avance » — a `__next__()` **et** `__iter__()` (qui renvoie `self`).

```
it = iter(nums)
next(it)          # 1
next(it)          # 2
next(it), next(it) # 3 puis * StopIteration
list(it)          # [] — ÉPUISÉ à jamais
```

→ **jetable** : à usage unique, garde sa position.

Conséquence concrète : un fichier ouvert, un `map`, un générateur sont des **itérateurs** → une seule passe !

Développer un itérateur : la version classe

Le pattern complet — un itérable qui fabrique des itérateurs indépendants :

```
class Countdown:
    """Itérable : chaque for repart du début."""
    def __init__(self, start: int):
        self.start = start

    def __iter__(self):
        # fabrique un NOUVEL itérateur
        return CountdownIterator(self.start)

class CountdownIterator:
    """Itérateur : le curseur, à usage unique."""
    def __init__(self, current: int):
        self.current = current

    def __iter__(self):
        # un itérateur se renvoie lui-même
        return self

    def __next__(self):
        if self.current <= 0:
            raise StopIteration
        self.current -= 1
        return self.current + 1

cd = Countdown(3)
list(cd), list(cd)      # [3, 2, 1], [3, 2, 1] - réutilisable ✓
```

Deux classes, de la plomberie... Les **générateurs** vont écrire tout ça pour nous.

Les générateurs : `yield` change tout

Une fonction contenant `yield` devient une **fabrique de générateurs** — Python écrit l'itérateur à notre place :

```
def countdown(start: int):
    while start > 0:
        yield start
        start -= 1

gen = countdown(3)
next(gen)
next(gen)
next(gen)
next(gen)
```

PAUSE : émet la valeur, gèle la fonction ici

⚠ RIEN ne s'exécute encore !
 # 3 – exécute jusqu'au 1er yield
 # 2 – REPRENDR après le yield, état intact
 # 1
 # **✳** StopIteration (fin de la fonction)

Le modèle mental : `yield` **suspend** la fonction — variables locales, position, pile : tout est conservé entre deux `next()`. C'est une fonction **pausable**.

Itérateur classe (slide précédente)	Générateur
2 classes, ~20 lignes	1 fonction, 4 lignes
état dans <code>self.xxx</code>	état = les variables locales
<code>raise StopIteration</code> explicite	<code>return</code> /fin de fonction suffit

Les 5 lignes de `countdown` remplacent **intégralement** les deux classes précédentes.

Exécution d'un générateur : pas à pas

```
def pipeline_demo():
    print("❶ démarrage")
    data = expensive_load()      # exécuté au 1er next() seulement !
    print("❷ données prêtes")
    yield data
    print("❸ entre les deux yields")
    yield transform(data)
    print("❹ nettoyage final")

g = pipeline_demo()             # <generator object> – AUCUN print, AUCUN chargement
```

```
next(g)  → ❶ démarrage          le code court jusqu'au 1er yield
           ❷ données prêtes
           → data

next(g)  → ❸ entre les deux yields  reprise PILE après le yield
           → transform(data)

next(g)  → ❹ nettoyage final      fin de fonction
           ✨ StopIteration
```

Deux propriétés fondamentales : évaluation **paresseuse** (rien avant le premier `next`) et calcul **à la demande** (un élément à la fois — jamais toute la collection en mémoire).

Expressions génératrices : la compréhension paresseuse

Même syntaxe qu'une compréhension, des **parenthèses** à la place des crochets :

```
squares_list = [n * n for n in range(1_000_000)]    # 1M d'éléments EN MÉMOIRE
squares_gen  = (n * n for n in range(1_000_000))    # un curseur de ~200 octets

import sys
sys.getsizeof(squares_list)    # ~8 448 728 octets
sys.getsizeof(squares_gen)    # ~200 octets – peu importe le volume !
```

```
# Seul argument d'une fonction → parenthèses superflues
total = sum(n * n for n in range(1_000_000))    # idiomatique ✓
any(u.is_admin for u in users)    # court-circuite en plus !

# Chainables comme les map/filter du chap. 4
lines = (l.strip() for l in open("app.log"))
errors = (l for l in lines if "ERROR" in l)
codes = (extract_code(l) for l in errors)
# ↑ trois lignes déclarées, RIEN n'a encore lu le fichier
```

Règle de choix : résultat consommé **une fois, en flux** → genexp ; besoin de `len()`, d'indexation, de plusieurs passes → liste.

Le pattern pipeline de générateurs

Le pipeline du chapitre 4, version générateurs — chaque étage **tire** sur le précédent :

```
def read_logs(path):
    with open(path) as f:
        for line in f:
            yield line.rstrip("\n")
    # ÉTAGE 1 : source

def grep(lines, pattern):
    for line in lines:
        if pattern in line:
            yield line
    # ÉTAGE 2 : filtre

def parse(lines):
    for line in lines:
        ts, level, msg = line.split(" ", 2)
        yield {"ts": ts, "level": level, "msg": msg}
    # ÉTAGE 3 : transformation

# Assemblage déclaratif — paresseux de bout en bout :
events = parse(grep(read_logs("huge.log"), "ERROR"))
for event in islice(events, 10):
    print(event["msg"])
    # ne lit QUE ce qu'il faut du fichier !
```

- Un fichier de **50 Go** se traite avec ~quelques Ko de RAM — ligne à ligne
- Chaque étage est **testable** isolément : `list(grep(["a ERROR b", "ok"], "ERROR"))`
- C'est l'architecture des outils Unix (`cat | grep | awk`) transposée en Python

yield from : déléguer à un sous-générateur

```
# ❌ La boucle de copie
def flatten(nested):
    for sub in nested:
        for item in sub:
            yield item

# ✅ yield from
def flatten(nested):
    for sub in nested:
        yield from sub
```

`yield from` **branche** directement le consommateur sur le sous-itérable — valeurs, exceptions et `close()` transitent sans intermédiaire.

```
# Le cas roi : la récursion génératrice (arborescences – chap. 4)
def walk_files(directory: Path):
    for entry in directory.iterdir():
        if entry.is_dir():
            yield from walk_files(entry)    # délègue à l'appel récursif
        else:
            yield entry

# Composition de sources
def all_events():
    yield from read_kafka()
    yield from read_fallback_file()        # enchaîne quand le premier s'épuise
```

```
big_tree_size = sum(f.stat().st_size for f in walk_files(Path("/data")))
```

⚠ La limite de récursion (chap. 4) s'applique toujours — profondeur d'arborescence raisonnable.

Générateurs infinis : penser en flux

Un générateur n'a pas besoin de fin — c'est le consommateur qui décide quand s'arrêter :

```
def sensor_readings():                # source infinie
    while True:
        yield read_sensor()

def ticket_ids(prefix: str):          # séquence infinie
    n = 0
    while True:
        n += 1
        yield f"{prefix}-{n:06d}"
```

```
from itertools import count, cycle, repeat, islice

count(10, 2)           # 10, 12, 14, ...      compteur infini
cycle("ABC")           # A, B, C, A, B, ...    rotation (round-robin !)
repeat(None)           # None, None, ...      avec zip : padding

# On borne TOUJOURS à la consommation :
first_5 = list(islice(ticket_ids("CMD"), 5))
for server, request in zip(cycle(servers), requests): # zip s'arrête au fini
    dispatch(server, request)
```

 **Jamais** `list()`, `sum()`, `sorted()`, `max()` sur un flux infini — boucle sans fin. Bornez avec `islice`, `takewhile` ou `zip` avec une séquence finie.

send(), throw(), close() : le canal retour

`yield` est **bidirectionnel** : le consommateur peut renvoyer des données dans le générateur :

```
def running_average():
    total, count = 0.0, 0
    avg = None
    while True:
        value = yield avg          # ➡ yield ÉMET avg, puis REÇOIT la valeur envoyée
        total += value
        count += 1
        avg = total / count

acc = running_average()
next(acc)          # amorçage obligatoire : avancer jusqu'au 1er yield
acc.send(10)       # 10.0
acc.send(20)       # 15.0
acc.send(30)       # 20.0
```

Méthode	Effet dans le générateur
<code>send(v)</code>	reprend, <code>yield</code> vaut <code>v</code>
<code>throw(exc)</code>	lève <code>exc</code> au point du <code>yield</code> (gérable par <code>try</code>)
<code>close()</code>	lève <code>GeneratorExit</code> → le générateur doit terminer

Note historique : ces « coroutines par générateurs » sont l'ancêtre direct d'`async / await` (chap. 12). Aujourd'hui, `send()` reste rare — on le rencontre dans du code legacy et certains frameworks.

Nettoyage garanti: `try/finally` dans un générateur

Un générateur abandonné à mi-parcours doit pouvoir libérer ses ressources :

```
def read_records(path):
    f = open(path)
    try:
        for line in f:
            yield parse(line)
    finally:
        f.close()          # exécuté MÊME si le consommateur abandonne
        print("fichier fermé")

records = read_records("data.csv")
first = next(records)
del records                # abandon → GeneratorExit → finally ✓
```

- À la destruction (ou sur `close()`), Python lève `GeneratorExit` au point du `yield` → les `finally` et `with` internes s'exécutent
- Préférez `with open(...)` dans le générateur — même garantie, plus idiomatique

```
from contextlib import closing
with closing(read_records("data.csv")) as records:    # fermeture DÉTERMINISTE
    for r in records:
        if r.corrupted:
            break                                     # le with ferme le générateur proprement
```

Ce besoin de nettoyage déterministe annonce les **context managers** (chap. 9).

Tests de performance : mémoire

La promesse des générateurs, mesurée (`tracemalloc` , chap. 1) :

```
import tracemalloc

def measure(make_pipeline):
    tracemalloc.start()
    result = sum(make_pipeline()) # consommation complète
    _, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return result, peak

# Pipeline LISTES : chaque étape matérialise 1M d'éléments
def with_lists():
    nums = list(range(1_000_000))
    squares = [n * n for n in nums]
    evens = [s for s in squares if s % 2 == 0]
    return evens

# Pipeline GÉNÉRATEURS : un élément à la fois traverse les 3 étages
def with_gens():
    nums = range(1_000_000)
    squares = (n * n for n in nums)
    return (s for s in squares if s % 2 == 0)
```

Pipeline	Pic mémoire	Résultat
listes	~76 Mo	identique
générateurs	~0,001 Mo	identique

Même calcul, **×50 000 sur la mémoire** — et le pipeline générateurs démarre instantanément (*time-to-first-result*).

Tests de performance : CPU — l'honnêteté des chiffres

La paresse n'est pas gratuite : chaque `next()` a un coût de **suspension/reprise**.

```
import timeit

timeit.timeit(lambda: sum([n * n for n in range(10_000)]), number=1000) # ~0.55 s
timeit.timeit(lambda: sum(n * n for n in range(10_000)), number=1000) # ~0.60 s
```

Générateur gagnant quand...

- Le volume **dépasse la RAM** (ou la menace)
- On ne consomme qu'une **partie** du flux (early exit, `islice`, `any`)
- Le *time-to-first-result* compte (streaming)
- Source **infinie** ou inconnue à l'avance

Liste gagnante quand...

- Petit volume consommé **entièrement**
- **Plusieurs passes** nécessaires
- Besoin de `len()`, tri, indexation
- CPU-bound pur sur données déjà en RAM (~5-10 % plus rapide)

```
# Le cas où le générateur écrase tout : l'arrêt anticipé
any(is_match(x) for x in huge_stream()) # s'arrête à la 1re correspondance ✓
any([is_match(x) for x in huge_stream()]) # calcule TOUT avant de répondre ✗
```

Réflexe : mesurez sur **vos** données (`timeit` + `tracemalloc`) — méthodologie complète au chap. 11.

Les pièges classiques des générateurs

1. L'épuisement silencieux

```
evens = (n for n in nums if n % 2 == 0)
list(evens)      # [0, 2, 4, ...]
sum(evens)       # 0 🚩 - déjà épuisé,
                # AUCUNE erreur !
```

Besoin de 2 passes → matérialiser (`list`) ou re-crée le générateur (fonction !).

2. `tee` : dupliquer un flux

```
from itertools import tee
a, b = tee(stream, 2)
```

⚠ `tee` **bufferise** l'écart entre les deux curseurs — si l'un prend 1M d'avance, le buffer fait 1M.

3. Late binding (chap. 3) — version genexp

```
gen = (x * factor for x in data)
factor = 100      # modifié APRÈS
list(gen)         # utilise 100 !
```

Seul l'**itérable source** est évalué immédiatement ; le reste à la consommation.

4. L'exception au mauvais moment

```
def parse_all(lines):
    for l in lines:
        yield int(l)      # ✨ explose à la
                        # CONSOMMATION
gen = parse_all(["1", "x"]) # aucune erreur ici
```

L'erreur surgit loin de sa cause → validez tôt, ou capturez **dans** le générateur.

Et les générateurs asynchrones ? (aperçu)

Le même modèle, transposé au monde `async` — détaillé au chapitre 12 :

```
async def fetch_pages(urls):
    # async + yield = générateur asynchrone
    async with aiohttp.ClientSession() as session:
        for url in urls:
            response = await session.get(url)  # n'attend PAS en bloquant le programme
            yield await response.json()       # émet dès qu'une page arrive

async def main():
    async for page in fetch_pages(urls):
        process(page)
```

Concept synchrone (ce chapitre)	Équivalent asynchrone (chap. 12)
<code>def + yield</code>	<code>async def + yield</code>
<code>for x in gen</code>	<code>async for x in agen</code>
<code>__iter__ / __next__</code>	<code>__aiter__ / __anext__</code>
paresse : suspendu entre deux <code>next</code>	+ suspendu pendant les I/O

À retenir pour l'instant : tout le modèle mental construit ici (flux, paresse, pipelines, épuisement) se transpose tel quel — vous avez déjà fait 80 % du chemin vers `asyncio`.

Bonnes pratiques — synthèse du chapitre

Conception :

- Générateur = le **défaut** pour produire des séquences ; classe itérateur seulement si l'état doit être inspectable/réutilisable
- Itérable **réutilisable** → classe avec `__iter__` qui *fabrique* des générateurs ; itérateur = à usage unique
- Pipelines : petites étapes nommées + assemblage déclaratif ; `yield from` pour déléguer et aplatir
- Ressources dans un générateur : `with / try-finally` internes, `contextlib.closing` côté consommateur

Performance :

- Le gain des générateurs est en **mémoire** et en **time-to-first-result**, pas en CPU pur
- Arrêt anticipé (`any` , `islice` , `break`) : toujours sur générateur, jamais sur liste matérialisée
- Deux passes nécessaires → `list()` assumé ; `tee` seulement si les curseurs restent proches

Les réflexes anti-pièges :

- Un itérateur épuisé est **vide sans erreur** — méfiance après tout `list()` / `sum()`
- Jamais d'agrégat (`list` , `sorted` , `sum`) sur un flux infini — borner d'abord
- `groupby` , `zip` , fichiers, `map` : tous à usage unique — re-crée plutôt que réutiliser

Travaux pratiques 1/2

TP 5.1 — Développement d'itérateurs (30 min)

1. Implémentez `SlidingWindow(seq, size)` : itérable **réutilisable** qui produit les fenêtres glissantes — `list(SlidingWindow([1,2,3,4], 2))` → `[(1,2), (2,3), (3,4)]` — d'abord en deux classes (itérable + itérateur), puis en classe dont `__iter__` est un générateur. Comparez le nombre de lignes
2. Vérifiez avec `collections.abc` que vos objets sont bien `Iterable` / `Iterator`, et que deux boucles `for` successives donnent le même résultat
3. Bonus : `Peekable(iterator)` — un wrapper d'itérateur avec une méthode `peek()` qui regarde sans consommer

TP 5.2 — Création de générateurs (45 min)

1. Réécrivez le pipeline de logs du cours pour **votre** fichier (fourni, 500 Mo) : `read_logs` → `grep("ERROR")` → `parse` → top 10 des messages (`Counter` du chap. 1) — le tout sans jamais charger le fichier
2. `batch(iterable, size)` : regroupe tout itérable en paquets de `size` éléments (le dernier peut être incomplet) — indispensable pour les insertions BDD par lots. Interdiction de `len()` : il doit marcher sur un flux infini
3. `walk_files(path, pattern)` avec `yield from` : parcours récursif filtré d'une arborescence
4. Un générateur `tail_f(path)` façon `tail -f` : émet les nouvelles lignes au fur et à mesure (boucle + `sleep` court), avec fermeture propre du fichier via `try/finally` — testez l'abandon avec `close()`

Travaux pratiques 2/2

TP 5.3 — Tests de performance (45 min)

1. Sur le pipeline du TP 5.2.1 : mesurez pic mémoire (`tracemalloc`) et durée (`timeit`) en version **listes** vs version **générateurs** — tableau comparatif à 3 volumes (1 %, 10 %, 100 % du fichier)
2. Mesurez le *time-to-first-result* : temps pour obtenir le **premier** événement parsé dans chaque version
3. Cas d'arrêt anticipé : trouver la **première** ligne contenant un motif rare — comparez `next(gen)` vs compréhension complète
4. **Conclusion écrite** (3 lignes) : pour ce cas d'usage, que recommandez-vous et pourquoi ?

Livrables : code + tests pytest + mini-rapport de mesures du TP 5.3.

Chapitre 6

Concepts POO avancés

Python Perfectionnement — Jour 2

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Encapsuler** proprement avec `property` — sans getters/setters à la Java
- **Implémenter** des descripteurs : le mécanisme derrière `property`, méthodes, `classmethod`
- **Maîtriser** l'héritage multiple : MRO, `super()` coopératif, mixins
- **Architecturer** avec les classes abstraites (`abc`) et les `Protocol`
- **Arbitrer** entre héritage et composition

Travaux pratiques : implémentation de descripteurs, exercices d'héritage multiple, architecture avec classes abstraites.

Les **métaclasses** et descripteurs avancés (protocoles complets) arrivent au **chapitre 8** — ce chapitre installe la mécanique.

Rappel : où vivent les attributs ?

```
class Account:
    bank = "ACME"           # attribut de CLASSE – partagé
    def __init__(self, owner):
        self.owner = owner  # attribut d'INSTANCE – propre à chacun

a = Account("alice")
```

La lecture `a.x` remonte la chaîne (chap. 2) : `a.__dict__` → `Account.__dict__` → bases (MRO).

Le piège du masquage

```
a.bank = "OTHER"  # crée un attribut
                  # d'INSTANCE qui MASQUE
Account.bank      # 'ACME' – inchangé
a.bank           # 'OTHER'
```

Le piège du mutable partagé

```
class Basket:
    items = []      # ✗ PARTAGÉ par toutes
                   # les instances !
    def __init__(self):
        self.items = []  # ✔ par instance
```

L'écriture `a.x = v` va toujours dans `a.__dict__` ... *sauf* si un **descripteur** intercepte — tout ce chapitre est là.

property : l'encapsulation pythonique

Pas de getters/setters préventifs en Python — on commence par un attribut **public**, on ajoute `property` quand le besoin arrive, sans casser les appelants :

```
class Temperature:
    def __init__(self, celsius: float):
        self.celsius = celsius          # appelle déjà le setter ↓ !

    @property
    def celsius(self) -> float:         # le GETTER
        return self._celsius

    @celsius.setter
    def celsius(self, value: float) -> None: # le SETTER – validation centralisée
        if value < -273.15:
            raise ValueError(f"{value} °C : sous le zéro absolu")
        self._celsius = value

    @property
    def fahrenheit(self) -> float:      # attribut CALCULÉ (lecture seule)
        return self._celsius * 9 / 5 + 32

t = Temperature(25)
t.celsius = -300      # ✖ ValueError – même syntaxe qu'un attribut !
t.fahrenheit          # 77.0
t.fahrenheit = 80     # ✖ AttributeError : pas de setter → lecture seule
```

L'API ne change jamais : `t.celsius` reste `t.celsius`, qu'il y ait validation ou non derrière. C'est le *Uniform Access Principle*.

property : conventions et limites

Les conventions

- `_celsius` : **privé par convention** — un seul underscore, pas de contrôle réel
- `__name` : *name mangling* (`__Classe__name`) — anti-collision en héritage, pas de la sécurité
- Une property doit rester **rapide et sans surprise** : c'est syntaxiquement un attribut

```
# ✗ Une property qui fait une requête SQL
@property
def orders(self):
    return db.query(...)    # caché, lent,
                           # appelé en boucle...

# ✓ Une méthode dit le coût
def fetch_orders(self): ...
# ✓ ou un calcul mémorisé
@functools.cached_property
def stats(self): ...      # chap. 3
```

La limite : la répétition

```
class Product:
    @property
    def price(self): return self._price
    @price.setter
    def price(self, v):
        if v < 0: raise ValueError(...)
        self._price = v

    @property
    def stock(self): return self._stock
    @stock.setter
    def stock(self, v):
        if v < 0: raise ValueError(...)
        self._stock = v
    # ... la MÊME validation, copiée-collée
```

Dupliquer la validation sur 10 champs ? Il nous faut une property **réutilisable et paramétrable** → le **descripteur**.

Les descripteurs : le protocole

Un **descripteur** est un objet qui, placé comme **attribut de classe**, intercepte l'accès aux instances :

```
class Descriptor:
    def __get__(self, obj, objtype=None): ... # lecture : obj.attr
    def __set__(self, obj, value): ...      # écriture : obj.attr = v
    def __delete__(self, obj): ...          # del obj.attr
```

```
class Positive:
    """Un champ numérique qui refuse les valeurs négatives."""
    def __set_name__(self, owner, name):
        self.name = name
        # appelé À LA CRÉATION de la classe hôte
        # 'price', 'stock'... automatique !

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        # accès via la CLASSE → le descripteur lui-même
        return obj.__dict__[self.name]

    def __set__(self, obj, value):
        if value < 0:
            raise ValueError(f"{self.name} doit être ≥ 0, reçu {value}")
        obj.__dict__[self.name] = value
        # stockage DANS l'instance
```

`__set_name__` (3.6+) : le descripteur apprend **son propre nom** — fini les noms passés en double.

Descripteurs : la validation déclarative

Le copier-coller de propriétés devient **une ligne par champ** :

```
class Product:
    price = Positive()          # UN descripteur par champ -
    stock = Positive()         # la validation est factorisée, déclarative
    name = NonEmptyString()

    def __init__(self, name, price, stock):
        self.name, self.price, self.stock = name, price, stock    # setters appelés !

p = Product("Clavier", 49.9, 12)
p.price = -5                # ✖ ValueError: price doit être ≥ 0, reçu -5
```

La mécanique de `p.price` :

```
lecture p.price    → type(p).__mro__ trouve Positive dans Product.__dict__
                  → data descriptor : son __get__ GAGNE sur p.__dict__
                  → Positive.__get__(desc, p, Product) → p.__dict__['price']
écriture p.price = -5 → Positive.__set__(desc, p, -5) → ValueError
```

C'est **exactement** ainsi que fonctionnent les champs des ORM (`models.DecimalField()` de Django), de SQLAlchemy, d'attrs/Pydantic v1...

Vous savez désormais lire leur code source.

Data vs non-data descriptors : la précedence

Type	Méthodes définies	Priorité vs <code>obj.__dict__</code>
Data descriptor	<code>__set__</code> (et/ou <code>__delete__</code>)	gagne sur l'instance
Non-data descriptor	<code>__get__</code> seulement	l'instance gagne

L'ordre complet de résolution de `obj.attr` :

```

1. data descriptor trouvé sur type(obj) (MRO)      → son __get__
2. obj.__dict__["attr"]                          → la valeur de l'instance
3. non-data descriptor / attribut de classe (MRO) → __get__ ou la valeur
4. __getattr__ (chap. 2)                         → le filet de sécurité
5. AttributeError

```

Pourquoi c'est génial — les *méthodes* sont des non-data descriptors :

```

class Service:
    def handler(self): ...           # fonction = non-data descriptor

s = Service()
s.handler                          # méthode liée via __get__
s.handler = lambda: "patché"      # l'instance GAGNE → monkey patch possible (chap. 2)

```

...tandis que `property` définit `__set__` → **data** : impossible de l'écraser par accident. Tout est cohérent.

Tout est descripteur : le modèle unifié

Les « magies » de Python sont le même mécanisme :

```
class Demo:
    @property          # data descriptor      → priorité absolue, contrôle total
    def x(self): ...

    @classmethod       # non-data descriptor → __get__ lie la CLASSE (cls)
    def create(cls): ...

    @staticmethod     # non-data descriptor → __get__ renvoie la fonction nue
    def helper(): ...

    def method(self): ... # fonction          → __get__ lie l'INSTANCE (self)
```

```
Demo.method          # <function Demo.method>      (accès classe : pas de liaison)
Demo().method        # <bound method Demo.method>   (accès instance : self pré-rempli)
Demo().method.__func__ # la fonction d'origine
```

`obj.method(arg)` n'est que :

`type(obj).method.__get__(obj)(arg)`

La « liaison » de `self` est un appel de `__get__`.

`functools.cached_property` (chap. 3) : un **non-data** descriptor — au 1er accès il écrit dans `obj.__dict__`, qui **gagne** ensuite → plus jamais rappelé. Élégant !

classmethod et staticmethod : les bons usages

@classmethod : constructeurs alternatifs

```
class Invoice:
    def __init__(self, ref, lines):
        self.ref, self.lines = ref, lines

    @classmethod
    def from_json(cls, payload: str):
        d = json.loads(payload)
        return cls(d["ref"], d["lines"])

    @classmethod
    def empty(cls):
        return cls(next_ref(), [])
```

- Plusieurs « façons de construire » **nommées** — plus clair qu'un `__init__` à 8 paramètres optionnels
- `cls` (pas le nom en dur !) → l'héritage construit la **bonne** sous-classe
- Pattern stdlib : `dict.fromkeys`, `datetime.fromtimestamp`, `Path.cwd`

@staticmethod : l'utilitaire rangé

```
class Invoice:
    @staticmethod
    def validate_ref(ref: str) -> bool:
        return bool(re.match(r"INV-\d{6}", ref))
```

- Ni `self` ni `cls` : une fonction **thématiquement** liée à la classe
- Testable sans instance ; surchargeable en sous-classe (contrairement à une fonction module)
- **Si elle n'utilise rien de la classe**, une fonction de module est souvent plus simple — ne sur-rangez pas

Règle rapide : touche à l'état d'instance → méthode ; construit/inspecte la classe → `classmethod` ; aucun des deux → `staticmethod` ou fonction libre.

Héritage multiple : le problème du diamant

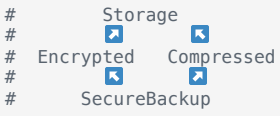
```
class Storage:
    def save(self): print("Storage.save")

class Encrypted(Storage):
    def save(self): print("chiffre...")

class Compressed(Storage):
    def save(self): print("compresse...")

class SecureBackup(Encrypted, Compressed):
    pass

SecureBackup().save()      # lequel ?? chiffre ? compresse ? Storage ?
```



```
graph TD
    Storage --> Encrypted
    Storage --> Compressed
    Encrypted --> SecureBackup
    Compressed --> SecureBackup
```

Réponse de Python : le MRO (*Method Resolution Order*), calculé par la linéarisation C3 :

```
SecureBackup.__mro__
# (SecureBackup, Encrypted, Compressed, Storage, object)
```

Garanties de C3 : chaque classe apparaît **une fois**, **avant** ses bases, et l'ordre des bases déclarées est **respecté** (`Encrypted` avant `Compressed`). Si aucun ordre cohérent n'existe → `TypeError` à la définition.

Donc : `SecureBackup().save()` → `Encrypted.save` ... mais comment **chaîner** chiffrement et compression ?

`super()` : coopératif, pas « parent »

L'idée fausse : `super()` $\neq$ « la classe mère ». `super()` = « le suivant dans le MRO » — qui dépend de la classe *finale* instanciée :

```
class Storage:
    def save(self):
        print("→ écriture disque")

class Encrypted(Storage):
    def save(self):
        print("chiffrement")
        super().save()          # suivant dans le MRO - PAS forcément Storage !

class Compressed(Storage):
    def save(self):
        print("compression")
        super().save()

class SecureBackup(Encrypted, Compressed):
    pass

SecureBackup().save()
# chiffrement      ← Encrypted
# compression      ← super() depuis Encrypted = Compressed (le MRO !)
# → écriture disque ← Storage
```

Dans `Encrypted.save`, `super()` appelle `Compressed.save` quand l'instance est un `SecureBackup` — chaque classe fait sa part puis **passe la main**. C'est l'héritage **coopératif** : toute la chaîne s'exécute, dans l'ordre du MRO.

Héritage coopératif : le contrat `**kwargs`

Pour que `super().__init__` traverse **toute** la chaîne avec des signatures différentes :

```
class Base:
    def __init__(self, **kwargs):
        super().__init__(**kwargs)          # relaie même dans la "dernière" – object accepte ()

class Persisted(Base):
    def __init__(self, *, db_url, **kwargs): # prend SON paramètre...
        self.db_url = db_url
        super().__init__(**kwargs)          # ...et RELAIE le reste

class Audited(Base):
    def __init__(self, *, audit_log, **kwargs):
        self.audit_log = audit_log
        super().__init__(**kwargs)

class Service(Persisted, Audited):
    def __init__(self, name, **kwargs):
        self.name = name
        super().__init__(**kwargs)

svc = Service("billing", db_url="postgres://...", audit_log="/var/log/audit")
# Service → Persisted → Audited → Base → object : chacun prélève, chacun relaie ✓
```

Les trois règles d'or : paramètres en *keyword-only* (`*`), chacun **prélève les siens** et relaie `**kwargs`, et `super().__init__` **partout** — même quand on croit être « en haut ».

Les mixins : l'héritage multiple utile

En pratique, 95 % de l'héritage multiple sain = **une seule vraie base + des mixins** :

```
class JSONMixin:
    """Ajoute la sérialisation JSON - AUCUN état propre."""
    def to_json(self) -> str:
        return json.dumps(vars(self))

class ComparableMixin:
    """Dérive tous les opérateurs de _key()."""
    def _key(self): raise NotImplementedError
    def __eq__(self, other): return self._key() == other._key()
    def __lt__(self, other): return self._key() < other._key()
    # (en vrai : functools.total_ordering fait ça tout seul !)

class Employee(JSONMixin, ComparableMixin):      # mixins À GAUCHE, base à droite
    def __init__(self, name, salary):
        self.name, self.salary = name, salary
    def _key(self): return (self.salary, self.name)
```

La charte du bon mixin :

- Pas de `__init__` (ou alors coopératif) — pas d'état, juste du **comportement**
- Un mixin = **une** capacité (`to_json`, comparaison, logging...) — nom en `...Mixin`
- Jamais instancié seul ; documenté sur ce qu'il **attend** de l'hôte (`_key()` ici)

Exemples réels : `LoginRequiredMixin` (Django), `socketserver.ThreadingMixin` (stdlib).

Classes abstraites : le contrat formel

`abc.ABC` : définir une **interface obligatoire** que les sous-classes doivent honorer :

```
from abc import ABC, abstractmethod

class PaymentProvider(ABC):
    """Contrat : tout fournisseur de paiement DOIT savoir faire ça."""

    @abstractmethod
    def charge(self, amount: float, token: str) -> str:
        """Débite et renvoie l'id de transaction."""

    @abstractmethod
    def refund(self, transaction_id: str) -> None: ...

    def charge_with_retry(self, amount, token, attempts=3):
        for _ in range(attempts):
            try:
                return self.charge(amount, token)
            except TransientError:
                continue
        raise PaymentFailed(amount)

    # méthode CONCRÈTE
    # qui s'appuie sur
    # les abstraites

PaymentProvider() # ✖ TypeError: Can't instantiate abstract class
class Stripe(PaymentProvider):
    def charge(self, amount, token): ...
    def refund(self, transaction_id): ... # les 2 fournies → instanciable ✔
```

L'erreur surgit à l'**instanciation** (tôt, claire) — pas au fond d'un appel raté en production.

ABC : le pattern *Template Method* intégré

L'ABC fixe le **squelette** de l'algorithme, les sous-classes remplissent les **trous** — premier design pattern officiel (chap. 7) :

```
class ETLJob(ABC):
    def run(self):
        # le SQUELETTE – final de fait, commun à tous
        raw = self.extract()
        clean = self.transform(raw)
        self.load(clean)
        self.notify(len(clean))

    @abstractmethod
    def extract(self) -> list: ...      # les TROUS à remplir
    @abstractmethod
    def transform(self, raw: list) -> list: ...
    @abstractmethod
    def load(self, data: list) -> None: ...

    def notify(self, count: int):
        # HOOK : défaut fourni, surcharge possible
        log.info("%d lignes chargées", count)

class SalesETL(ETLJob):
    def extract(self): return read_csv("sales.csv")
    def transform(self, raw): return [normalize(r) for r in raw]
    def load(self, data): bulk_insert("sales", data)

SalesETL().run()      # l'ORCHESTRATION est garantie identique pour tous les jobs
```

`@abstractmethod` se combine : `@property + @abstractmethod`, `@classmethod + @abstractmethod` ...

collections.abc : des méthodes gratuites par contrat

Héritez d'une ABC de la stdlib : fournissez le **minimum**, recevez **tout le reste** :

```
from collections.abc import Mapping

class FrozenConfig(Mapping):          # un dict immutable, en 12 lignes
    def __init__(self, **kwargs):
        self._data = dict(kwargs)

    def __getitem__(self, key): return self._data[key]      # ← les 3 requis
    def __len__(self): return len(self._data)
    def __iter__(self): return iter(self._data)

cfg = FrozenConfig(host="localhost", port=8080)
cfg["host"]          # fourni par VOUS
cfg.get("timeout", 30) # GRATUIT (Mapping)
"port" in cfg         # GRATUIT
cfg.keys(), cfg.items() # GRATUITS
cfg == {"host": "localhost", "port": 8080} # GRATUIT - même l'égalité !
```

Vous fournissez	ABC	Vous recevez
<code>__getitem__</code> , <code>__len__</code> , <code>__iter__</code>	Mapping	<code>get</code> , <code>keys</code> , <code>items</code> , <code>values</code> , <code>==</code> , <code>in</code>
<code>__getitem__</code> , <code>__len__</code>	Sequence	<code>in</code> , <code>iter</code> , <code>index</code> , <code>count</code> , <code>reversed</code>
<code>__contains__</code> , <code>__iter__</code> , <code>__len__</code>	Set	<code>in</code> , <code>len</code> , <code>__and__</code> , <code>__or__</code> , <code>__xor__</code> , <code>__sub__</code>

C'est l'itérable « réutilisable » du chap. 5, industrialisé.

Protocol : le duck typing vérifié statiquement

L'alternative **structurelle** aux ABC — pas d'héritage requis, la **forme** suffit :

```
from typing import Protocol, runtime_checkable

@runtime_checkable
class Exportable(Protocol):
    def to_csv(self) -> str: ...      # le contrat = juste la signature

class Report:
    def to_csv(self) -> str: return "..." # AUCUN import, AUCUN héritage !

def export(items: list[Exportable]) -> None:    # mypy vérifie STRUCTURELLEMENT
    for item in items:
        write(item.to_csv())

export([Report()])
isinstance(Report(), Exportable)              # ✅ mypy : Report a to_csv() -> conforme
                                              # True (grâce à @runtime_checkable)
```

Protocol quand...

- Le contrat porte sur du code **tiers** (non modifiable)
- Découplage maximal : zéro dépendance vers l'interface
- Le duck typing du chap. 2, enfin **outillé**

ABC quand...

- Vous voulez **forcer** l'erreur à l'instanciation
- Vous fournissez des **implémentations partielles** (template method, mixins de `collections.abc`)
- La hiérarchie est sous votre contrôle

Héritage ou composition ? L'arbitrage

« *Favor object composition over class inheritance* » — Design Patterns (GoF), 1994.

✗ L'héritage par commodité

```
class UserService(Database, Logger,  
                  Cache, Mailer):  
    ...
```

- `UserService` **est-il** une Database ? Non !
- Couplage : tout changement de base se propage
- Explosion combinatoire des sous-classes
- MRO illisible, tests lourds

✓ La composition : « a un »

```
class UserService:  
    def __init__(self, db: Repository,  
                  logger: Logger,  
                  cache: Cache):  
        self.db = db          # injecté →  
        self.logger = logger  # remplaçable,  
        self.cache = cache    # mockable  
  
    def get_user(self, uid):  
        if (u := self.cache.get(uid)):  
            return u  
        return self.db.fetch(uid)
```

Typez les dépendances avec des **Protocol/ABC** → architecture testable (injection de dépendances).

Grille de décision : « **est un** » + substituable partout (Liskov) → héritage. « **a un** » / « **utilise** » → composition. Capacité transverse sans état → mixin. Doute → composition.

Bonnes pratiques — synthèse du chapitre

Encapsulation :

- Attribut public d'abord ; `property` quand la validation/le calcul arrive — l'API ne bouge pas
- Property = rapide et prévisible ; coûteux → méthode explicite ou `cached_property`
- Validation répétée sur plusieurs champs → **descripteur** avec `__set_name__`

Héritage multiple :

- Lisez le `__mro__` au moindre doute ; `super()` = « suivant dans le MRO », pas « parent »
- `__init__` coopératifs : keyword-only + prélever + relayer `**kwargs` + `super()` partout
- Mixins : sans état, une capacité, à gauche de la base

Architecture :

- ABC pour les contrats internes avec implémentation partielle ; `Protocol` pour découpler / typer du code tiers
- `collections.abc` avant de réécrire un conteneur
- Composition par défaut, héritage pour les vraies relations « est un »

Travaux pratiques 1/2

TP 6.1 — Implémentation de descripteurs (45 min)

1. Descripteur `Bounded(min, max)` : validation d'intervalle réutilisable — `temperature = Bounded(-50, 60)` ; utilisez `__set_name__`
2. Descripteur `TypedField(type_)` : vérifie le type à l'écriture — puis fusionnez : `Bounded` accepte un paramètre `type_` . Comparez avec votre validateur par annotations du TP 2.3 : quelle approche préférez-vous ?
3. Descripteur `Audited()` : journalise chaque lecture/écriture (qui, quoi, quand) dans une liste de classe — inspectez l'historique après quelques manipulations
4. Bonus : `LazyField(factory)` — non-data descriptor qui calcule au 1er accès puis écrit dans `obj.__dict__` (le mécanisme de `cached_property` !)

TP 6.2 — Exercices d'héritage multiple (45 min)

1. Tracez **sur papier** le MRO de `class D(B, C)` avec `B(A)` , `C(A)` — vérifiez avec `D.__mro__` , puis cassez-le : trouvez une déclaration qui lève `TypeError: Cannot create a consistent MRO`
2. Implémentez la chaîne `SecureBackup(Encrypted, Compressed)` du cours avec des `save()` coopératifs qui transforment réellement des `bytes` (`zlib` , XOR simple) — vérifiez que l'ordre des bases change le résultat
3. Construisez `Service(Persisted, Audited)` avec les `__init__` coopératifs `**kwargs` — ajoutez un 3e mixin `Throttled(max_rps=...)` sans toucher aux autres
4. Écrivez un `ReprMixin` (un `__repr__` générique listant les attributs publics) et appliquez-le à vos classes des TP précédents

Travaux pratiques 2/2

TP 6.3 — Architecture avec classes abstraites (45 min)

1. Concevez `Repository(ABC)` : `get` , `add` , `delete` abstraites + `get_or_default` concrète — implémentez `InMemoryRepository` (dict) et `JSONFileRepository`
2. Le métier (`OrderService`) ne dépend que de `Repository` : montrez qu'on échange les implémentations **sans modifier le service**, et testez-le avec l'in-memory (zéro mock !)
3. Refaites le contrat en `Protocol` : qu'est-ce qui change dans les imports et le couplage ? Faites tourner mypy sur les deux versions
4. Bonus : `FrozenConfig(Mapping)` du cours + une `HistoryList(Sequence)` qui garde l'historique de ses lectures

Livrables : code + tests pytest + le schéma MRO du TP 6.2.1 ; mypy sans erreur sur le TP 6.3.3.

Chapitre 7

Design patterns

Python Perfectionnement — Jour 2

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Reconnaître et nommer** les patterns classiques (GoF) — un vocabulaire d'équipe
- **Implémenter** les patterns créationnels : Singleton, Factory, Builder
- **Implémenter** les patterns structurels : Adapter, Façade, Proxy, Composite
- **Implémenter** les patterns comportementaux : Strategy, Observer, Command
- **Adapter** chaque pattern à Python — souvent **plus simple** que la version Java/C++
- **Éviter** la sur-ingénierie : le pattern est un outil, pas un objectif

Travaux pratiques : patterns créationnels, patterns structurels et comportementaux.

Plusieurs patterns sont **déjà acquis** : registre (chap. 3), pipeline (chap. 4), Iterator (chap. 5), Template Method (chap. 6) — nous allons les nommer et compléter la panoplie.

Design patterns : de quoi parle-t-on ?

Un **design pattern** = une solution éprouvée et **nommée** à un problème de conception récurrent. Le catalogue de référence : *Design Patterns* (GoF — Gamma, Helm, Johnson, Vlissides, 1994).

Famille	Question résolue	Exemples
Créationnels	<i>comment</i> créer les objets ?	Singleton, Factory, Builder

La spécificité Python : des patterns qui fondent

Les patterns GoF compensent les **rigidités** de Java/C++ 1994. Python en dissout plusieurs :

- Fonctions de 1re classe (chap. 3) → **Strategy** et **Command** sans classes
- `yield` (chap. 5) → **Iterator** natif • modules → **Singleton** gratuit
- Duck typing / `Protocol` (chap. 6) → **Adapter** souvent inutile

La valeur durable : le *vocabulaire partagé* (« mets une façade devant ce sous-système ») et l'*intention* — plus que la lettre de l'implémentation.

Singleton : une seule instance... vraiment ?

Intention : garantir une instance unique accessible globalement (config, pool de connexions).

Version `__new__`

```
class Settings:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance

Settings() is Settings()    # True
```

⚠ `__init__` re-exécuté à chaque appel ; héritage piégeux ;
thread-safety à ajouter (verrou — chap. 12).

✓ La version pythonique : le module

```
# settings.py – un module N'EST importé qu'UNE fois
host = "localhost"
db_pool = create_pool()

# ailleurs :
from settings import db_pool
```

Le cache des imports (`sys.modules`) **est** un singleton industriel :
paresseux, thread-safe à l'import, zéro code.

Singleton : la critique

Ou : l'instance unique explicite

```
class Cache: ...  
cache = Cache()      # exportée par le module
```

Critique (clean architecture) : un singleton = un **état global déguisé** → tests couplés, dépendances cachées. Préférez l'**injection** (chap. 6) ; le singleton reste acceptable pour la config en lecture seule et les ressources réellement uniques.

Factory : découpler la création de l'usage

Intention : le code client demande « un parseur pour ce fichier » sans connaître les classes concrètes.

```
class Parser(ABC):
    @abstractmethod
    def parse(self, path: Path) -> List[dict]: ...

class CSVParser(Parser): ...
class JSONParser(Parser): ...
class XMLParser(Parser): ...
```

```
# La factory pythonique : un REGISTRE (chap. 3) - pas de cascade if/elif
PARSERS: dict[str, type[Parser]] = {}

def register_parser(*extensions):
    def decorator(cls):
        for ext in extensions:
            PARSERS[ext] = cls
        return cls
    return decorator

@register_parser(".csv", ".tsv")
class CSVParser(Parser): ...

def make_parser(path: Path) -> Parser:
    try:
        return PARSERS[path.suffix]()
    except KeyError:
        raise ValueError(f"Format non g     : {path.suffix} "
                        f"(connus : {sorted(PARSERS)})")
```

Ajouter un format = **une classe d      **, z     modification du c       (*open/closed*, et le pattern plugins du chap. 2 en est la version distribu    ).

Factory : les variantes utiles

Les classes sont des objets (chap. 2)

La « factory » la plus simple : passer la classe elle-même —

```
def load_all(path, item_cls: type[Item]):
    return [item_cls(**row) for row in read(path)]

load_all("data.csv", PremiumItem)  # la classe
                                  # EST la factory
```

Constructeurs alternatifs

`@classmethod` (chap. 6) = factory interne :

```
Invoice.from_json(payload)
datetime.fromtimestamp(ts)
```

Règle : commencez par le dict-registre ou le `classmethod` ; l'Abstract Factory ne se justifie que pour des **familles** d'objets cohérentes (thèmes UI, drivers BDD complets...).

Abstract Factory : la famille cohérente

Quand plusieurs objets doivent être assortis :

```
class UIFactory(Protocol):
    def button(self) -> Button: ...
    def dialog(self) -> Dialog: ...

class DarkUI:  # famille "dark"
    def button(self): return DarkButton()
    def dialog(self): return DarkDialog()

def build_form(ui: UIFactory):
    return [ui.button(), ui.dialog()]  # toujours assortis ✓
```

En Python, souvent : un simple **module** ou un dict de callables par famille.

Builder : construire pas à pas 1/2

Intention : construire un objet complexe par étapes, sans `__init__` à 15 paramètres.

Le Builder fluide

```
class QueryBuilder:
    def __init__(self, table: str):
        self._table, self._wheres = table, []
        self._order, self._limit = None, None

    def where(self, cond: str):
        self._wheres.append(cond)
        return self          # ← chaînage

    def order_by(self, col: str):
        self._order = col
        return self

    def limit(self, n: int):
        self._limit = n
        return self

    def build(self) -> str:
        sql = f"SELECT * FROM {self._table}"
        if self._wheres:
            sql += " WHERE " + " AND ".join(self._wheres)
        ...
        return sql

q = (QueryBuilder("orders")
     .where("amount > 100")
     .where("status = 'paid'")
     .order_by("created_at")
     .limit(10)
     .build())
```

Builder : construire pas à pas 2/2

L'alternative pythonique d'abord !

Les **arguments nommés + défauts** couvrent 90 % des cas Java du Builder :

```
@dataclass(frozen=True)
class HttpClient:
    base_url: str
    timeout: float = 30.0
    retries: int = 3
    verify_ssl: bool = True
    headers: dict = field(default_factory=dict)

client = HttpClient("https://api.x.io",
                   timeout=5, retries=0)
```

Le Builder **garde sa valeur** quand :

- la construction est **incrémentale** (requêtes, documents, pipelines)
- des **validations croisées** s'appliquent au `build()` final
- l'objet final doit être **immutable** (le builder mutable, le produit non)

Exemples réels : `pandas` chaining, SQLAlchemy `select()`, `pathlib` (`p / "a" / "b"`).

Adapter : faire coïncider deux interfaces

Intention : utiliser une classe existante là où une **autre interface** est attendue — sans la modifier.

```
# Notre application attend ce contrat (chap. 6) :
class Notifier(Protocol):
    def send(self, recipient: str, message: str) -> None: ...

# La lib tierce a une AUTRE interface (non modifiable) :
class SlackSDK:
    def post_message(self, *, channel: str, blocks: list[dict]) -> dict: ...
```

```
class SlackNotifier:
    def __init__(self, sdk: SlackSDK):          # L'ADAPTER : traduit notre contrat → le leur
        self._sdk = sdk                        # composition (chap. 6), pas héritage

    def send(self, recipient: str, message: str) -> None:
        self._sdk.post_message(
            channel=f"#{recipient}",
            blocks=[{"type": "section", "text": {"type": "mrkdwn", "text": message}}],
        )

def alert_oncall(notifier: Notifier):          # le métier ne connaît QUE Notifier
    notifier.send("astreinte", "Disque plein sur prod-db-01")

alert_oncall(SlackNotifier(SlackSDK()))      # ✅ - demain : TeamsNotifier, SMSNotifier...
```

L'adapter est la **couche anticorruption** : le vocabulaire de la lib tierce ne contamine jamais le métier. Grâce au duck typing, pas de classe de base commune — juste la bonne **forme**.

Decorator (pattern) : empiler des responsabilités

Intention : ajouter des comportements à un objet **sans** modifier sa classe — par emboîtement.

```
class DataSource(Protocol):
    def read(self) -> bytes: ...

class FileSource:
    def __init__(self, path): self.path = path
    def read(self) -> bytes: return Path(self.path).read_bytes()

class GzipDecorator:
    def __init__(self, wrapped: DataSource):      # enveloppe un DataSource...
        self._wrapped = wrapped
    def read(self) -> bytes:
        return gzip.decompress(self._wrapped.read())

class CacheDecorator:
    def __init__(self, wrapped: DataSource):
        self._wrapped, self._cache = wrapped, None
    def read(self) -> bytes:
        if self._cache is None:
            self._cache = self._wrapped.read()
        return self._cache

source = CacheDecorator(GzipDecorator(FileSource("data.gz"))) # emboîtement à la demande
```

Pattern Decorator (objets) : composer des comportements à l'exécution, instance par instance.

`@décorateur` (chap. 3) : même intention sur des **fonctions**, appliqué à la **définition**. Deux outils, une idée : envelopper.

Façade : simplifier un sous-système

Intention : offrir un point d'entrée **simple** à un ensemble complexe — sans en interdire l'accès fin.

```
# Sans façade : chaque appelant doit connaître la chorégraphie complète
ffmpeg = FfmpegWrapper(); s3 = S3Client(); db = MediaDB(); cdn = CDNWrapper()
raw = ffmpeg.probe(path); ffmpeg.transcode(path, "h264", preset="fast")
thumb = ffmpeg.extract_frame(path, at="00:00:03")
s3.upload(...); db.register(...); cdn.purge(...)      # ...et dans le bon ordre !
```

```
class MediaPublisher:
    # LA FAÇADE - l'API que les gens veulent
    def __init__(self, ffmpeg, s3, db, cdn):
        # injectées → testable (chap. 6)
        self._ffmpeg, self._s3, self._db, self._cdn = ffmpeg, s3, db, cdn

    def publish(self, path: Path) -> MediaInfo:
        """Transcode, héberge, référence et invalide le cache - en un appel."""
        meta = self._ffmpeg.probe(path)
        video = self._ffmpeg.transcode(path, "h264", preset="fast")
        thumb = self._ffmpeg.extract_frame(path, at="00:00:03")
        url = self._s3.upload(video, thumb)
        info = self._db.register(url, meta)
        self._cdn.purge(url)
        return info
```

- La façade **orchestre**, les sous-systèmes restent accessibles pour les besoins pointus
- Vous en utilisez déjà : `requests.get(...)` (façade sur `urllib3`), `subprocess.run(...)`, `shutil`
- C'est la « coquille impérative » du chap. 4, vue côté objet

Proxy : un intermédiaire qui contrôle l'accès

Intention : même interface que l'objet réel, mais avec un **contrôle** au milieu — paresse, droits, cache, distance.

```
class LazyDBConnection:
    """Proxy VIRTUEL : la vraie connexion n'existe qu'au premier usage."""
    def __init__(self, dsn: str):
        self._dsn, self._real = dsn, None

    def __getattr__(self, name):
        # chap. 2 : appelé si introuvable
        if self._real is None:
            print("connexion établie (lazy)")
            self._real = psycopg.connect(self._dsn)
        return getattr(self._real, name) # délègue TOUT au réel

conn = LazyDBConnection(DSN) # instantané - aucune connexion
conn.execute("SELECT 1")    # ← c'est ICI que la connexion se crée
```

Variante	Contrôle ajouté	Exemple réel
Virtuel	création paresseuse	ORM : relations lazy-loadées
Protection	vérification de droits	obj accessible selon le rôle
Distant	appels réseau cachés	stubs gRPC, <code>xmlrpc.client</code>
Cache/mémo	évite les recalculs	le <code>CacheDecorator</code>  frontière floue

`__getattr__` rend le proxy **générique** : ~10 lignes pour proxifier n'importe quoi (mocks de test, `unittest.mock.MagicMock` est un proxy enregistreur !).

Composite : traiter l'arbre comme la feuille

Intention : manipuler uniformément un élément **simple** et un **groupe** d'éléments.

```
class Task(Protocol):
    def duration(self) -> float: ...

@dataclass
class Step:                                # la FEUILLE
    name: str
    hours: float
    def duration(self) -> float:
        return self.hours

@dataclass
class Phase:                               # le COMPOSITE : contient des Task...
    name: str
    children: list[Task] = field(default_factory=list)
    def duration(self) -> float:           # ...et EST une Task
        return sum(child.duration() for child in self.children)

project = Phase("Refonte", [
    Step("Audit", 16),
    Phase("Dev", [Step("API", 40), Step("Front", 60)]),    # imbrication libre
    Step("Recette", 24),
])
project.duration()    # 140.0 - récursion invisible pour l'appelant
```

- Le client appelle `duration()` **sans savoir** s'il tient une feuille ou un sous-arbre
- Partout où il y a un arbre : systèmes de fichiers, UI, orgas, expressions (AST), menus
- Le parcours se fait en générateur récursif `yield from` (chap. 5) pour les gros arbres

Strategy : l'algorithme interchangeable

Intention : faire varier un algorithme indépendamment du code qui l'utilise.

Version GoF (classes)

```
class ShippingStrategy(ABC):
    @abstractmethod
    def cost(self, order) -> float: ...

class Express(ShippingStrategy):
    def cost(self, order):
        return 15 + order.weight * 0.8

class Standard(ShippingStrategy):
    def cost(self, order):
        return 5 + order.weight * 0.2

class Checkout:
    def __init__(self, shipping: ShippingStrategy):
        self.shipping = shipping
    def total(self, order):
        return order.amount + self.shipping.cost(order)
```

Justifiée si la stratégie a état + plusieurs méthodes.

Version pythonique (fonctions)

Une stratégie = une fonction (chap. 3/4) :

```
def express(order) -> float:
    return 15 + order.weight * 0.8

def standard(order) -> float:
    return 5 + order.weight * 0.2

class Checkout:
    def __init__(self, shipping: Callable[[Order], float]):
        self.shipping = shipping

Checkout(express)
Checkout(partial(flat_rate, price=7.9)) # paramétrée
Checkout(lambda o: 0.0) # promo !
```

Avec le registre du chap. 3 :

```
@shipping_method("express")
def express(order): ...
checkout = Checkout(SHIPPING[user_choice])
```

Observer : publier sans connaître ses abonnés

Intention : quand un objet change, **notifier** tous les intéressés — sans couplage direct.

```
class EventBus:
    def __init__(self):
        self._subscribers: defaultdict[str, list[Callable]] = defaultdict(list)

    def subscribe(self, event_type: str, handler: Callable) -> Callable:
        self._subscribers[event_type].append(handler)
        return lambda: self._subscribers[event_type].remove(handler) # désabonnement !

    def publish(self, event_type: str, **payload):
        for handler in list(self._subscribers[event_type]): # copie : un handler peut
            try:                                             # se désabonner en route
                handler(**payload)
            except Exception:
                log.exception("handler %s a échoué", handler) # un échec ≠ tous

bus = EventBus()
bus.subscribe("order.paid", send_invoice)
bus.subscribe("order.paid", update_stock) # N réactions indépendantes
bus.subscribe("order.paid", notify_warehouse)

bus.publish("order.paid", order_id=42, amount=99.9) # l'émetteur ignore QUI écoute
```

- Les détails qui comptent en prod : **désabonnement**, **isolation des erreurs**, copie de la liste
- RxPY (chap. 3) = Observer industrialisé (flux + opérateurs) ; les *signals* Django, Qt, GameDev : partout
- Version **distribuée** au chap. 13 : le message broker est un Observer inter-processus

Command : l'action devient un objet

Intention : encapsuler une action et son contexte pour la stocker, la rejouer, l'annuler.

```
class Command(Protocol):
    def execute(self) -> None: ...
    def undo(self) -> None: ...

@dataclass
class AddItem:
    doc: Document
    item: str
    def execute(self): self.doc.items.append(self.item)
    def undo(self): self.doc.items.remove(self.item)

@dataclass
class RenameDoc:
    doc: Document
    new: str
    old: str = ""
    def execute(self):
        self.old, self.doc.name = self.doc.name, self.new # mémorise pour undo
    def undo(self):
        self.doc.name = self.old

class History:
    # l'invocateur générique
    def __init__(self): self._done: list[Command] = []
    def run(self, cmd: Command):
        cmd.execute()
        self._done.append(cmd)
    def undo_last(self):
        if self._done:
            self._done.pop().undo()
```

Undo/redo, files de tâches (Celery sérialise des *commands* — chap. 13), transactions, macros, replay d'événements. Sans besoin d' `undo`, une simple `partial` (chap. 3) suffit comme commande.

Les patterns que Python vous offre déjà

Pattern GoF	En Python, c'est...	Vu au
Iterator	<code>__iter__ / yield</code> — natif dans le langage	chap. 5
Template Method	ABC + méthodes abstraites (<code>ETLJob</code>)	chap. 6
Prototype	<code>copy.copy()</code> / <code>copy.deepcopy()</code> / <code>dataclasses.replace()</code>	chap. 1
Flyweight	interning des <code>int / str</code> , <code>sys.intern</code> , <code>lru_cache</code>	chap. 1, 3
Chain of Responsibility	une liste de handlers qu'on parcourt — ou middlewares	—
Memento	<code>vars(obj).copy()</code> , <code>pickle</code> , <code>dataclasses.asdict</code>	chap. 1, 2
Visitor	<code>singledispatch</code> !	chap. 3

```
# Chain of Responsibility en 6 lignes — une liste de fonctions (chap. 4)
def handle(request, handlers: list[Callable]):
    for handler in handlers:
        if (response := handler(request)) is not None:
            return response          # le 1er qui sait répondre gagne
    raise UnhandledRequest(request)
```

Leçon générale : en Python, le pattern se présente souvent comme « *une structure de données + des fonctions* » plutôt que comme une hiérarchie de classes.

Anti-patterns : quand le remède devient le mal

La *pattern fever*

```
# Pour lire UN fichier de config :
class ConfigReaderFactoryProvider:
    def get_factory(self) -> ConfigReaderFactory: ...
class ConfigReaderFactory:
    def create_reader(self) -> ConfigReader: ...
class ConfigReader:
    def read(self) -> Config: ...

# Python :
config = tomlib.load(open("config.toml", "rb"))
```

Les signes : plus de classes que de logique, des noms en ...

Manager / ...Provider / ...Impl, de l'indirection sans **variation** réelle.

Les règles de sobriété

1. **YAGNI** : le pattern se mérite — il faut **au moins 2 variantes réelles** pour une Strategy/Factory
2. Commencez **simple** (fonction, dict), refactorisez **vers** le pattern quand la douleur apparaît
3. Le pattern est un **vocabulaire**, pas une cible : « ceci est un proxy » aide ; « il me faut un proxy » inquiète
4. En Python, cherchez d'abord : une fonction ? un dict ? un module ? un générateur ? une dataclass ?

Simple is better than complex. Flat is better than nested.
— le Zen s'applique aussi aux patterns.

Bonnes pratiques — synthèse du chapitre

La carte mentale des patterns en Python :

Besoin	Réflexe Python	Pattern sous-jacent
Créer selon un critère	dict-registre + décorateur	Factory
Construction complexe	dataclass + kwargs ; builder si incrémental	Builder
Instance unique	module / instance exportée	Singleton
Interface incompatible	classe de traduction fine (composition)	Adapter
Sous-système complexe	une classe d'orchestration injectée	Façade
Contrôler l'accès	<code>__getattr__</code> qui délègue	Proxy

Les trois questions avant d'introduire un pattern :

1. Ai-je **deux variations réelles** (pas hypothétiques) à gérer ?
2. La version « fonction + dict + module » est-elle vraiment insuffisante ?
3. Le nom du pattern rendra-t-il le code **plus lisible** pour l'équipe ?

Travaux pratiques 1/2

TP 7.1 — Patterns créationnels (45 min)

1. **Factory-registre** : système d'export multi-formats (`csv` , `json` , `xlsx`) — `@register_exporter(".csv")` + `make_exporter(path)` ; ajoutez le format `md` pour vérifier l'open/closed (zéro ligne modifiée dans le cœur)
2. **Builder** : `ReportBuilder` fluide (`.title(...).section(...).table(...).chart(...).build()`) produisant un rapport **immutable** (`@dataclass(frozen=True)`) — avec validation croisée au `build()` (un rapport sans titre ni section est rejeté)
3. **Singleton critique** : implémentez la config en 3 versions (module, `__new__` , instance exportée), puis écrivez le test qui devient **pénible** avec le singleton `__new__` (état partagé entre tests) — concluez

TP 7.2 — Patterns structurels (45 min)

1. **Adapter** : votre application logue via `AuditSink(Protocol)` (`record(event: dict)`) — adaptez successivement `logging` , un fichier JSONL, et une API HTTP factice, sans toucher au métier
2. **Proxy** : `RateLimitedProxy(service, max_per_min)` générique à base de `__getattr__` + la logique `deque` du TP 3.1 — appliquez-le à deux services différents pour prouver la généricité
3. **Composite** : modélisez un devis (lignes, sections, sous-sections) avec `total()` uniforme + un parcours `walk()` en `yield from` qui aplatit l'arbre

Travaux pratiques 2/2

TP 7.3 — Patterns comportementaux (45 min)

1. **Strategy** : moteur de calcul de frais de port — 3 stratégies-fonctions + un registre ; ajoutez une stratégie « promo » composée de deux autres (`min` des deux)
2. **Observer** : `EventBus` complet (désabonnement, isolation des erreurs, wildcard `"order.*"` en bonus) — branchez 3 handlers sur un événement `order.paid` et testez qu'un handler défaillant n'empêche pas les autres
3. **Command** : mini-éditeur de liste avec historique undo/redo complet (deux piles) — implémentez `AddItem` , `RemoveItem` , `RenameItem` et une macro (Composite de Commands !)

Livrables : code + tests pytest. Pour chaque TP : 2 lignes en commentaire — *quel problème ce pattern résout ici, et qu'aurait coûté son absence.*

Chapitre 8

Métaprogrammation avancée

Python Perfectionnement — Jour 3

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Expliquer** comment Python crée les classes — et intercepter cette création
- **Écrire** des métaclasse : validation, enregistrement, contrôle d'instanciation
- **Choisir** l'outil le plus simple : `__init_subclass__` couvre 90 % des besoins
- **Combiner** descripteurs et métaclasse : le cœur d'un mini-ORM déclaratif
- **Personnaliser** le système d'import : `sys.meta_path`, finders et loaders, `__getattr__` de module

Travaux pratiques : exploration des métaclasse, protocol descriptors avancés, customisation d'imports.

L'aboutissement de l'arc ouvert au **chap. 2** (`type` à 3 arguments) et poursuivi au **chap. 6** (descripteurs).

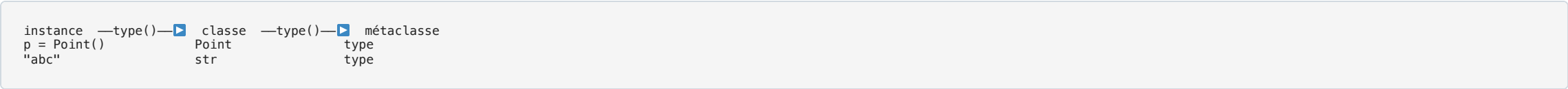
Rappel fondateur : les classes sont des objets

Au chapitre 2, nous avons vu que `class` n'est que du sucre pour `type(name, bases, namespace)`.

```
class Point:
    pass

type(Point)      # <class 'type'> - la classe de la classe !
type(int), type(str)  # (<class 'type'>, <class 'type'>)
type(type)       # <class 'type'> - type est sa propre classe 🤖
```

La chaîne complète :



Relation	Mécanisme
une classe fabrique ses instances	<code>Point(...)</code> → <code>Point.__call__</code>
une métaclasses fabrique ses classes	<code>class Point:</code> → <code>type(...)</code>

Une **métaclasses** = une sous-classe de `type` qui intercepte la fabrication des classes. Si on peut sous-classer `type` ... on peut personnaliser la création de toute classe.

Ce qui se passe vraiment à `class Model`:

```
class Model(Base, metaclass=Meta, debug=True):    # ← l'instruction...
    table = "users"
    def save(self): ...
```

...déclenche **quatre étapes** précises :

```
1. Python détermine la métaclasse      Meta (sinon : celle de Base, sinon type)
2. namespace = Meta.__prepare__(...)    le dict qui recevra le corps de la classe
3. exécution du CORPS de la classe      remplit namespace : {'table': ..., 'save': ...}
4. Model = Meta('Model', (Base,), namespace, debug=True) ← l'appel final !
```

L'étape 4 est un **appel ordinaire** — décomposable comme tout appel (chap. 6) :

```
Meta.__call__(...)    # hérité de type, orchestre :
→ Meta.__new__(mcls, name, bases, namespace, **kwargs)    # CRÉE l'objet classe
→ Meta.__init__(cls, name, bases, namespace, **kwargs)    # l'initialise
```

Donc : surcharger `Meta.__new__` = modifier/valider/enrichir **chaque classe** qui passe par la métaclasse — au moment de sa définition, une seule fois, à l'import.

Première métaclasse : valider les classes à la définition

Cas d'usage : imposer des conventions d'équipe — détectées à l'import, pas en production.

```
class APIHandlerMeta(type):
    def __new__(mcls, name, bases, namespace, **kwargs):
        cls = super().__new__(mcls, name, bases, namespace, **kwargs)

        if not bases:                                # la classe RACINE est exemptée
            return cls

        # Règle 1 : toute méthode publique doit être documentée
        for attr, obj in namespace.items():
            if callable(obj) and not attr.startswith("_") and not obj.__doc__:
                raise TypeError(f"{name}.{attr} : docstring obligatoire")

        # Règle 2 : l'attribut route est requis et bien formé
        route = namespace.get("route")
        if not (isinstance(route, str) and route.startswith("/")):
            raise TypeError(f"{name} : attribut 'route' manquant ou invalide")

        return cls

class APIHandler(metaclass=APIHandlerMeta):
    pass

class UserHandler(APIHandler):
    route = "/users"
    def get(self):
        return db.fetch_users()

# ✱ TypeError à l'IMPORT :
# UserHandler.get : docstring obligatoire
```

L'erreur éclate **au chargement du module** — impossible de déployer une classe non conforme.

Métaclasse : l'auto-enregistrement

Le registre du chap. 3, version **automatique** — hériter suffit à s'enregistrer :

```
class PluginMeta(type):
    registry: dict[str, type] = {}

    def __new__(mcls, name, bases, namespace, **kwargs):
        cls = super().__new__(mcls, name, bases, namespace, **kwargs)
        if bases:
            # pas la racine
            key = namespace.get("format_name", name.lower())
            mcls.registry[key] = cls
        return cls

class Exporter(metaclass=PluginMeta):
    pass

class CSVExporter(Exporter):
    format_name = "csv"
    # AUCUN décorateur, AUCUN appel –
    # la définition EST l'enregistrement

class JSONExporter(Exporter):
    format_name = "json"

PluginMeta.registry      # {'csv': CSVExporter, 'json': JSONExporter}

def export(data, fmt: str):
    return PluginMeta.registry[fmt]().run(data)
    # la Factory du chap. 7, auto-alimentée
```

Combiné à la découverte de modules du chap. 2 (`pkgutil` + `importlib`) : déposer un fichier définissant une sous-classe = plugin opérationnel. C'est le mécanisme des `Model` Django et des handlers Tornado.

`__call__` de métaclassse : contrôler l'instanciation

`Instance = Classe(...)` passe par `Meta.__call__` — on peut donc intercepter chaque instanciation :

```
class SingletonMeta(type):
    def __init__(cls, name, bases, namespace, **kwargs):
        super().__init__(name, bases, namespace, **kwargs)
        cls._instance = None

    def __call__(cls, *args, **kwargs):
        # appelé pour CHAQUE Classe(...)
        if cls._instance is None:
            cls._instance = super().__call__(*args, **kwargs) # __new__ + __init__
        return cls._instance

class ConnectionPool(metaclass=SingletonMeta):
    def __init__(self, dsn):
        self.dsn = dsn

ConnectionPool("pg://a") is ConnectionPool("pg://b") # True — et __init__ ne
# tourne qu'UNE fois ✓
```

Plus propre que le `__new__` du chap. 7 (pas de ré-init), **héritable** (chaque sous-classe a SON instance)...

Autres usages de <code>Meta.__call__</code>	Exemple
Cache d'instances (Flyweight, chap. 7)	<code>Color("red") is Color("red")</code>
Interdire l'instanciation directe	forcer le passage par une factory
Post-validation automatique	appeler <code>self.validate()</code> après <code>__init__</code>

...mais la critique du chap. 7 reste : un singleton est un état global. **Sachez le faire, hésitez à le faire.**

`__prepare__` : choisir le dict du corps de classe

L'étape 2 de la création : fournir un **espace de noms sur mesure** où le corps s'exécute :

```
class OrderedEnumMeta(type):
    @classmethod
    def __prepare__(mcls, name, bases, **kwargs):
        return CollectingDict() # à la place d'un dict ordinaire

class CollectingDict(dict):
    def __init__(self):
        super().__init__()
        self.member_names = []

    def __setitem__(self, key, value):
        # espionne CHAQUE affectation
        if not key.startswith("_") and key not in self:
            self.member_names.append(key) # l'ORDRE de déclaration est capturé
        super().__setitem__(key, value)
```

```
class Status(metaclass=OrderedEnumMeta):
    DRAFT = auto() # ① le corps s'exécute DANS CollectingDict
    REVIEW = auto() # ② chaque ligne passe par __setitem__
    PUBLISHED = auto() # ③ member_names = ['DRAFT', 'REVIEW', 'PUBLISHED']
```

- C'est ainsi qu' `enum.Enum` détecte les doublons et l'ordre, et que les ORM ordonnent leurs colonnes
- Possibilités exotiques : un namespace qui **autorise les redéfinitions** (surcharge de méthodes par signature), un DSL où chaque nom est pré-rempli...
- Depuis 3.7 le `dict` standard est ordonné (chap. 1) → `__prepare__` n'est utile que pour un comportement **actif** comme ci-dessus

L'alternative moderne : `__init_subclass__`

90 % des métaclasses peuvent disparaître — depuis Python 3.6, une classe peut réagir elle-même à ses sous-classes :

```
class Exporter:
    registry: dict[str, type] = {}

    def __init_subclass__(cls, /, format_name: str | None = None, **kwargs):
        super().__init_subclass__(**kwargs) # coopératif (chap. 6 !)
        key = format_name or cls.__name__.lower()
        Exporter.registry[key] = cls # auto-enregistrement
        if not cls.__doc__:
            raise TypeError(f"{cls.__name__} : docstring obligatoire") # validation

class CSVExporter(Exporter, format_name="csv"): # paramètre DANS l'héritage !
    """Export CSV."""
```

	Métaclasse	<code>__init_subclass__</code>
Syntaxe	<code>metaclass=Meta</code>	simple méthode de la base
Peut modifier la création (<code>__new__</code> , <code>__prepare__</code> , <code>__call__</code>)	✓	✗ — la classe existe déjà
Conflits en héritage multiple	⚠ fréquents	aucun
Lisibilité pour l'équipe	faible	bonne

Règle de décision : enregistrement, validation, injection d'attributs → `__init_subclass__`. Contrôle de l'instanciation, namespace custom, modification structurelle → métaclasse. (Et `__set_name__`, chap. 6, a la même mission pour les descripteurs.)

Le conflit de métaclasses

Le piège classique en héritage multiple — deux bases, deux métaclasses **incompatibles** :

```
class Meta1(type): ...
class Meta2(type): ...
class A(metaclass=Meta1): ...
class B(metaclass=Meta2): ...

class C(A, B): ...
# ✖ TypeError: metaclass conflict: the metaclass of a derived class
#   must be a (non-strict) subclass of the metaclasses of all its bases
```

La règle : la métaclasse d'une classe doit être compatible avec celles de **toutes** ses bases.

```
class Meta12(Meta1, Meta2): ...      # la résolution : une métaclasse COMBINÉE
class C(A, B, metaclass=Meta12): ... # ✔ (si Meta1 et Meta2 coopèrent – super() !)
```

Cas réel fréquent : hériter d' `ABC` (métaclasse `ABCMeta`) **et** d'une classe à métaclasse maison :

```
class PluginBase(ABC, metaclass=PluginMeta):      # ✖ conflit ABCMeta vs PluginMeta
class PluginMeta(ABCMeta): ...                   # ✔ faire hériter SA métaclasse d'ABCMeta
```

C'est la raison de préférer `__init_subclass__` : zéro conflit possible. Les métaclasses sont *virales* — toute la hiérarchie en hérite.

Descripteurs avancés : le stockage revisité

Au chap. 6, `Positive` stockait dans `obj.__dict__[self.name]`. Affinons pour les cas exigeants :

Compatible `__slots__` (chap. 1) Le descripteur-décorateur

Pas de `__dict__` ? Le descripteur stocke sous un nom **privé**, déclaré dans les slots :

```
class Field:
    def __set_name__(self, owner, name):
        self.public = name
        self.private = "_" + name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return getattr(obj, self.private)

    def __set__(self, obj, value):
        self.validate(value)
        setattr(obj, self.private, value)

class Point:
    __slots__ = ("_x", "_y")    # les noms PRIVÉS
    x = Field()
    y = Field()
```

Un descripteur **paramétré par une fonction** — `property` générique maison :

```
class validated:
    """@validated avec une fonction de contrôle."""
    def __init__(self, validator):
        self.validator = validator

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, objtype=None):
        return obj.__dict__[self.name]

    def __set__(self, obj, value):
        obj.__dict__[self.name] = \
            self.validator(value)    # normalise !

class Order:
    @validated
    def quantity(value):
        if value < 1:
            raise ValueError(...)
        return int(value)
        # la fonction
        # DEVIENT le
        # validateur
```

Le grand assemblage : un mini-ORM déclaratif

Métaclasses + descripteurs + `__set_name__` — la mécanique de Django/SQLAlchemy, à nu :

```
class Field:
    # DESCRIPTEUR typé (chap. 6)
    def __init__(self, col_type: str, primary: bool = False):
        self.col_type, self.primary = col_type, primary
    def __set_name__(self, owner, name):
        self.name = name
    def __get__(self, obj, objtype=None):
        return self if obj is None else obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        obj.__dict__[self.name] = value

class ModelMeta(type):
    # MÉTACLASSE collectrice
    def __new__(mcls, name, bases, ns, **kw):
        cls = super().__new__(mcls, name, bases, ns, **kw)
        cls._fields = {k: v for k, v in ns.items() if isinstance(v, Field)}
        cls._table = ns.get("__table__", name.lower())
        return cls

class Model(metaclass=ModelMeta):
    # __init__ GÉNÉRIQUE piloté par _fields
    def __init__(self, **kwargs):
        for fname in self._fields:
            setattr(self, fname, kwargs.get(fname))

    @classmethod
    def create_table_sql(cls) -> str:
        cols = ", ".join(f"{n} {f.col_type}" for n, f in cls._fields.items())
        return f"CREATE TABLE {cls._table} ({cols})"
```

Le mini-ORM à l'usage

```
class User(Model):
    __table__ = "users"
    id = Field("INTEGER", primary=True)
    name = Field("TEXT")
    email = Field("TEXT")

User.create_table_sql()
# 'CREATE TABLE users (id INTEGER PRIMARY KEY, name TEXT, email TEXT)'

u = User(id=1, name="Alice", email="alice@example.com")
u.name # 'Alice' - via Field.__get__
User._fields # {'id': Field, 'name': Field, 'email': Field}
```

Qui fait quoi — la cartographie des responsabilités :

Brique	Rôle	Chapitre d'origine
Field (descripteur)	stockage + validation par attribut	6
__set_name__	chaque Field connaît son nom	6
ModelMeta	collecte _fields, déduit __table__ — par classe	8
__init__ générique	hydratation pilotée par _fields	2 (setattr)

En ~30 lignes : le squelette conceptuel de SQLAlchemy `DeclarativeBase`, Django `models.Model`, Pydantic `BaseModel`. La version réelle ajoute : héritage des fields, relations, requêtes... mais **rien de magique en plus**.

Customisation d'imports : la machinerie

`import x` n'est pas une boîte noire — c'est un protocole **entièrement personnalisable** :

```
import requests
├── 1. sys.modules ?      déjà importé → renvoyé tel quel (le cache-singleton du chap. 7)
├── 2. sys.meta_path      la liste des FINDERS, interrogés dans l'ordre :
│   ├── BuiltinImporter   modules compilés dans l'interpréteur
│   ├── FrozenImporter    modules gelés
│   └── PathFinder        cherche dans sys.path (le cas usuel)
│       └── find_spec(name) → ModuleSpec (où il est, comment le charger)
└── 3. le LOADER du spec
    ├── create_module(spec) créer l'objet module (ou None = défaut)
    └── exec_module(module) exécuter le code dans module.__dict__
```

```
import sys
sys.meta_path
# [<distutils_hack.DistutilsMetaFinder>, <BuiltinImporter>, <FrozenImporter>, <PathFinder>]
#      ↑ regardez : setuptools s'insère DÉJÀ dans vos imports !
```

Insérer son propre finder dans `sys.meta_path` = intercepter tous les imports : audit, politique de sécurité, chargement depuis des sources exotiques (réseau, BDD, archives chiffrées)...

Import hook 1 : auditer et bloquer

Un finder **passif** (audit) ou **bloquant** (politique d'imports) — utile en CI et en sandbox :

```
import sys
from importlib.abc import MetaPathFinder

class ImportPolicy(MetaPathFinder):
    """Journalise tout import et bloque les modules interdits."""
    FORBIDDEN = {"telnetlib", "pickle"}          # politique d'équipe / sécurité

    def __init__(self):
        self.seen: list[str] = []

    def find_spec(self, fullname, path, target=None):
        self.seen.append(fullname)                # AUDIT : qui importe quoi
        if fullname.split(".")[0] in self.FORBIDDEN:
            raise ImportError(
                f"{fullname} est interdit par la politique projet "
                f"(utilisez json plutôt que pickle)")
        return None                               # None = "pas pour moi" → suivant

policy = ImportPolicy()
sys.meta_path.insert(0, policy)                  # en TÊTE : consulté avant tout

import json                                     # ok – et tracé dans policy.seen
import pickle                                  # ✖ ImportError: pickle est interdit...
```

Renvoyer `None` laisse la main aux finders suivants : on **observe sans perturber**. C'est le principe des outils de mesure de dépendances et des imports lazy de certains frameworks.

Import hook 2 : charger depuis une source exotique

Un finder + loader complet : ici, des modules stockés dans un simple dict (imaginez : BDD, S3, code chiffré...) :

```
import sys
from importlib.abc import Loader, MetaPathFinder
from importlib.machinery import ModuleSpec

REMOTE_MODULES = {
    "rules.pricing": "def discount(total):\n    return total * 0.9\n",
}

class DictFinder(MetaPathFinder, Loader):
    def find_spec(self, fullname, path, target=None):
        if fullname in REMOTE_MODULES:
            return ModuleSpec(fullname, self)      # je sais le charger : moi-même
        return None

    def exec_module(self, module):
        code = REMOTE_MODULES[module.__name__]
        exec(code, module.__dict__)              # l'exec LÉGITIME (chap. 2 !)

sys.meta_path.append(DictFinder())

from rules.pricing import discount              # un import... sans fichier !
discount(100.0)                                 # 90.0
```

Cas réels : règles métier rechargeables à chaud, plugins depuis une BDD, notebooks importables, code signé/chiffré. ⚠ Vous exécutez du code : la **confiance dans la source** est le prérequis absolu.

PEP 562 : `__getattr__` au niveau module

Le filet de sécurité du chap. 2, transposé aux **modules** — sans la lourdeur d'un import hook :

```
# bigpkg/__init__.py
"""Package avec imports paresseux et dépréciations propres."""
import importlib, warnings

_LAZY = {"heavy_ml": ".heavy_ml", "viz": ".viz"}      # sous-modules coûteux
_RENAMED = {"old_helper": "new_helper"}

def __getattr__(name):
    # appelé si l'attribut est introuvable
    if name in _LAZY:
        module = importlib.import_module(_LAZY[name], __name__)
        globals()[name] = module      # mis en cache : plus jamais rappelé
        return module
    if name in _RENAMED:
        warnings.warn(f"{name} est déprécié, utilisez {_RENAMED[name]}",
                      DeprecationWarning, stacklevel=2)
        return globals()[_RENAMED[name]]
    raise AttributeError(f"module {__name__!r} has no attribute {name!r}")
```

```
import bigpkg          # instantané – heavy_ml N'est PAS chargé
bigpkg.heavy_ml.train(...) # chargé ICI, au premier accès
bigpkg.old_helper      # DeprecationWarning + redirection
```

C'est ainsi que les **gros packages scientifiques** (SciPy, TensorFlow...) gardent un `import` rapide — pertinent pour vos propres bibliothèques internes (et mesurable : chap. 11).

Bonnes pratiques — synthèse du chapitre

« *Metaclasses are deeper magic than 99% of users should ever worry about. If you wonder whether you need them, you don't.* » — Tim Peters

L'échelle de la métaprogrammation — toujours commencer en haut :

1. fonction / décorateur (chap. 3)	presque toujours suffisant
2. <code>__set_name__</code> (chap. 6)	le descripteur apprend son contexte
3. <code>__init_subclass__</code>	enregistrement, validation de hiérarchies
4. métaclasse	namespace custom, contrôle d'instanciation
5. import hooks	chargement exotique, politiques globales

Réflexes :

- Une métaclasse est **virale** (toute la hiérarchie) et **conflictuelle** (héritage multiple) — `__init_subclass__` d'abord
- Validez **à l'import** (métaclasse, `__init_subclass__`) plutôt qu'à l'exécution : l'erreur la plus précoce est la moins chère
- Import hooks : `None` = « pas pour moi » ; insérez-vous en tête pour bloquer, en queue pour compléter
- Documentez **lourdement** : la métaprogrammation surprend toujours le mainteneur suivant
- Le but reste l'**API simple pour l'utilisateur** (déclaratif à la Django) — la complexité se concentre dans le framework, une fois

Travaux pratiques 1/2

TP 8.1 — Exploration des métaclasses (45 min)

1. Écrivez une métaclasse `TraceMeta` qui affiche `name`, `bases` et les clés du `namespace` à chaque création de classe — appliquez-la à une petite hiérarchie et observez l'ordre des événements à l'import
2. `InterfaceMeta` : vérifie que toute sous-classe concrète implémente les méthodes listées dans `__required__` de sa base — comparez le message d'erreur avec celui d' `abc.ABC` (chap. 6)
3. Réécrivez l'exercice 2 avec `__init_subclass__` — comptez les lignes, provoquez le conflit de métaclasses avec `ABC` dans la version 2, constatez son absence dans la version 3
4. `SingletonMeta` du cours : ajoutez la thread-safety avec un `threading.Lock` (avant-goût du chap. 12)

TP 8.2 — Protocol descriptors avancés (45 min)

1. Étendez le mini-ORM du cours : `Field(col_type, nullable=False, default=None)` avec validation à l'écriture (`TypeError` / `ValueError` explicites) et valeurs par défaut appliquées dans `__init__`
2. Ajoutez `insert_sql(self)` → `INSERT INTO users (id, name) VALUES (?, ?)` + le tuple des valeurs — branchez-le sur `sqlite3` en mémoire et faites un aller-retour complet (create/insert/select)
3. Rendez les `Model` compatibles `__slots__` : la métaclasse génère `__slots__` à partir des fields collectés (stockage privé `_name` — pattern du cours)
4. Bonus : un descripteur `@versioned` qui historise chaque écriture (`timestamp`, `old`, `new`) par instance

Travaux pratiques 2/2

TP 8.3 — Customisation d'imports (45 min)

1. `ImportPolicy` du cours : enrichissez-la — liste blanche par préfixe (`myapp.*` , `stdlib`), rapport final des imports triés par fréquence (`Counter` , chap. 1), activable/désactivable par context manager (avant-goût du chap. 9)
2. `DictFinder` : remplacez le dict par un répertoire de fichiers `.rules` (du Python avec une extension maison) — `import rules.pricing` doit charger `rules/pricing.rules`
3. PEP 562 : prenez un de vos packages des TP précédents et rendez un sous-module lourd **paresseux** ; mesurez le temps d'import avant/après avec `python -X importtime -c "import monpkg"`
4. **Discussion** : pour chaque exercice, l'alternative sans hook (convention, lint, doc) — où passe la frontière coût/bénéfice ?

Livrables : code + tests pytest ; pour le TP 8.3.3, les deux mesures `importtime` commentées.

Chapitre 9

Context managers avancés

Python Perfectionnement — Jour 3

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Expliquer** le protocole `__enter__ / __exit__` et la sémantique des exceptions
- **Implémenter** des context managers en classe et avec `@contextmanager`
- **Exploiter** la boîte à outils `contextlib` : `suppress`, `closing`, `ExitStack` ...
- **Gérer dynamiquement** N ressources avec `ExitStack` (acquisition transactionnelle)
- **Appliquer** les patterns : transaction, verrou, état temporaire, pooling
- **Tester** vos context managers, y compris les scénarios d'échec

Travaux pratiques : implémentation de managers, tests de scénarios complexes, optimisation des ressources.

Le `try/finally` des générateurs (chap. 5), `mock.patch` (chap. 2), les verrous (chap. 12) : tous des context managers — voici leur mécanique.

Le problème : garantir le nettoyage

Toute ressource acquise doit être libérée — quoi qu'il arrive :

✗ Le code optimiste

```
f = open("data.csv")
data = parse(f)      # ✗ exception →
f.close()            # close() JAMAIS exécuté
                    # → fuite de descripteur
```

😐 La version correcte... verbeuse

```
f = open("data.csv")
try:
    data = parse(f)
finally:
    f.close()          # garanti, mais 4 lignes
                    # de plomberie par ressource
```

✓ Le `with` : le try/finally encapsulé

```
with open("data.csv") as f:
    data = parse(f)
# fermé ICI — succès, exception ou return
```

Partout dans la stdlib :

```
with lock:                # verrou (chap. 12)
with sqlite3.connect(db) as conn: # transaction
with tempfile.TemporaryDirectory() as tmp:
with patch("mod.func") as m: # chap. 2
with socket.create_connection(addr) as s:
```

Fichiers, verrous, transactions, connexions, états temporaires : **acquérir / utiliser / libérer** — le pattern le plus fréquent de la programmation. Le `with` le rend **infaillible**.

Le protocole : `__enter__` et `__exit__`

```
with EXPR as target:
    BODY

# ce que Python exécute réellement :
#
# cm = EXPR
# target = cm.__enter__()
# try:
#     BODY
# except BaseException as exc:
#     if not cm.__exit__(type(exc), exc, exc.__traceback__):
#         raise          # relancée si __exit__ → falsy
# else:
#     cm.__exit__(None, None, None)
```

Méthode	Rôle	Retour
<code>__enter__(self)</code>	acquérir la ressource	la valeur liée par <code>as</code> (souvent <code>self</code>)
<code>__exit__(self, exc_type, exc_value, tb)</code>	libérer, toujours appelé	<code>True</code> = exception étouffée

Trois points clés :

- `as` reçoit le **retour de** `__enter__` — pas l'objet du `with` (souvent identiques, pas toujours : `open()` → le fichier, mais d'autres renvoient un objet dédié)
- `__exit__` reçoit l'exception **en trois morceaux** → il peut la logger, la transformer, l'étouffer
- Une exception **dans** `__enter__` : `__exit__` n'est PAS appelé — l'acquisition doit être atomique

Implémenter un context manager : la classe

```
import time

class Timer:
    """Chronomètre un bloc - réutilise le perf_counter du chap. 1."""
    def __init__(self, label: str = ""):
        self.label = label
        self.elapsed = 0.0

    def __enter__(self):
        self.start = time.perf_counter()
        return self          # ← accessible via "as"

    def __exit__(self, exc_type, exc_value, tb):
        self.elapsed = time.perf_counter() - self.start
        status = "✖" if exc_type else "✓"
        print(f"{status} {self.label}: {self.elapsed:.4f}s")
        return False         # ne JAMAIS étouffer par défaut

with Timer("chargement") as t:
    data = load_dataset()
    # ✓ chargement: 1.2345s

t.elapsed                  # 1.2345 - l'objet SURVIT au bloc : résultat exploitable
```

Quand préférer la classe : un état à exposer après le bloc (`t.elapsed`), des méthodes auxiliaires, ou un objet déjà existant qui *devient* context manager (une connexion, un pool...).

`__exit__` : décider du sort de l'exception

`__exit__` est un point de contrôle — quatre stratégies possibles :

```
class Transaction:
    def __init__(self, conn):
        self.conn = conn

    def __enter__(self):
        self.conn.execute("BEGIN")
        return self.conn

    def __exit__(self, exc_type, exc_value, tb):
        if exc_type is None:
            self.conn.execute("COMMIT")      # ① succès → valider
            return False
        self.conn.execute("ROLLBACK")      # ② échec → annuler...
        log.error("transaction annulée : %s", exc_value)
        if isinstance(exc_value, RecoverableError):
            return True                    # ③ ...étouffer si gérable
        return False                      # ④ ...sinon laisser remonter
```

```
with Transaction(conn):
    conn.execute("INSERT ...")
    conn.execute("UPDATE ...")           # ✱ → ROLLBACK automatique, exception propagée
```

 `return True` est un acte fort : l'appelant ne saura jamais qu'il y a eu une exception. Étouffez ciblé (un type précis, documenté) — jamais un `return True` inconditionnel qui avale même `KeyboardInterrupt`.

@contextmanager : la version générateur

`contextlib.contextmanager` transforme un **générateur** (chap. 5) en context manager — le `yield` marque la frontière enter/exit :

```
from contextlib import contextmanager

@contextmanager
def timer(label: str = ""):
    start = time.perf_counter()          # — partie __enter__
    try:
        yield lambda: time.perf_counter() - start  # la valeur du "as"
    finally:
        # — partie __exit__
        print(f"{label}: {time.perf_counter() - start:.4f}s")

with timer("requête") as t:
    rows = run_query()
    print(f"déjà {t():.2f}s après la requête")
```

La règle absolue : `try/finally` autour du `yield`. Si le corps du `with` lève, l'exception est injectée au point du `yield` (le `throw()` du chap. 5 !) — sans `finally`, pas de nettoyage :

```
@contextmanager
def transaction(conn):
    conn.execute("BEGIN")
    try:
        yield conn
        conn.execute("COMMIT")          # atteint SEULEMENT si le bloc réussit
    except Exception:
        conn.execute("ROLLBACK")        # l'exception arrive ICI, au yield
        raise                          # re-lever : ne pas étouffer en silence
```

Classe ou @contextmanager ? Et la réutilisabilité

Le choix

Critère	@contextmanager	Classe
Concision	✓ ~5 lignes	~15 lignes
État exposé après	difficile	✓ t.elapsed
Logique enter/exit	linéaire, lisible	éclatée en 2 méthodes
Réutilisable N fois	✗ générateur épuisé !	✓ si bien conçu

Défaut raisonnable : @contextmanager ; classe quand l'objet a une vie au-delà du bloc.

⚠ Le piège de la réutilisation

```
tm = timer("x")          # UN générateur
with tm: ...              # ok - consommé
with tm: ...              # ✗ AttributeError:
                        # générateur épuisé (chap. 5 !)
```



```
# ✓ refabriquer à chaque fois
with timer("x"): ...
with timer("x"): ...
```

Vocabulaire :

- **réutilisable** : plusieurs with successifs (ex. Timer classe)
- **réentrant** : with imbriqués sur le MÊME objet (ex. threading.RLock — rare !)

Un fichier n'est ni l'un ni l'autre ; documentez ce que VOTRE cm supporte.

La boîte à outils `contextlib` 1/2

```

from contextlib import suppress, closing, nullcontext

# suppress : l'ignorance ASSUMÉE et ciblée d'exceptions
with suppress(FileNotFoundError):
    os.remove("cache.tmp")          # absent ? pas grave.
# ↑ replace : try: os.remove(...) except FileNotFoundError: pass

# closing : adapte tout objet ayant .close() – déjà vu au chap. 5 !
with closing(legacy_connection()) as conn: # pas de __enter__/__exit__ ? closing les fournit
    conn.query(...)

# nullcontext : le context manager NEUTRE – l'élément identité des with
def process(path, lock=None):
    cm = lock if lock is not None else nullcontext()
    with cm:
        do_work(path)

# nullcontext(valeur) : fournit aussi une valeur au "as"
def read(source: str | Path | TextIO):
    cm = open(source) if isinstance(source, (str, Path)) else nullcontext(source)
    with cm as f:
        return f.read()
    # fichier OUVERT par nous → fermé ;
    # objet déjà ouvert reçu → PAS fermé ✓

```

`nullcontext` est le **Null Object** du chap. 7 appliqué aux context managers.

La boîte à outils `contextlib` 2/2

```
from contextlib import redirect_stdout, chdir
import io

# redirect_stdout : capturer les print d'un code qu'on ne contrôle pas
buffer = io.StringIO()
with redirect_stdout(buffer):
    legacy_report()
text = buffer.getvalue()          # imprime n'importe où...
                                # ...récupéré proprement

# chdir (3.11+) : répertoire courant temporaire
with chdir("/tmp/workdir"):
```

Le pattern commun : l'état temporaire

Tous ces outils suivent le même schéma — *sauvegarder, modifier, restaurer* :

```
@contextmanager
def env_var(name: str, value: str):
    """Variable d'environnement temporaire — pattern à connaître par cœur."""
    sentinel = object()          # distinguer "absente" de "None" (chap. 1)
    old = os.environ.get(name, sentinel)
    os.environ[name] = value
    try:
        yield
    finally:
        if old is sentinel:
            os.environ.pop(name, None)      # restaurer l'ABSENCE
        else:
            os.environ[name] = old
```

C'est exactement la mécanique de `mock.patch` (chap. 2) et de `decimal.localcontext`.

ExitStack : N ressources, dynamiquement

Problème : ouvrir un nombre de ressources **inconnu à l'écriture** — impossible avec des `with` imbriqués :

```
from contextlib import ExitStack

def merge_files(paths: list[Path], output: Path):
    with ExitStack() as stack:
        files = [stack.enter_context(open(p)) for p in paths]    # N fichiers !
        out = stack.enter_context(open(output, "w"))
        for line in heapq.merge(*files):    # fusion triée paresseuse (chap. 1+5)
            out.write(line)
    # TOUS fermés ici - dans l'ordre INVERSE d'ouverture (LIFO)
```

Garantie transactionnelle à l'acquisition : si le 7e `open` échoue, les 6 premiers sont **déjà** sous la protection de la stack → refermés proprement. Le bug des « acquisitions partielles » disparaît.

```
with ExitStack() as stack:
    conn = stack.enter_context(db_connect())    # ressources hétérogènes
    lock = stack.enter_context(file_lock(".lock"))
    stack.callback(notify, "fin de traitement")    # callback arbitraire à la sortie
    tmp = stack.enter_context(TemporaryDirectory())
    process(conn, tmp)
```

`callback(fn, *args)` enrôle n'importe quelle fonction de nettoyage — même sans context manager.

ExitStack : les patterns avancés

pop_all() : le commit d'acquisition

Acquérir N ressources puis les garder si tout s'est bien passé :

```
class ResourceSet:
    def __init__(self, specs):
        with ExitStack() as stack:
            self.conns = [
                stack.enter_context(connect(s))
                for s in specs
            ]
            # ✳ ici → tout est libéré
            self._stack = stack.pop_all()
            # ✔ succès TOTAL → la responsabilité
            #   est TRANSFÉRÉE hors du with

    def close(self):
        self._stack.close()  # libération différée
```

Tout-ou-rien : soit **toutes** les connexions, soit **aucune** — jamais d'état bancal.

CM dans un objet long-vécu

Une classe qui POSSÈDE des ressources gérées :

```
class Service(ExitStack):  # hériter !
    def __init__(self, config):
        super().__init__()
        self.db = self.enter_context(
            connect(config.dsn))
        self.cache = self.enter_context(
            RedisPool(config.redis))

    with Service(cfg) as svc:
        svc.run()
    # db ET cache fermés, ordre inverse ✔
```

Et la version asynchrone existe : `AsyncExitStack` (chap. 12).

with multiples et exceptions dans `__enter__`

```
# Plusieurs cm dans UN with – équivaut à des with IMBRIQUÉS (le 2e peut utiliser le 1er)
with open(src) as fin, open(dst, "w") as fout:
    fout.writelines(fin)

# 3.10+ : parenthèses pour la lisibilité quand ça s'allonge
with (
    open(src) as fin,
    open(dst, "w") as fout,
    Timer("copie") as t,
):
    fout.writelines(fin)
```

Sémantique d'échec (imbrication oblige) :

```
with A() as a, B() as b:    ≡    with A() as a:
    ...                    with B() as b:
                            ...

B.__enter__ échoue → A.__exit__ EST appelé (A protège déjà) ✓
A.__enter__ échoue → rien à nettoyer, exception directe ✓
```

C'est pour cela que `__enter__` doit rester **minimal et atomique** : tout ce qui peut échouer après une acquisition partielle *dans le même* `__enter__` doit être protégé manuellement — ou délégué à un `ExitStack` interne.

Aperçu : les context managers asynchrones

Le protocole transposé au monde `async` (chap. 12) — quand l'acquisition **elle-même** fait des I/O :

```
class AsyncConnection:
    async def __aenter__(self):
        # acquisition NON bloquante
        self.conn = await asyncpg.connect(DSN)
        return self.conn

    async def __aexit__(self, exc_type, exc_value, tb):
        await self.conn.close()
        # libération non bloquante aussi
        return False

async def main():
    async with AsyncConnection() as conn:
        # async with
        rows = await conn.fetch("SELECT ...")
```

Synchrone (ce chapitre)	Asynchrone (chap. 12)
<code>__enter__</code> / <code>__exit__</code>	<code>__aenter__</code> / <code>__aexit__</code>
<code>with</code>	<code>async with</code>
<code>@contextmanager</code>	<code>@asynccontextmanager</code>
<code>ExitStack</code>	<code>AsyncExitStack</code>

Tout le modèle mental se transpose — comme pour les générateurs asynchrones (chap. 5). Les patterns transaction, état temporaire, pool : identiques, avec `await` aux frontières.

Tester les context managers

Le cm est du code critique (c'est lui qui nettoie !) — il mérite ses scénarios d'échec :

```
class FakeConn:
    # un espion minimal (sans lib de mock)
    def __init__(self): self.statements = []
    def execute(self, sql): self.statements.append(sql)

def test_transaction_commits_on_success():
    conn = FakeConn()
    with transaction(conn):
        conn.execute("INSERT 1")
    assert conn.statements == ["BEGIN", "INSERT 1", "COMMIT"]

def test_transaction_rolls_back_and_propagates():
    conn = FakeConn()
    with pytest.raises(ValueError):
        with transaction(conn):
            conn.execute("INSERT 1")
            raise ValueError("boom")
    assert conn.statements[-1] == "ROLLBACK" # ...et le rollback a eu lieu

def test_resources_freed_even_on_partial_failure():
    opened = []
    def failing_factory(i):
        if i == 2: raise IOError("disque plein")
        return make_tracked_resource(opened, i)
    with pytest.raises(IOError):
        acquire_all([failing_factory(i) for i in range(3)])
    assert all(r.closed for r in opened) # les acquis AVANT l'échec sont libérés
```

La check-list de test d'un cm : succès → effets ; exception → nettoyage + propagation ; exception étouffée → cas documentés seulement ; acquisition partielle → rien ne fuit.

Optimisation des ressources : le pool

Ouvrir une connexion coûte cher (TCP + TLS + auth $\approx$ dizaines de ms) — le **pool** amortit, le cm rend l'emprunt **sûr** :

```
import queue
from contextlib import contextmanager

class ConnectionPool:
    def __init__(self, factory, size: int = 5):
        self._pool = queue.Queue(maxsize=size)  # thread-safe (chap. 12)
        for _ in range(size):
            self._pool.put(factory())

    @contextmanager
    def acquire(self, timeout: float = 5.0):
        conn = self._pool.get(timeout=timeout)  # emprunt (bloque si vide)
        try:
            yield conn
        except Exception:
            conn = self._reset_or_replace(conn)  # connexion suspecte ? on l'assainit
            raise
        finally:
            self._pool.put(conn)  # restitution GARANTIE

pool = ConnectionPool(lambda: psycopg.connect(DSN), size=10)

def handle_request(req):
    with pool.acquire() as conn:  # emprunt borné par le bloc
        return conn.execute(req.query)
    # impossible d'oublier de rendre la connexion ✓
```

Le cm comme garde-fou économique : la ressource chère est *forcément* restituée — fuites impossibles par construction. Même pattern : pools de threads (chap. 12), sémaphores, licences logicielles, rate limiters.

Bonnes pratiques — synthèse du chapitre

Conception :

- Tout couple *acquérir/libérer* mérite un context manager — si vous écrivez `try/finally` deux fois, factorisez
- `@contextmanager` par défaut ; classe si l'état doit survivre au bloc ou être réutilisable
- **`try/finally` autour du `yield`** — l'oubli n°1, qui annule toute la garantie
- `__enter__` minimal et atomique ; acquisitions multiples → `ExitStack`

Exceptions :

- `return True` dans `__exit__` : ciblé, documenté, jamais inconditionnel
- Transaction : `commit` dans le chemin nominal, `rollback` + `raise` dans l'`except`
- `suppress(UneException)` plutôt que `try/except/pass` — l'intention est lisible

Architecture :

- État temporaire (`env`, `cwd`, `config`, `patch`) : le pattern *sauvegarder/modifier/restaurer* en cm
- Ressources chères : pool + `acquire()` en cm — la restitution devient structurelle
- `nullcontext` pour unifier les chemins avec/sans ressource ; `pop_all()` pour le tout-ou-rien
- Testez les **scénarios d'échec** : un cm qui ne nettoie que le chemin heureux est un faux ami

Travaux pratiques 1/2

TP 9.1 — Implémentation de managers (45 min)

1. `Timer` en deux versions (classe et `@contextmanager`) — la classe doit être **réutilisable** (plusieurs `with` successifs) et cumuler `total_elapsed` ; prouvez par un test que la version générateur ne l'est pas
2. `transaction(conn)` complet sur `sqlite3` : `commit/rollback` + paramètre `swallow=(ExceptionType,...)` pour les exceptions à étouffer (documentées !)
3. `env_vars(**kwargs)` : version multi-variables du pattern état temporaire — gérez la restauration de l'absence (sentinelle)
4. `@retry_context(attempts=3)` : un cm... impossible ! Comprenez pourquoi (le bloc `with` ne peut pas être rejoué) et transformez-le en décorateur (chap. 3) — notez la frontière entre les deux outils

TP 9.2 — Tests de scénarios complexes (45 min)

1. Écrivez la suite de tests complète de votre `transaction` : succès, exception propagée, exception étouffée, exception **dans le commit lui-même** (que se passe-t-il ?)
2. `acquire_all(factories)` avec `ExitStack` + `pop_all()` : tout-ou-rien sur N ressources — testez l'échec à la k-ième acquisition avec des ressources espionnes qui tracent `open / close`
3. Testez `merge_files` du cours avec 100 fichiers temporaires (`tempfile` + `ExitStack` dans le test lui-même !) — vérifiez qu'aucun descripteur ne fuit (`psutil.Process().num_fds()` avant/après)
4. Convertissez vos fixtures de test répétitives en fixtures pytest avec `yield` — observez : une fixture pytest **est** un `@contextmanager`

Travaux pratiques 2/2

TP 9.3 — Optimisation des ressources (45 min)

1. Implémentez le `ConnectionPool` du cours avec des connexions `sqlite3` — benchmark (`timeit` , chap. 1) : 1000 requêtes avec pool vs 1000 ouvertures/fermetures
2. Ajoutez : taille max + timeout (lever `PoolExhausted`), health-check à l'emprunt, statistiques (`hits` , `waits` , temps d'attente moyen)
3. `RateLimiter(max_per_second)` en cm : `with limiter:` bloque juste ce qu'il faut — réutilisez la logique `deque` du TP 3.1
4. **Mesurez** l'impact du pool sous charge : 8 threads × 100 requêtes (avant-goût du chap. 12) — tableau avant/après

Livrables : code + tests pytest (le TP 9.2 est lui-même le livrable de test) + benchmarks commentés du TP 9.3.

Chapitre 10

Écosystème Python avancé

Python Perfectionnement — Jour 3

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Outils** un projet moderne : environnements, dépendances, `pyproject.toml`, qualité
- **Packager et publier** un artefact Python
- **Manipuler** des données avec NumPy (vectorisation) et pandas (DataFrames)
- **Visualiser** avec l'écosystème matplotlib / seaborn / plotly
- **Appliquer** un workflow machine learning complet avec scikit-learn
- **Construire** une API web moderne (FastAPI) et **sécuriser** son code Python

Travaux pratiques : data science et visualisation, machine learning appliqué, web et sécurité.

Un chapitre **panorama** : l'objectif est de savoir *ce qui existe, quand l'utiliser et par où commencer* — chaque outil mériterait sa formation dédiée.

L'outillage moderne d'un projet Python

```
mon-projet/
├── pyproject.toml
├── src/monpkg/
├── tests/
└── .venv/
```

- ← LE fichier unique : métadonnées, dépendances, config outils
- ← src-layout : le package importable
- ← environnement isolé (jamais commité)

Environnements & dépendances

```
# Le standard : venv + pip
python -m venv .venv && source .venv/bin/activate
pip install -e ".[dev]" # install éditable

# La nouvelle génération : uv (10-100x plus rapide)
uv venv && uv pip install -e ".[dev]"
uv add httpx # gère pyproject + lock
uv run pytest # sans activer le venv
```

Alternatives : Poetry, PDM, Hatch — même philosophie :

`pyproject.toml` + lockfile.

La qualité, automatisée

```
ruff check . && ruff format . # lint + format (remplace
                             # flake8/isort/black, en Rust)
mypy src/                    # typage statique (chap. 6)
pytest --cov=monpkg          # tests + couverture
pre-commit install           # tout ça à CHAQUE commit
```

```
# pyproject.toml - un seul endroit
[tool.ruff]
line-length = 100
[tool.mypy]
strict = true
```

Packager et publier un artefact

`pyproject.toml` (PEP 621) décrit **tout** — finis `setup.py` et ses exécutions arbitraires :

```
[project]
name = "monpkg"
version = "1.2.0"
description = "Outils de traitement des commandes"
requires-python = ">=3.11"
dependencies = ["httpx>=0.27", "pydantic>=2.7"]

[project.optional-dependencies]
dev = ["pytest", "ruff", "mypy"]

[project.scripts]
monpkg-cli = "monpkg.cli:main"          # crée une COMMANDE installable !

[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"
```

```
python -m build          # → dist/monpkg-1.2.0-py3-none-any.whl (+ .tar.gz)
twine upload dist/*      # publication PyPI (ou dépôt privé : --repository-url)
uvx monpkg-cli --help    # et l'utilisateur final n'a même plus besoin d'installer
```

Wheel = archive prête à installer (rapide, sans exécution de code) ; **sdist** = sources. Livrez les deux. Pour une appli interne : un dépôt privé (Artifactory, GitLab PyPI) + `pip install --index-url .`

NumPy : penser en tableaux, pas en boucles

Le socle de **tout** le calcul scientifique Python : un tableau **typé, contigu, en C** (l'`array` du chap. 1, industrialisé) :

```
import numpy as np

prices = np.array([120.5, 99.9, 45.0, 210.0])      # dtype=float64, mémoire compacte
qty     = np.array([3, 10, 25, 1])

# VECTORISATION : l'opération s'applique à TOUT le tableau, en C
revenue = prices * qty                          # [361.5, 999.0, 1125.0, 210.0] – pas de boucle !
total   = revenue.sum()
prices  * 1.2                                   # broadcasting scalaire : +20 % partout
```

Pourquoi c'est ~100× plus rapide qu'une boucle Python (mesuré, chap. 1) :

Boucle Python	NumPy vectorisé
chaque élément = un objet (28+ octets)	valeurs brutes contiguës (8 octets)
interprétation de chaque itération	une boucle compilée en C
cachés C/DI mal utilisés	accès mémoire séquentiel optimal

```
%timeit sum(p * q for p, q in zip(prices_list, qty_list))  # 1M éléments : ~80 ms
%timeit (prices_np * qty_np).sum()                        # ~1.2 ms
```

La règle NumPy : si vous écrivez `for` sur un tableau, il existe presque toujours une version vectorisée.

NumPy : slicing, masques et broadcasting

Le slicing du chap. 1, étendu à N dimensions — et des **vues** (pas des copies, comme `memoryview`) :

```
m = np.arange(12).reshape(3, 4)      # matrice 3x4
m[1, 2]                             # élément (ligne 1, col 2)
m[:, 1]                             # toute la colonne 1
m[::2, 1:3]                         # lignes paires, colonnes 1-2 — une VUE !

# Masques booléens : le filtrage vectorisé
temps = np.array([21.5, 38.2, 19.8, 41.0, 25.3])
temps[temps > 30]                    # array([38.2, 41. ]) — WHERE en une expression
temps[(temps > 20) & (temps < 30)]    # & | ~ (pas and/or !)
np.where(temps > 30, "alerte", "ok")  # le if/else vectorisé

# Broadcasting : aligner des formes différentes
sales = np.random.rand(12, 4)       # 12 mois x 4 régions
month_avg = sales.mean(axis=1)       # moyenne par mois → shape (12,)
region_avg = sales.mean(axis=0)      # moyenne par région → shape (4,)
normalized = sales - region_avg      # (12,4) - (4,) : la ligne est "étirée" sur les 12 mois
```

Broadcasting : NumPy aligne les dimensions de droite à gauche (compatibles si égales ou 1). Puissant — mais vérifiez les `shape` : la moitié des bugs NumPy sont des broadcasts silencieusement faux.

pandas : le DataFrame

La table de données **étiquetée** — l'outil de facto de l'analyse en Python :

```
import pandas as pd

df = pd.read_csv("sales.csv", parse_dates=["date"])    # aussi : read_parquet, read_sql...
df.head()
#           date  region  product  qty  price
# 0 2026-01-04   Nord   Clavier    3   49.9
# 1 2026-01-04   Sud    Souris   10   19.9

df.info()      # dtypes, nulls, mémoire — TOUJOURS commencer là
df["revenue"] = df["qty"] * df["price"]      # colonne vectorisée (NumPy en dessous)

# Sélection : .loc (étiquettes) / .iloc (positions)
df.loc[df["region"] == "Nord", ["product", "revenue"]]
df.iloc[5, 2:]

# Le combo central : split-apply-combine
df.groupby("region")["revenue"].sum()
df.groupby(["region", "product"]).agg(
    ca=("revenue", "sum"),
    panier_moyen=("revenue", "mean"),
    ventes=("qty", "count"),
)

df.pivot_table(values="revenue", index="region", columns="product", aggfunc="sum")
```

Le `groupby` du chap. 4 (`itertools`), version industrielle — sans tri préalable requis.

pandas : les réflexes performance

✗ Les anti-patterns classiques

```
# 1. Itérer sur les lignes
for _, row in df.iterrows():      # ~1000x
    total += row["qty"] * row["price"] # trop lent

# 2. apply ligne à ligne pour de l'arithmétique
df.apply(lambda r: r["qty"] * r["price"],
         axis=1)                  # boucle déguisée

# 3. Concaténer dans une boucle
for chunk in chunks:
    df = pd.concat([df, chunk])    # O(n²) !
```

```
# ✅ Les versions vectorisées
df["qty"] * df["price"]
pd.concat(list_of_chunks)          # UNE fois
np.where(df["qty"] > 10, "gros", "petit")
```

Les leviers qui comptent

```
# dtypes adaptés = mémoire ÷ 5 à 10
df["region"] = df["region"].astype("category")
df["qty"] = df["qty"].astype("int32")
df.memory_usage(deep=True)         # mesurer !

# Parquet plutôt que CSV (typé, colonnaire,
# compressé : ~10x plus rapide à relire)
df.to_parquet("sales.parquet")

# Gros volumes : lire par morceaux (chap. 5 !)
for chunk in pd.read_csv("huge.csv",
                        chunksize=100_000):
    process(chunk)
```

Au-delà : Polars (Rust, paresseux, multi-cœurs — API proche, gains 5-30x), DuckDB (SQL sur fichiers). À évaluer dès que pandas peine.

Visualisation : choisir son outil

```
import matplotlib.pyplot as plt

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
ca_mensuel.plot.bar(ax=ax1, title="CA mensuel")
df.plot.scatter(x="qty", y="revenue", ax=ax2, alpha=0.3)
fig.savefig("rapport.png", dpi=150)
```

l'API OBJET (pas plt.plot
global : composable, testable)

Bibliothèque	Position	Quand
matplotlib	le socle bas niveau — tout est réglable	figures de rapport, contrôle fin
seaborn	statistique, belle par défaut (sur matplotlib)	exploration : distributions, corrélations
plotly	interactif (zoom, survol), HTML	dashboards, partage web
pandas .plot	raccourci matplotlib intégré	coup d'œil rapide en analyse

```
import seaborn as sns
sns.heatmap(df.corr(numeric_only=True), annot=True)
sns.boxplot(data=df, x="region", y="revenue")

import plotly.express as px
px.line(ca, x="date", y="revenue", color="region")
```

matrice de corrélation en 1 ligne
distributions comparées
interactif immédiat

Réflexe pro : seaborn/plotly pour **explorer**, matplotlib objet pour **publier** — et toujours titres + unités + sources sur une figure qui sort de votre machine.

scikit-learn : le workflow machine learning

L'API la plus influente du ML : **tout** modèle suit `fit` / `predict` — un Protocol de fait (chap. 6) :

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

X = df[["amount", "n_orders", "days_since_last", "country_code"]] # features
y = df["churned"] # cible (0/1)

# 1. TOUJOURS séparer avant tout : le test set est SACRÉ
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 2. Entraîner
model = RandomForestClassifier(n_estimators=200, random_state=42)
model.fit(X_train, y_train)

# 3. Évaluer sur des données JAMAIS VUES
print(classification_report(y_test, model.predict(X_test)))
#           precision    recall  f1-score
#      0      0.91      0.97      0.94
#      1      0.78      0.52      0.62    ← le churn est plus dur à attraper
```

Lire les métriques : l'accuracy seule ment dès que les classes sont déséquilibrées — regardez précision (pureté des alertes) **et** rappel (taux de détection) de la classe **minoritaire**.

scikit-learn : le Pipeline, ou le ML propre

Le préprocessing **fait partie du modèle** — le `Pipeline` l'encapsule et évite la fuite de données :

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

preprocess = ColumnTransformer([
    ("num", StandardScaler(), ["amount", "n_orders", "days_since_last"]),
    ("cat", OneHotEncoder(handle_unknown="ignore"), ["country_code"]),
])

pipe = Pipeline([
    ("prep", preprocess),
    ("clf", LogisticRegression(max_iter=1000)),
])

scores = cross_val_score(pipe, X_train, y_train, cv=5, scoring="f1")
print(f"F1 : {scores.mean():.3f} ± {scores.std():.3f}") # robuste, pas un coup de chance

pipe.fit(X_train, y_train) # le scaler n'apprend QUE sur le train ✓
joblib.dump(pipe, "model.joblib") # déployer LE TOUT (prep inclus !)
```

Pourquoi c'est crucial : scaler avant le split = le test « fuit » dans l'entraînement → scores trop beaux, désillusion en prod. Le Pipeline rend la fuite **structurellement impossible** — le `with` du ML, en quelque sorte (chap. 9).

⚠ Et le ML n'est pas magique : *garbage in, garbage out* — 80 % du travail réel est dans les données.

Le web côté client : httpx / requests

```
import httpx

# Le réflexe de base – TOUJOURS un timeout (le défaut "infini" tue des prods)
r = httpx.get("https://api.example.com/users", timeout=5.0,
              params={"active": "true"}, headers={"Authorization": f"Bearer {token}"})
r.raise_for_status()          # 4xx/5xx → exception (sinon échec SILENCIEUX)
users = r.json()
```

```
# En production : un CLIENT réutilisé (pool de connexions – chap. 9 !)
with httpx.Client(base_url="https://api.example.com",
                  timeout=httpx.Timeout(5.0, connect=2.0),
                  transport=httpx.HTTPTransport(retries=3)) as client:
    users = client.get("/users").json()
    orders = client.get("/orders").json()      # même connexion TCP réutilisée ✓
```

	requests	httpx
Maturité	le standard historique	moderne, API compatible
Async	✗	✓ AsyncClient (chap. 12)
HTTP/2	✗	✓

Les quatre réflexes client : timeout explicite, `raise_for_status()` , client réutilisé (jamais `get()` nu en boucle), retries avec backoff (votre `@retry` du chap. 3 !).

Le web côté serveur : FastAPI

Le framework qui **assemble tout ce que vous avez appris** : annotations (chap. 2), introspection de signatures (chap. 2), validation déclarative (chap. 8), injection de dépendances (chap. 6) :

```
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel, Field

app = FastAPI(title="Orders API")

class OrderIn(BaseModel):
    product: str = Field(min_length=1)
    qty: int = Field(gt=0, le=1000)
    # Pydantic : votre Serializable du
    # chap. 2, version industrielle

class OrderOut(OrderIn):
    id: int
    total: float

@app.post("/orders", response_model=OrderOut, status_code=201)
def create_order(order: OrderIn, repo: Repo = Depends(get_repo)):
    # le registre du chap. 3 !
    # l'injection du chap. 6 !
    if not repo.in_stock(order.product):
        raise HTTPException(409, detail=f"{order.product} : rupture de stock")
    return repo.create(order)
```

- Le **corps JSON est validé** par les annotations — un `qty: -3` reçoit un 422 détaillé, automatiquement
- Documentation interactive **gratuite** : `/docs` (Swagger UI) générée par introspection
- `Depends` : l'injection de dépendances testable (un faux `Repo` en test — chap. 6/7)
- Async natif quand vous en aurez besoin (chap. 12) ; servi par `uvicorn`

Sécurité 1/2 : les fondamentaux du code

Injections : ne JAMAIS concaténer Secrets et aléatoire

```
# ✗ Injection SQL
cur.execute(f"SELECT * FROM users "
            f"WHERE name = '{name}'")
# name = "x'; DROP TABLE users; --"

# ✔ Requêtes PARAMÉTRÉES – toujours
cur.execute("SELECT * FROM users "
            "WHERE name = ?", (name,))

# ✗ Injection shell
os.system(f"convert {filename} out.png")
# ✔ liste d'arguments, sans shell
subprocess.run(["convert", filename,
               "out.png"], check=True)
```

```
# ✗ random n'est PAS cryptographique
token = "".join(random.choices(chars, k=32))

# ✔ le module secrets
import secrets
token = secrets.token_urlsafe(32)
secrets.compare_digest(sig, expected) # comparaison
                                      # à temps constant

# ✗ Mot de passe : jamais en clair, jamais en SHA simple
# ✔ hachage DÉDIÉ, lent, salé
from argon2 import PasswordHasher # pip install argon2-cffi
ph = PasswordHasher()
h = ph.hash(password)
ph.verify(h, attempt) # ✨ si faux
```

Même famille : `eval` sur entrée externe (chap. 2), chemins non validés (`../etc/passwd` → `Path.resolve()` + vérifier le préfixe).

Et les secrets de config : **variables d'environnement** / vault — jamais en dur, jamais commités (`.env` + `.gitignore`).

Sécurité 2/2 : la chaîne d'approvisionnement

Votre code n'est qu'une fraction de ce qui tourne — vos dépendances sont votre surface d'attaque :

```
pip-audit                # CVE connues dans VOS dépendances installées
# Found 2 known vulnerabilities in 1 package
# requests 2.25.0 → CVE-2023-32681 (fuite d'en-têtes Proxy-Authorization)

bandit -r src/            # lint sécurité de VOTRE code
# B608: possible SQL injection / B301: pickle usage / B105: hardcoded password

uv lock                  # versions ÉPINGLÉES + hashes → builds reproductibles
```

La check-list sécurité d'un projet Python :

Risque	Parade
Dépendance vulnérable	<code>pip-audit</code> en CI, lockfile, mises à jour régulières
Typosquatting (<code>requests</code> ...)	vérifier les noms, dépôt privé en entreprise
<code>pickle</code> de données externes	jamais — <code>json</code> (c'est du code exécutable, chap. 2)
Désérialisation YAML	<code>yaml.safe_load</code> , jamais <code>yaml.load</code>
Secrets commités	— ignoré, scan (gitLeaks), rotation si fuite

Validation des entrées : aux frontières, systématiquement — Pydantic (vu à l'instant) est aussi un outil de sécurité.

Bonnes pratiques — synthèse du chapitre

Outillage & packaging :

- `pyproject.toml` unique, `src-layout`, `lockfile`, `ruff` + `mypy` + `pytest` en pre-commit/CI
- Livrez des **wheels** ; les `[project.scripts]` font de votre lib un outil installable

Data :

- NumPy/pandas : **vectoriser**, jamais itérer — et mesurer (`%timeit` , `memory_usage`)
- `category` + Parquet + chunks : les trois leviers pandas qui changent tout
- Pandas sature ? Polars / DuckDB avant de réécrire en autre chose

ML :

- Split d'abord, Pipeline toujours, cross-validation pour conclure — la fuite de données est l'ennemi n°1
- Les métriques de la classe minoritaire, pas l'accuracy globale

Web & sécurité :

- Client HTTP : `timeout` + `raise_for_status` + client réutilisé + retries
- FastAPI/Pydantic : la validation aux frontières est gratuite — prenez-la
- Paramétrer (SQL, subprocess), `secrets` pour l'aléa, Argon2 pour les mots de passe
- `pip-audit` + `bandit` + `lockfile` : la sécurité des dépendances s'automatise

Travaux pratiques 1/2

TP 10.1 — Data science et visualisation (60 min)

À partir du dataset de ventes du TP 4.2 (revisité avec les bons outils !) :

1. Chargez en pandas : `info()` , conversion des dtypes (`category` , dates), `memory_usage(deep=True)` avant/après — quantifiez le gain
2. Refaites les analyses du TP 4.2 en pandas (CA par région, top produits, CA cumulé mensuel) — comparez la concision et la vitesse (`%timeit`) avec vos versions `itertools`
3. Détectez les anomalies : jours où le CA s'écarte de $\pm 2\sigma$ de la moyenne mobile 7 jours (vectorisé, `rolling`)
4. Produisez une figure de synthèse 2x2 (matplotlib objet) : CA mensuel (barres), répartition par région (boxplot seaborn), heatmap régionxproduit, CA cumulé — titres, unités, export PNG
5. Bonus : la même page en plotly, interactive

TP 10.2 — Machine learning appliqué (60 min)

Sur un dataset churn fourni (~10 000 clients, 12 features, classes déséquilibrées 85/15) :

1. Exploration : distributions, corrélations (heatmap), taux de churn par segment — 3 figures
2. Baseline honnête : `DummyClassifier` puis `LogisticRegression` en Pipeline complet (imputation, scaling, encodage) — `cross_val_score` en F1
3. Améliorez : `RandomForestClassifier` , comparez via la même CV ; affichez la matrice de confusion et expliquez précision vs rappel **en termes métier** (que coûte un faux positif ? un faux négatif ?)
4. Sérialisez le pipeline (`joblib`) et écrivez `predict_churn(client: dict) -> float` qui le recharge — testée avec pytest

Travaux pratiques 2/2

TP 10.3 — Web et sécurité (60 min)

1. Construisez l'API FastAPI du mini-ORM du TP 8.2 : CRUD `/users` avec modèles Pydantic, erreurs 404/409 propres, repository injecté via `Depends` — testez avec `TestClient` et un repo in-memory (chap. 6 !)
2. Ajoutez l'authentification : POST `/auth/register` (hash Argon2), POST `/auth/token` — comparez le temps de `ph.verify` et d'une comparaison SHA-256 : pourquoi cette lenteur est une **feature** ?
3. Audit du code fourni `legacy_api.py` (volontairement truffé de failles) : SQL concaténé, `yaml.load`, `random` pour les tokens, secret en dur, `pickle` d'entrée externe — trouvez les 5, corrigez, et vérifiez avec `bandit` avant/après
4. Lancez `pip-audit` sur le projet : analysez le rapport et mettez à jour ce qui doit l'être

Livrables : notebooks/scripts + figures (10.1), pipeline sérialisé + tests (10.2), API testée + rapport bandit avant/après (10.3).

Chapitre 11

Optimisation de performance

Python Perfectionnement — Jour 4

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

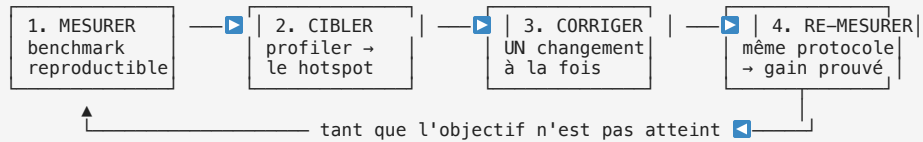
- **Appliquer** une méthodologie d'optimisation rigoureuse : mesurer → cibler → corriger → re-mesurer
- **Comprendre** pourquoi CPython est lent — et où il ne l'est pas
- **Benchmark**er proprement avec `timeit`, `pyperf`, `hyperfine`
- **Profiler** le CPU (`cProfile`, `py-spy`, `line_profiler`) et la mémoire (`tracemalloc`)
- **Optimiser** : algorithmes, structures, idiomes, vectorisation, compilation (Numba/Cython)
- **Décider** quand s'arrêter — le coût de la performance se paie en lisibilité

Travaux pratiques : profilage d'applications, optimisation de code, benchmarking.

Les outils semés depuis le chap. 1 (`timeit`, `tracemalloc`, `getsizeof`) deviennent ici une **méthodologie complète**.

La méthodologie : les quatre règles d'or

« *Premature optimization is the root of all evil.* » — D. Knuth (déjà cité au chap. 1 — maintenant, appliquons-le)



Règle 1 — Un objectif chiffré d'abord : « l'import doit passer sous 30 s », pas « ça doit aller plus vite ». Sans cible, l'optimisation ne s'arrête jamais.

Règle 2 — L'intuition ment : les développeurs se trompent massivement sur la localisation des hotspots. Le profileur tranche.

Règle 3 — La loi d'Amdahl : optimiser une fonction qui pèse 5 % du temps total rapporte au mieux... 5 %. Attaquez le plus gros poste.

Règle 4 — Sous tests : optimiser sans filet de tests = réécrire des bugs plus rapides. La suite pytest passe avant ET après.

Pourquoi CPython est lent — et où il ne l'est pas

```
import dis
def add(a, b):
    return a + b
```

Les trois taxes structurelles

1. **Interprétation** : pas de compilation native (le JIT arrive doucement — 3.13+)
2. **Boxing** : `1 + 2` = deux objets alloués, déréférencés, un troisième créé
3. **Dynamisme** : chaque `obj.attr`, chaque `+` est résolu à l'exécution (MRO, descripteurs... chap. 6)

C'est le prix de la flexibilité que vous exploitez depuis 10 chapitres.

Versions récentes : CPython 3.11 (+25 % : *specializing interpreter*), 3.12, 3.13 (JIT expérimental) — **mettre à jour est une optimisation gratuite.**

Mais Python est rapide quand...

- Le travail se passe en **C** : builtins, `str.join`, `sort`, dict/set (chap. 1), `itertools` (chap. 4), NumPy (chap. 10)
- Le programme attend les **I/O** : le réseau à 50 ms écrase les µs d'interprétation (chap. 12)

La stratégie générale : faire faire le gros du travail à du code compilé, et que Python **orchestre**.

Benchmarking : `timeit` sans se mentir

```
import timeit

# UNE micro-mesure isolée - timeit désactive le GC, répète, et boucle
timeit.repeat("d['k']", setup="d = {'k': 1}", repeat=5, number=10_000_000)
# [0.21, 0.20, 0.20, 0.21, 0.20] ← prendre le MIN (le moins perturbé), pas la moyenne
```

Les pièges qui invalident un benchmark :

Piège	Parade
Mesurer une seule exécution	<code>repeat=5</code> , garder le min
Le setup compté dans la mesure	argument <code>setup=</code> / fonctions séparées
État qui s'accumule (liste qui grossit, cache chaud — chap. 3 !)	recréer l'état dans <code>setup</code> , <code>cache_clear()</code>
Machine occupée, fréquence CPU variable	machine au repos, branchée, plusieurs runs

```
# pyperf : timeit industrialisé (calibration, stats, comparaison)
python -m pyperf timeit -s "d = {'k': 1}" "d['k']" # Mean +- std dev: 21.3 ns +- 0.4 ns
hyperfine "python script_v1.py" "python script_v2.py" # benchmark de PROGRAMMES entiers
```

Micro-benchmark $\neq$ réalité : 20 ns vs 25 ns sur une opération ne dit rien du programme complet — validez toujours le gain en situation (règle 4).

Profilage CPU : **cProfile**, la vue d'ensemble

```
python -m cProfile -o profile.out -s cumulative my_app.py    # profiler un programme entier
```

```
import cProfile, pstats

cProfile.run("process_orders(orders)", "profile.out")
stats = pstats.Stats("profile.out")
stats.sort_stats("cumulative").print_stats(8)
```

ncalls	tottime	percall	cumtime	percall	filename:lineno(function)
1	0.002	0.002	12.480	12.480	app.py:12(process_orders)
500000	0.890	0.000	11.920	0.000	app.py:31(enrich_order)
500000	10.200	0.000	10.200	0.000	app.py:48(find_customer)
1500000	0.610	0.000	0.610	0.000	{method 'get' of 'dict'}

LE hotspot

Lire un profil — les deux colonnes qui comptent :

- **tottime** : temps passé **dans** la fonction elle-même (hors appels) → *c'est elle qu'on optimise*
- **cumtime** : temps fonction + tout ce qu'elle appelle → *c'est le chemin critique*

Ici : `find_customer` = 10,2 s sur 12,5 s (82 %) — appelée 500 000 fois. Une recherche linéaire dans une liste, à coup sûr (chap. 1)... La loi d'Amdahl dit : c'est **le seul** endroit où travailler.

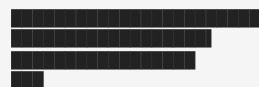
⚠ `cProfile` ralentit l'exécution (~30 %) et sous-estime les fonctions C — c'est une **boussole**, pas un chronomètre.

Profilage visuel : flamegraphs et `py-spy`

Visualiser un profil

```
pip install snakeviz
snakeviz profile.out      # le .out de cProfile,
                          # navigable dans le navigateur
```

Le **flamegraph** : chaque barre = une fonction, sa largeur = son temps cumulé, l'empilement = la pile d'appels. Le hotspot **saute aux yeux** — plus efficace qu'un tableau de 200 lignes.



```
process_orders
enrich_order
find_customer
parse
```

`py-spy` : profiler SANS toucher au code

Profileur **échantillonneur** (sampling) en Rust — il observe le processus de l'extérieur :

```
pip install py-spy
py-spy top --pid 12345      # "top" des fonctions
                             # d'un process EN COURS
py-spy record -o flame.svg -- python app.py
py-spy dump --pid 12345     # stacks instantanées :
                             # où est-il bloqué ?
```

- ~0 % d'overhead → utilisable en production
- Aucune modification du code, attachable à chaud
- Le réflexe quand « la prod rame » : `py-spy dump` d'abord

Profilage chirurgical : `line_profiler`

Le hotspot trouvé, quelle ligne coûte ? `cProfile` s'arrête à la fonction — `line_profiler` descend :

```
pip install line_profiler
kernprof -lv my_app.py           # profile les fonctions décorées @profile
```

Line #	Hits	Time	Per Hit	% Time	Line Contents
48	500000	180000	0.4	1.7	def find_customer(cid, customers):
49	500000	10020000	20.0	96.4	matches = [c for c in customers
					if c.id == cid]
					if not matches:
50	500000	150000	0.3	1.4	raise KeyError(cid)
51	12	90	7.5	0.0	return matches[0]
52	499988	160000	0.3	1.5	

96 % ICI

Verdict sans appel : la ligne 49 **parcourt toute la liste** à chaque appel — $O(n) \times 500\,000$ appels (chap. 1 !).

```
# La correction - un index construit UNE fois : O(n) total au lieu de O(nxm)
customers_by_id = {c.id: c for c in customers}           # au chargement
def find_customer(cid, customers_by_id):
    return customers_by_id[cid]                           # O(1)
# Re-mesure (règle 4) : 12.5 s → 0.6 s. Gain x20, trois lignes changées.
```

Profilage mémoire : `tracemalloc` en pratique

Entrevu au chap. 1 — voici le workflow complet pour « qui alloue, et où ça fuit » :

```
import tracemalloc

tracemalloc.start(25)                # garder 25 frames de pile

snap1 = tracemalloc.take_snapshot()  # AVANT
run_batch()
snap2 = tracemalloc.take_snapshot()  # APRÈS

for stat in snap2.compare_to(snap1, "lineno")[:5]:    # le DIFF - l'outil anti-fuite
    print(stat)
# app.py:88: size=142 MiB (+142 MiB), count=1200000 (+1200000)  📌 accumulation !
# parser.py:31: size=18 MiB (+18 MiB), ...

current, peak = tracemalloc.get_traced_memory() # et le PIC, pas juste le courant
```

Les suspects habituels

- Cache sans borne (`lru_cache(maxsize=None)` , dict global qui grossit — chap. 3)
- Listes matérialisées au lieu de générateurs (chap. 5)
- Références retenues : closures (chap. 3), handlers jamais désabonnés (chap. 7)

L'artillerie au-dessus

```
pip install memray                # Bloomberg
memray run app.py                 # flamegraph
memray flamegraph output.bin      # MÉMOIRE,
                                  # alloc natives
                                  # includes
```

`objgraph` pour traquer les cycles de références.

Optimisation niveau 1 : l'algorithme et les structures

Le levier qui écrase tous les autres — aucun compilateur ne transforme un $O(n^2)$ en $O(n)$:

```
# AVANT – réel, vu en audit : déduplication "qui marche" ... en  $O(n^2)$ 
unique = []
for record in records:
    if record not in unique:
        unique.append(record)
# 200 000 enregistrements
# 'in' sur une LISTE :  $O(n)$  – chap. 1 !
# total : ~40 s

# APRÈS –  $O(n)$ , et plus lisible
unique = list(dict.fromkeys(records))
# dédouble EN GARDANT l'ordre : ~0.02 s
```

Symptôme au profil	Diagnostic probable	Correction
<code>in / index / remove</code> sur liste en boucle	recherche $O(n)$ répétée	<code>set / dict</code> (chap. 1)
<code>pop(0)</code> , <code>insert(0, ...)</code>	décalages $O(n)$	<code>deque</code> (chap. 1)
tri dans une boucle	re-tri permanent	trier une fois / <code>bisect</code> / <code>heapq</code>
même calcul recommencé	absence de cache	<code>lru_cache</code> (chap. 3), pré-calcul
<code>+=</code> sur <code>str</code> en boucle	recopie quadratique	<code>"".join(parts)</code>
tout chargé puis filtré	matérialisation inutile	générateurs (chap. 5)

Réflexe : devant un hotspot, demander d'abord « *quelle est la complexité ?* » — pas « *comment accélérer ces lignes ?* ».

Optimisation niveau 2 : les idiomes CPython

À algorithme égal, l'implémentation compte — **uniquement sur le hotspot prouvé** :

```
# 1. Pousser les boucles vers le C (chap. 4 !)
total = 0
for x in data:                # interprété à chaque tour
    total += x
total = sum(data)              # la boucle est EN C : ~5x plus rapide

# 2. Sortir les invariants de la boucle (résolutions d'attributs comprises)
for row in rows:
    out.append(transform(row.value * tax_rate))
append = out.append            # résolution UNE fois
factor = tax_rate              # variable LOCALE (accès plus rapide)
for row in rows:
    append(transform(row.value * factor))    # gain typique : 15-30 % sur boucle chaude

# 3. Les vérités utiles (mesurées, pas devinées)
f"{a}-{b}"                    # f-string : plus rapide que + et que format()
[x * 2 for x in data]         # compréhension : plus rapide que boucle + append
if key in d:                  # EAFP vs LBYL : try/except gagne si l'exception est RARE
tuple vs list                 # création plus rapide, hashable – mais même vitesse d'itération
```

⚠ Ces gains (10-30 %) sont **un ordre de grandeur** sous l'algorithmique ($\times 10-1000$). Et chaque idiome se **re-mesure** : ils évoluent avec les versions de CPython (3.11+ a changé beaucoup de rapports de force).

Optimisation niveau 3 : vectoriser ou compiler 1/2

Le hotspot est numérique, l'algorithme est bon, les idiomes ne suffisent pas — **sortez de l'interpréteur** :

Vectoriser (NumPy — chap. 10)

```
# distance euclidienne sur 1M de points
d = [math.sqrt((x1-x2)**2 + (y1-y2)**2)
      for (x1,y1),(x2,y2) in zip(a, b)] # ~800 ms

d = np.sqrt(((a - b) ** 2).sum(axis=1)) # ~8 ms
```

Si les données sont des tableaux : **toujours essayer NumPy d'abord** — zéro dépendance exotique, x100 courant.

Numba : le JIT à un décorateur

```
from numba import njit

@njit # compilé à la volée (LLVM)
def mandelbrot(c, max_iter):
    z = 0j
    for n in range(max_iter):
        z = z * z + c # les BOUCLES redeviennent
        if abs(z) > 2: # gratuites !
            return n
    return max_iter
```

Idéal : boucles numériques impossibles à vectoriser. Limites : code « numérique » seulement, 1er appel lent (compilation).

Optimisation niveau 3 : vectoriser ou compiler 2/2

Cython : Python annoté → C

```
# fast.pyx
def integrate(double a, double b, int n):
    cdef double s = 0, dx = (b - a) / n
    cdef int i
    for i in range(n):
        s += f(a + i * dx)
    return s * dx
```

Compilé en extension C — types statiques = boxing supprimé.
Plus d'outillage que Numba, mais s'applique à du code généraliste et s'intègre au packaging (chap. 10).

Et aussi

- **mypyc** : compile du code *type-hinté* (mypy)
- **PyPy** : interpréteur JIT complet — drop-in si vos dépendances C le permettent
- **Rust/PyO3** : la voie moderne pour une extension native pérenne (c'est `ruff`, `pydantic-core`, `polars` ...)

Étude de cas : le pipeline complet

Un traitement de 2,1 M de lignes — chaque étape **mesurée** (le protocole en action) :

Étape	Action	Durée	Gain
0	Baseline (code initial, sous tests )	184 s	—
1	Profil <code>cProfile</code> : 71 % dans <code>find_rate</code> (liste, O(n))	—	—
2	<code>dict</code> d'index à la place de la recherche linéaire (chap. 1)	61 s	×3,0
3	Re-profil : 48 % dans le parsing → générateurs + <code>csv</code> au lieu de <code>split</code> maison (chap. 5)	29 s	×2,1
4	Re-profil : 60 % dans le calcul des montants → vectorisation NumPy du bloc numérique	9 s	×3,2

Les leçons :

- Le hotspot **se déplace** après chaque correction → re-profiler à chaque itération, jamais d'optimisation « en rafale »
- Le classement des gains : structure de données (×3) ≥ vectorisation (×3,2) > paresse (×2,1) » idiomes (×1,1)
- L'étape 5 a coûté autant d'effort que les étapes 2-4 réunies, pour 11 % — **savoir s'arrêter** : l'objectif (< 10 s) était atteint
- À chaque étape : la suite de tests passe — le résultat est resté **juste**

Le coût caché : performance vs lisibilité

L'optimisation a un prix que le benchmark n'affiche pas — le code devient plus dur à maintenir :

Avant : évident

```
def monthly_totals(orders):
    totals = defaultdict(float)
    for order in orders:
        month = order.date.strftime("%Y-%m")
        totals[month] += order.amount
    return dict(totals)
```

Après : x4 plus rapide... et opaque

```
def monthly_totals(orders):
    # strftime sorti de la boucle via cache de
    # mois + groupby sur date tronquée (perf!)
    key = operator.attrgetter("date.year", "date.month")
    pairs = sorted((key(o), o.amount) for o in orders)
    return {f"{y}-{m:02d}": sum(a for _, a in grp)
            for (y, m), grp in groupby(pairs, itemgetter(0))}
```

Les règles d'arbitrage :

- Une optimisation qui obscurcit doit être **rentable** (sur le chemin critique prouvé) et **commentée** (le *pourquoi* + le chiffre : « x4 sur 2M de commandes, profil du 2026-06-04 »)
- Gardez la version lisible **en test de référence** : elle valide la version rapide (`assert fast(x) == simple(x)`)
- N'optimisez jamais le code froid : 95 % de votre code n'a **aucun besoin** d'être rapide
- La performance est une **exigence comme une autre** : chiffrée dans la définition of done, vérifiée en CI (benchmark de non-régression — `pytest-benchmark`)

Bonnes pratiques — synthèse du chapitre

La méthodologie (à afficher au mur) :

1. Objectif chiffré + tests verts → 2. benchmark reproductible → 3. profil (`cProfile` / `py-spy`) → 4. le **plus gros** hotspot seulement → 5. un changement → 6. re-mesure + tests → boucler ou **s'arrêter**

La hiérarchie des leviers (toujours dans cet ordre) :

#	Levier	Gain typique	Coût
1	Algorithme & structures (chap. 1)	×10 – ×1000	faible — souvent PLUS lisible
2	Paresse & caches (chap. 3, 5)	×2 – ×50	faible
3	Vectorisation NumPy (chap. 10)	×10 – ×100	moyen — données en tableaux
4	...	...	...

Les outils par question :

« C'est lent ? » → `hyperfine` / `timeit` · « Où ? » → `cProfile` + `snakeviz` · « En prod, là ? » → `py-spy` · « Quelle ligne ? » → `line_profiler` · « La mémoire ? » → `tracemalloc` / `memray`

Travaux pratiques 1/2

TP 11.1 — Profilage d'applications (45 min)

On vous fournit `slow_app.py` (traitement de commandes : ~90 s sur le jeu de données) — **interdiction de lire le code avant de profiler** :

1. Benchmark de référence avec `hyperfine` (3 runs) — notez moyenne et écart-type : c'est votre baseline
2. `cProfile` + `snakeviz` : identifiez les 3 plus gros postes en `totttime` ; formulez une **hypothèse** pour chacun avant d'ouvrir le code
3. `line_profiler` sur le hotspot n°1 : quelle ligne, quel pourcentage ?
4. `tracemalloc` en mode comparaison de snapshots : l'application a aussi une fuite — trouvez la ligne d'accumulation
5. Rédigez le **diagnostic** (5 lignes max) : postes, causes, corrections proposées, gains attendus

TP 11.2 — Optimisation de code (60 min)

Corrigez `slow_app.py` en suivant la méthodologie — un commit par optimisation :

1. Vérifiez que la suite pytest fournie passe (votre filet)
2. Appliquez vos corrections du TP 11.1 **une par une** : structure de données, puis générateurs, puis cache — re-profilez entre chaque (le hotspot se déplace !)
3. Le bloc numérique restant : vectorisez-le avec NumPy
4. Tenez le **journal de bord** : tableau étape / changement / durée / gain cumulé (comme l'étude de cas du cours)
5. Objectif : < 5 s en gardant tous les tests verts — puis ARRÊTEZ-VOUS (même si vous voyez encore des micro-gains : argumentez pourquoi)

Travaux pratiques 2/2

TP 11.3 — Benchmarking (45 min)

1. Écrivez un micro-benchmark `pyperf` comparant 4 implémentations de `count_words` (dict + if, `dict.setdefault`, `defaultdict`, `Counter`) sur 3 tailles de corpus — présentez les résultats en tableau et expliquez le classement
2. Reprenez un benchmark **volontairement faussé** fourni (cache chaud, setup compté, résultat ignoré, tailles différentes) : trouvez les 4 fautes et corrigez-le — les conclusions s'inversent-elles ?
3. Mettez en place `pytest-benchmark` sur 2 fonctions critiques de votre TP 11.2 avec un seuil de régression (`--benchmark-compare-fail=mean:10%`) — simulez une régression et vérifiez que la CI la bloque
4. Bonus : comparez votre TP 11.2 sous Python 3.11 et 3.13 (`uv python install 3.13`) — quantifiez le gain gratuit

Livrables : diagnostic (11.1), journal de bord + code commité étape par étape (11.2), benchmarks reproductibles (11.3).

Chapitre 12

Programmation parallèle

Python Perfectionnement — Jour 4

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Distinguer** concurrence et parallélisme, charges I/O-bound et CPU-bound
- **Expliquer** le GIL et ses conséquences réelles (et son avenir)
- **Choisir** le bon modèle : `threading`, `multiprocessing` ou `asyncio`
- **Utiliser** `concurrent.futures` : l'API moderne unifiée des pools
- **Écrire** du code asynchrone idiomatique : `async` / `await`, `gather`, `TaskGroup`
- **Optimiser** avec des Worker Pools dimensionnés et mesurés

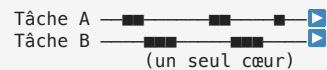
Travaux pratiques : threading versus multiprocessing, asyncio en pratique, optimisation avec Worker Pools.

Le levier n°7 du chap. 11. Les aperçus semés depuis le début (async generators chap. 5, `async with` chap. 9, `AsyncClient` chap. 10) convergent ici.

Le vocabulaire d'abord : concurrence ≠ parallélisme

Concurrence

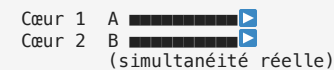
Gérer plusieurs choses à la fois — en les entrelaçant, éventuellement sur un seul cœur.



Un serveur qui jongle entre 1000 connexions : il n'en *traite* qu'une à la fois, mais aucune n'attend la fin des autres.

Parallélisme

Exécuter plusieurs choses au même instant — il faut plusieurs cœurs.



Quatre calculs de simulation sur quatre cœurs : le travail total est divisé.

La question qui décide de tout : où part le temps ? (le profil du chap. 11 le dit)

Charge	Le programme passe son temps à...	Il faut...
I/O-bound	<i>attendre</i> (réseau, disque, BDD)	la concurrence (entrelacer les attentes)
CPU-bound	<i>calculer</i>	le parallélisme (plus de cœurs)

Le GIL : le verrou qui définit Python

Le **Global Interpreter Lock** : dans CPython, un seul thread exécute du bytecode à la fois — même avec 32 cœurs.

```
# CPU-bound : 2 threads ne servent À RIEN
def count(n):
    while n > 0:
        n -= 1

count(100_000_000)          # séquentiel      : 4.1 s
# 2 threads × count(50_000_000)  # threads      : 4.3 s 🚧 (pire : contention !)
# 2 process × count(50_000_000)  # multiprocessing : 2.1 s ✅
```

Pourquoi il existe : il protège les structures internes de CPython (compteurs de références) — il a rendu l'écosystème C simple et rapide... au prix du parallélisme CPU en threads.

Ce que le GIL ne bloque PAS — les deux échappatoires :

- **Les I/O** : un thread qui attend le réseau/disque **relâche le GIL** → les threads sont parfaits en I/O-bound
- **Le code C** : NumPy, hashlib... relâchent le GIL pendant leurs calculs (chap. 10-11)

L'avenir : Python 3.13+ propose un mode *free-threaded* (PEP 703, expérimental) — le GIL devient optionnel. À surveiller ; en production aujourd'hui : raisonnez **avec** le GIL.

threading : la concurrence à mémoire partagée

```
import threading

def fetch_and_store(url, results, idx):
    results[idx] = httpx.get(url, timeout=5).json()    # l'attente réseau RELÂCHE le GIL

urls = [f"https://api.example.com/page/{i}" for i in range(20)]
results = [None] * len(urls)
threads = [threading.Thread(target=fetch_and_store, args=(u, results, i))
            for i, u in enumerate(urls)]

for t in threads: t.start()
for t in threads: t.join()    # attendre la fin de tous

# Séquentiel : 20 × 300 ms = 6 s → 20 threads : ~350 ms (les attentes se SUPERPOSENT)
```

Les threads partagent tout (même mémoire, mêmes objets) — c'est leur force (zéro copie) et leur danger :

Atout	Revers
communication gratuite (objets partagés)	race conditions sur tout état mutable (chap. 1 !)
légers (~8 Mo de pile, démarrage rapide)	bugs non déterministes, durs à reproduire
parfaits pour superposer des I/O	inutiles pour le CPU (GIL)

En pratique on ne crée plus de `Thread` à la main → **pools** (dans 3 slides). Mais il faut comprendre ce qu'ils cachent.

Race conditions : le bug qui n'arrive que le vendredi

```
counter = 0
def increment(n):
    global counter
    for _ in range(n):
        counter += 1          # ⚠️ trois opérations : LOAD, ADD, STORE

threads = [threading.Thread(target=increment, args=(1_000_000,)) for _ in range(4)]
for t in threads: t.start()
for t in threads: t.join()
print(counter)               # attendu 4 000 000 – obtenu : 2 837 419 🤖
```

Le GIL garantit l'atomicité du **bytecode**, pas de vos **opérations** : entre le LOAD et le STORE, un autre thread a pu écrire — des incréments sont **perdus**.

```
lock = threading.Lock()

def increment(n):
    global counter
    for _ in range(n):
        with lock:           # le context manager du chap. 9 – acquisition/libération garanties
            counter += 1      # section critique : UN thread à la fois ✅
```

Les règles des verrous : toujours via `with` ; sections critiques **courtes** (jamais d'I/O sous verrou) ; un ordre d'acquisition global et documenté (sinon : **deadlock** — A attend B qui attend A) ; `RLock` si le même thread doit ré-entrer.

La voie sûre : ne pas partager — `queue.Queue`

Le meilleur verrou est celui qu'on n'écrit pas — faites circuler les données au lieu de les partager :

```
import queue, threading

tasks: queue.Queue = queue.Queue(maxsize=100)      # thread-safe, bornée (backpressure !)
results: queue.Queue = queue.Queue()

def worker():
    while True:
        item = tasks.get()                        # bloque en attendant ; thread-safe par construction
        if item is None:                          # la "pilule empoisonnée" = signal d'arrêt
            break
        results.put(process(item))
        tasks.task_done()

workers = [threading.Thread(target=worker, daemon=True) for _ in range(4)]
for w in workers: w.start()

for job in jobs:
    tasks.put(job)                               # bloque si la file est pleine → régulation amont
tasks.join()                                     # attendre que tout soit traité
for _ in workers:
    tasks.put(None)                             # un signal d'arrêt PAR worker
```

C'est le pattern **producteur/consommateur** — le cœur de tous les Worker Pools, et la version locale du message queuing du chap.

13. La `deque` du chap. 1 est sa cousine *non bloquante* ; `queue.Queue` ajoute le rendez-vous entre threads.

multiprocessing : le vrai parallélisme CPU

Des processus séparés = des interpréteurs séparés = un GIL chacun :

```
from multiprocessing import Pool

def simulate(seed: int) -> float:      # CPU-bound pur
    ...
    return result

if __name__ == "__main__":
    with Pool(processes=8) as pool:      # ⚠ OBLIGATOIRE (spawn ré-importe le module !)
        results = pool.map(simulate, range(1000))  # context manager (chap. 9)
                                                    # réparti sur 8 cœurs → ÷ ~8
```

Le prix du cloisonnement — tout passe par la **sérialisation** (pickle) :

Coût	Conséquence pratique
démarrage d'un process (~50-300 ms)	amortir : pools réutilisés, jamais un process par micro-tâche
pickle des arguments et retours	objets simples et compacts ; lambdas et closures non picklables (chap. 3 !)
mémoire dupliquée par process	attention aux gros datasets × 8 workers

Règle de rentabilité : le calcul par tâche doit **largement dépasser** le coût de transfert — `f(x) = x * 2` sur 1M d'éléments est *plus lent* en multiprocessing qu'en séquentiel. Gros blocs, pas de miettes.

concurrent.futures : l'API unifiée moderne

La même interface pour threads et process — le choix devient **une ligne** :

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed

# I/O-bound → threads
with ThreadPoolExecutor(max_workers=20) as pool:
    pages = list(pool.map(fetch_page, urls))          # l'API map du chap. 4 !

# CPU-bound → process (même code, un mot changé)
with ProcessPoolExecutor(max_workers=8) as pool:
    stats = list(pool.map(simulate, scenarios, chunksize=50))
```

```
# Contrôle fin : submit → Future (la promesse d'un résultat)
with ThreadPoolExecutor(max_workers=20) as pool:
    future_to_url = {pool.submit(fetch_page, u): u for u in urls}

    for future in as_completed(future_to_url):        # dans l'ordre d'ACHÈVEMENT
        url = future_to_url[future]
        try:
            page = future.result(timeout=10)          # relève l'exception du worker ICI
        except httpx.HTTPError as exc:
            log.warning("%s a échoué : %s", url, exc) # UN échec ≠ tout le lot
```

- `map` : simple, résultats **dans l'ordre** ; `submit` + `as_completed` : flux, gestion d'erreurs par tâche
- Une exception dans un worker est **capturée dans la Future** — elle ne ressort qu'au `.result()` : ne l'oubliez jamais silencieusement
- `chunksize` (process) : regrouper les petites tâches pour amortir le pickle

asyncio : la concurrence sans threads

Troisième modèle : un seul thread, une boucle d'événements, et des coroutines qui rendent la main à chaque attente :

```
import asyncio, httpx

async def fetch(client: httpx.AsyncClient, url: str) -> dict:    # une COROUTINE (chap. 5 :
    response = await client.get(url)                            # await = "je m'interromps ICI,      un générateur évolué)
    return response.json()                                       #         boucle, occupe-toi des autres"

async def main():
    async with httpx.AsyncClient(timeout=5) as client:           # async with - chap. 9 !
        results = await asyncio.gather(*(fetch(client, u) for u in urls))
    return results

asyncio.run(main())      # 1000 URLs : ~2 s là où 1000 threads seraient ingérables
```

	threads	asyncio
Ordonnancement	préemptif (l'OS coupe n'importe où)	coopératif (interruption SEULEMENT aux <code>await</code>)
Race conditions	partout	quasi disparues (points de bascule visibles)
Coût par tâche	~Mo (pile)	~Ko (objet coroutine) → dizaines de milliers de tâches
Contrainte	aucune sur le code appelé	tout l'écosystème doit être async

Le `yield` du chap. 5 qui suspendait une fonction... `await` est son descendant direct.

asyncio : structurer avec TaskGroup et timeout

`gather` lance tout et c'est tout — Python 3.11 apporte la concurrence structurée :

```
async def process_order(order_id: int):
    async with asyncio.TaskGroup() as tg:          # le with qui ENCADRE la concurrence
        user_task = tg.create_task(fetch_user(order_id))
        stock_task = tg.create_task(check_stock(order_id))
        price_task = tg.create_task(compute_price(order_id))
    # ~ ici, TOUTES les tâches sont terminées (ou annulées si l'une a échoué)
    return build_summary(user_task.result(), stock_task.result(), price_task.result())
```

- Une tâche échoue → les **autres sont annulées** proprement, l'exception remonte (en `ExceptionGroup`)
- Fini les tâches orphelines qui continuent dans le vide — la durée de vie est **bornée par le bloc** (l'esprit du chap. 9 appliqué à la concurrence)

```
async with asyncio.timeout(3.0):                  # 3.11+ : budget temps GLOBAL
    data = await fetch_everything()                # TimeoutError au-delà

results = await asyncio.gather(*coros, return_exceptions=True)  # l'alternative "best effort" :
ok = [r for r in results if not isinstance(r, BaseException)]    # collecter succès ET échecs

sem = asyncio.Semaphore(10)                       # limiter la concurrence sortante
async def polite_fetch(url):
    async with sem:
        return await client.get(url)               # max 10 requêtes EN MÊME TEMPS
                                                    # (sinon : vous DDoS-ez l'API cible)
```

Le piège mortel d'asyncio : bloquer la boucle

Un seul thread : toute opération bloquante gèle **toutes** les tâches — le serveur entier :

```
async def handler(request):
    data = requests.get(url)      # ✗ requests est SYNCHRONE : la boucle entière
    time.sleep(1)                # ✗ gèle TOUT le monde pendant 1 s
    result = heavy_computation() # ✗ CPU-bound : pareil - personne ne progresse
```

```
async def handler(request):
    data = await client.get(url) # ✓ l'équivalent async (httpx)
    await asyncio.sleep(1)       # ✓ rend la main pendant l'attente

    result = await asyncio.to_thread(heavy_computation) # ✓ le code bloquant DÉPORTÉ
                                                         # dans un thread (sync → async !)
    big = await loop.run_in_executor(process_pool, simulate, n) # CPU lourd → process
```

Les réflexes :

- Dans une coroutine : **jamais** `requests`, `time.sleep`, ni calcul long — leurs équivalents `async` ou `to_thread`
- Détecter les gels : `asyncio.run(main(), debug=True)` logue les callbacks > 100 ms ; `py-spy dump` (chap. 11) montre où la boucle est coincée
- L'async est **viral** : une fonction `async` appelée = toute la pile devient `async` — décision d'**architecture**, pas de détail

Quel modèle choisir ? L'arbre de décision

```
Où part le temps ? (profil – chap. 11)
├── ATTENTE (I/O-bound : réseau, BDD, disque)
│   ├── Beaucoup de connexions simultanées (>~100), écosystème async OK
│   │   └── ▶ asyncio (serveurs, scrapers massifs, websockets, FastAPI)
│   └── Volume modéré, ou bibliothèques synchrones imposées (drivers legacy...)
│       └── ▶ ThreadPoolExecutor (le pragmatique : 90 % des scripts)
├── CALCUL (CPU-bound)
│   ├── Numérique vectorisable ─▶ NumPy/Numba D'ABORD (chap. 11 – souvent suffisant !)
│   └── Sinon ─▶ ProcessPoolExecutor (gros blocs, chunksize)
└── LES DEUX (pipeline : télécharger PUIS parser PUIS calculer)
    └── ▶ composer : asyncio/threads pour l'I/O + ProcessPool pour le calcul
        (run_in_executor fait le pont)
```

Trois rappels d'expérience : la vectorisation bat souvent le multiprocessing (sans les coûts de pickle) · commencez par

`ThreadPoolExecutor` (le plus simple à raisonner) · `asyncio` se choisit **au début** d'un projet, rarement en migration.

Worker Pools : le dimensionnement 1/2

Combien de workers ? Ça se calcule puis ça se mesure — pas un chiffre magique :

CPU-bound : borné par les cœurs

```
import os
n = os.process_cpu_count()      # 3.13+
# (avant : os.cpu_count())

ProcessPoolExecutor(max_workers=n)
```

Au-delà : du *context switching* pur — ça **ralentit**. En dessous : des cœurs perdus. Laissez 1 cœur à l'OS sur une machine partagée.

I/O-bound : borné par l'attente

$$\text{workers} \approx n_{\text{cœurs}} \times (1 + T_{\text{attente}} / T_{\text{calcul}})$$

Tâche à 95 % d'attente → des **dizaines** de threads par cœur sont rentables. La vraie limite : la ressource **en face** (pool BDD, rate limit d'API, descripteurs).

Worker Pools : le dimensionnement 2/2

La méthode : courbe de saturation

```
for n in [1, 2, 4, 8, 16, 32, 64]:  
    with ThreadPoolExecutor(n) as pool:  
        t = chrono(pool.map(task, jobs))  
    print(n, t)
```

workers	durée	débit
1	120 s	8/s
4	31 s	32/s
16	9 s	111/s
32	8 s	125/s
64	8 s	125/s

▣ le coude
(saturé : la
BDD est la limite)

Le coude de la courbe est votre réponse — au-delà, vous ajoutez de la charge, pas du débit. Re-mesurer quand le matériel ou la charge change (chap. 11 : règle 4).

Worker Pools : le pipeline complet

L'étude de cas : 10 000 images à télécharger (I/O) puis redimensionner (CPU) — **composer** les modèles :

```
from concurrent.futures import ThreadPoolExecutor, ProcessPoolExecutor, as_completed

def download(url: str) -> bytes:
    return httpx.get(url, timeout=10).content # I/O-bound -> threads

def resize(raw: bytes) -> bytes:
    return Image.open(io.BytesIO(raw)).resize((800, 600)).tobytes() # CPU-bound -> process

with ThreadPoolExecutor(max_workers=32) as io_pool, \
    ProcessPoolExecutor(max_workers=8) as cpu_pool: # chap. 9 : with multiples !

    download_futures = [io_pool.submit(download, u) for u in urls]

    resize_futures = [
        cpu_pool.submit(resize, f.result()) # le pipeline s'enchaîne :
        for f in as_completed(download_futures) # on redimensionne dès qu'UNE
    ] # image arrive (pas de barrière !)

    done = sum(1 for f in as_completed(resize_futures) if save(f.result()))
```

Version	Durée
séquentielle	~55 min
threads seuls (32)	~9 min (le resize sérialisé par le GIL)
threads + process en pipeline	~2,5 min

Le `as_completed` évite la **barrière** : c'est le `pipeline` du chap. 4, version parallèle.

Bonnes pratiques — synthèse du chapitre

Avant tout :

- **Profilez d'abord** (chap. 11) : I/O-bound ou CPU-bound ? La réponse choisit l'outil
- Le parallélisme est le levier n°7 — algorithme, structures et vectorisation d'abord

Le choix en une ligne : I/O modéré → `ThreadPoolExecutor` · I/O massif → `asyncio` · CPU → NumPy puis `ProcessPoolExecutor` · mixte → pipeline composé

Threads :

- Partagez le moins possible : `queue.Queue` plutôt que des verrous, fonctions pures (chap. 4) plutôt que de l'état
- Verrous : `with lock` (chap. 9), sections courtes, ordre global documenté
- Les exceptions des workers dorment dans les `Future` — `.result()` systématique

Process : `if __name__ == "__main__"` obligatoire · arguments picklables et compacts · gros blocs (`chunksize`) · rentable seulement si le calcul domine le transfert

Asyncio : jamais de code bloquant dans la boucle (`to_thread` est le pont) · `TaskGroup` + `timeout` plutôt que des `gather` orphelins · `Semaphore` pour ménager les services appelés · `debug=True` en dev

Et toujours : dimensionner par la **courbe de saturation**, pas au doigt mouillé — le coude est la réponse.

Travaux pratiques 1/2

TP 12.1 — Threading versus multiprocessing (45 min)

1. On fournit deux fonctions : `fetch_quote(symbol)` (API simulée, ~200 ms d'attente) et `price_option(params)` (Monte-Carlo, ~80 ms de CPU pur). Pour chacune, mesurez sur 200 tâches : séquentiel, `ThreadPoolExecutor(8)`, `ProcessPoolExecutor(8)` — tableau 2×3 des durées
2. Expliquez chaque case du tableau **par le GIL** (3 lignes par case anormale) — pourquoi les threads n'aident pas `price_option` ? pourquoi les process n'aident guère `fetch_quote` ?
3. Reproduisez la race condition du cours (compteur partagé), corrigez-la deux fois : avec un `Lock`, puis **sans verrou** (chaque thread compte localement, agrégation à la fin) — mesurez les trois versions
4. Provoquez un deadlock à deux verrous (A→B vs B→A), observez le gel, corrigez par l'ordre d'acquisition global

TP 12.2 — Asyncio en pratique (45 min)

1. Convertissez le client du TP 12.1.1 en asyncio (`httpx.AsyncClient` + `gather`) : 200 appels — comparez avec la version threads (durée ET mémoire via `tracemalloc`)
2. Ajoutez la robustesse : `Semaphore(10)`, `asyncio.timeout(5)` global, `return_exceptions=True` avec rapport succès/échecs (l'API simulée échoue aléatoirement à 10 %)
3. Réécrivez avec `TaskGroup` : vérifiez qu'un échec **non géré** annule bien les tâches en cours (loguez les annulations)
4. Glissez un `time.sleep(2)` « accidentel » dans un handler : observez le gel avec `debug=True` puis `py-spy dump` (chap. 11), corrigez avec `to_thread` — chronométrez avant/après

Travaux pratiques 2/2

TP 12.3 — Optimisation avec Worker Pools (60 min)

1. Le pipeline images du cours (jeu fourni : 500 images) : implémentez les 3 versions (séquentiel, threads seuls, threads+process en pipeline) — tableau des durées
2. Tracez la **courbe de saturation** du pool de téléchargement (1 → 64 workers, débit en ordonnée) avec matplotlib (chap. 10) — identifiez le coude et justifiez votre `max_workers`
3. Ajoutez backpressure : remplacez la soumission massive par un `queue.Queue(maxsize=50)` entre producteur et workers — vérifiez que la mémoire reste plate (`tracemalloc`) même avec 10 000 tâches en entrée
4. **Synthèse écrite** (5 lignes) : pour ce pipeline, où est passé chaque gain, et quelle est la prochaine limite ?

Livrables : tableaux de mesures + courbe de saturation + code des 3 TP avec tests pytest sur la logique métier (les fonctions pures se testent sans pool — chap. 4 !).

Chapitre 13

Applications distribuées

Python Perfectionnement — Jour 4

Objectifs du chapitre

À l'issue de ce chapitre, vous saurez :

- **Décider** quand distribuer une application — et quand s'en abstenir
- **Concevoir** une architecture distribuée : services, communication, résilience
- **Implémenter** le message queuing : work queues, pub/sub, garanties de livraison
- **Utiliser** les outils Python : Redis, RabbitMQ/pika, Celery
- **Tester en charge** (Locust) et **monitorer** : logs structurés, métriques, tracing

Travaux pratiques : conception d'architecture distribuée, implémentation du message queuing, tests de charge et monitoring.

Le chap. 12 parallélisait **dans** une machine ; ici, on franchit la frontière du processus et de la machine — les files `queue.Queue` deviennent des **brokers**, les pools deviennent des **services**.

Pourquoi distribuer — et à quel prix 1/2

Les raisons légitimes

- **Scalabilité horizontale** : une machine a un plafond (cœurs, RAM) — N machines n'en ont pas
- **Disponibilité** : une instance tombe, les autres servent
- **Découplage d'équipes/cycles** : déployer la facturation sans toucher au catalogue
- **Charges hétérogènes** : le resize d'images (CPU) scalé indépendamment de l'API (I/O)

La règle d'or : un monolithe bien structuré (modules, interfaces propres — chap. 6/7) se distribue **plus tard sans douleur**.

Distribuer un code mal découpé distribue surtout le désordre. *Commencez modulaire, distribuez sous contrainte prouvée* (chap. 11 : mesurer !).

Pourquoi distribuer — et à quel prix 2/2

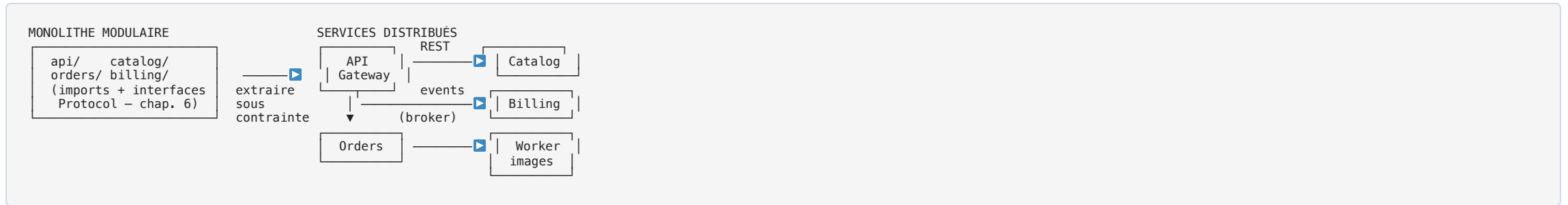
La facture (toujours sous-estimée)

Les **8 illusions** du distribué (Deutsch) — les croire coûte des incidents :

1. le réseau est fiable
2. la latence est nulle
3. la bande passante est infinie
4. le réseau est sûr
5. la topologie ne change pas
6. il y a un administrateur
7. le transport est gratuit
8. le réseau est homogène

Chaque appel réseau peut **échouer, traîner, arriver deux fois ou jamais** — tout le chapitre découle de cette phrase.

Architecture : du monolithe aux services



Les questions de conception — dans l'ordre :

1. **Découpage** : par domaine métier (orders, billing...), jamais par couche technique
2. **Communication** : synchrone (REST/gRPC — réponse immédiate requise) ou **asynchrone** (événements — le reste)
3. **Données** : chaque service possède **ses** données ; les autres y accèdent par son API, jamais par sa base
4. **Pannes** : que se passe-t-il si CE service est lent ? indisponible ? — la réponse doit exister **avant** la prod

L'injection de dépendances du chap. 6 paie ici : un `Repository` Protocol s'implémente en local **ou** en client HTTP — le métier ne voit pas la différence.

Communication synchrone : la résilience obligatoire

Un appel REST entre services = tous les réflexes du chap. 10, **plus** la protection contre les cascades :

La CASCADE DE PANNES : Billing est lent → Orders attend → API attend → tout le site est gelé
(vos 32 workers – chap. 12 – sont tous bloqués sur le même service malade)

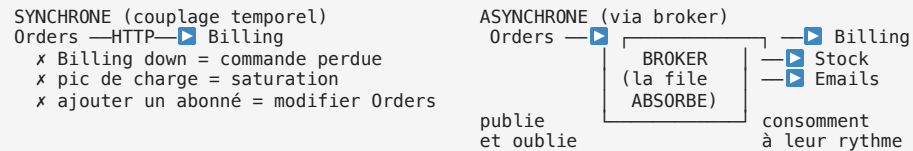
```
import httpx, stamina
class BillingClient:
    def __init__(self):
        self._client = httpx.Client(
            base_url="http://billing:8000",
            timeout=httpx.Timeout(2.0, connect=0.5),    # ① timeout COURT et systématique
        )
        @stamina.retry(on=httpx.TransportError, attempts=3) # ② retry + backoff + jitter
        def get_invoice(self, order_id: int) -> dict:    # (sur les erreurs TRANSITOIRES
```

Protection	Rôle	Outil
Timeout	borner l'attente — toujours, partout	httpx (chap. 10)
Retry + backoff + jitter	absorber les erreurs transitoires	stamina , votre @retry (chap. 3)
Circuit breaker	service malade → échouer vite sans l'appeler, retester	<code>retryer</code> maison

⚠ Le retry exige l'**idempotence** de l'appelé (slide garanties) — re-tenter un débit bancaire non idempotent = double facturation.

Message queuing : découpler par le broker

L' `EventBus` du chap. 7 et la `queue.Queue` du chap. 12, version **inter-processus, persistante, multi-machines** :



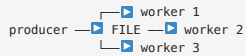
Ce que le broker apporte :

Propriété	Effet concret
Découplage temporel	Billing peut être down 5 min — les messages attendent, rien n'est perdu
Lissage de charge	le pic de commandes du Black Friday s'étale sur la file (backpressure — chap. 12)
Découplage des émetteurs	Orders publie <code>order.paid</code> ; ajouter un consommateur = zéro changement côté Orders
Scalabilité par consommateurs	la file s'allonge ? on ajoute des workers (le pool du chap. 12, multi-machines)

Le prix : cohérence à terme (eventual consistency) — entre la publication et le traitement, le système est transitoirement « en retard ». L'UX et le métier doivent l'accepter ; si la réponse doit être immédiate → synchrone.

Les deux patterns fondamentaux

Work queue : distribuer du travail



Chaque message est traité par **UN SEUL** worker (compétition). C'est le producteur/consommateur du chap. 12, multi-machines.

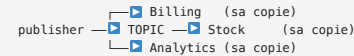
Usage : tâches lourdes hors requête HTTP — génération de PDF, resize d'images, envoi d'emails, imports.

La distinction à bien poser (elle structure le système) :

- « *Fais ce travail* » (impératif, un destinataire) → **commande** → work queue
- « *Ceci s'est produit* » (fait accompli, N intéressés) → **événement** → pub/sub

Les brokers réels combinent les deux : RabbitMQ via les *exchanges*, Kafka via les *consumer groups*.

Pub/Sub : diffuser des événements



Chaque abonné reçoit **TOUS** les messages (diffusion). C'est l'Observer du chap. 7, distribué.

Usage : propager des **faits** métier — `order.paid` , `user.registered` — sans connaître les intéressés.

Garanties de livraison : le contrat à connaître

Que se passe-t-il si un worker **meurt en plein traitement** ? Tout se joue sur l'**acquiescement** (ack) :

Garantie	Mécanique	Risque résiduel
At-most-once	ack <i>avant</i> traitement	message perdu si crash
At-least-once  le standard	ack <i>après</i> traitement	message rejoué si crash après traitement, avant ack
Exactly-once	transactions bout en bout	rare, coûteux — en pratique : <i>at-least-once + idempotence</i>

Conséquence inévitable : vos consommateurs verront des **doublons**. La réponse n'est pas dans le broker, elle est dans **votre code** :

```
def handle_payment(msg):
    # IDEMPOTENCE : rejouer le message ne change pas le résultat final
    if db.exists("processed_events", msg["event_id"]):
        return ack(msg)
    with transaction(db):
        db.insert("payments", build_payment(msg))
        db.insert("processed_events", {"id": msg["event_id"]})
    ack(msg)
```

① clé d'idempotence
déjà vu → ack et c'est tout
② effet + marquage ATOMIQUES
(chap. 9 !)
③ ack EN DERNIER

Et les échecs répétés ? Un message qui fait crasher le worker à chaque tentative (*poison message*) doit partir en **Dead Letter Queue** après N essais — sinon il bloque la file pour toujours. La DLQ s'alarme et s'inspecte : c'est votre boîte à incidents.

Implémentation 1 : Redis, le broker minimaliste

Redis : structure de données en mémoire, réseau — parfait pour débiter (et souvent suffisant) :

```
import redis, json
r = redis.Redis(host="localhost", decode_responses=True)

# — WORK QUEUE : une liste + pop bloquant —————
def produce(task: dict):
    r.lpush("tasks", json.dumps(task))          # le producteur empile

def worker():
    while True:
        _, raw = r.brpop("tasks")              # bloque jusqu'au prochain message
        process(json.loads(raw))               # (BRPOP = le .get() du chap. 12)

# — PUB/SUB : diffusion (volatile : abonné absent = message manqué !) —
r.publish("events.orders", json.dumps({"type": "order.paid", "id": 42}))

sub = r.psubsub()
sub.subscribe("events.orders")
for message in sub.listen():
    if message["type"] == "message":
        handle(json.loads(message["data"]))
```

À retenir	
LPUSH / BRPOP	work queue at-most-once basique ; fiabiliser → LMOVE vers une liste « en cours »
Pub/Sub Redis	éphémère — pour du durable : Redis Streams (XADD / XREADGROUP : ack, groupes, relecture)
Bonus	le même Redis sert de cache et de verrou distribué (SET nx ex) — l'outil à tout faire

Implémentation 2 : RabbitMQ, le broker complet

AMQP : routage riche, ack natifs, DLQ intégrées — le standard des architectures à messages :

```
import pika, json

conn = pika.BlockingConnection(pika.ConnectionParameters("localhost"))
ch = conn.channel()

# — PRODUCTEUR : publier vers un EXCHANGE avec une clé de routage —
ch.exchange_declare("orders", exchange_type="topic", durable=True)
ch.basic_publish(
    exchange="orders", routing_key="order.paid.fr",
    body=json.dumps(event),
    properties=pika.BasicProperties(delivery_mode=2), # message PERSISTANT
)

# — CONSOMMATEUR : une queue, un binding, un ack EXPLICITE —
ch.queue_declare("billing", durable=True,
    arguments={"x-dead-letter-exchange": "dlx"}) # DLQ branchée !
ch.queue_bind("billing", exchange="orders", routing_key="order.paid.*")
ch.basic_qos(prefetch_count=8) # max 8 messages non-ackés/worker

def on_message(ch, method, props, body):
    try:
        handle(json.loads(body))
        ch.basic_ack(method.delivery_tag) # at-least-once : ack APRÈS
    except TransientError:
        ch.basic_nack(method.delivery_tag, requeue=True) # re-tenter
    except Exception:
        ch.basic_nack(method.delivery_tag, requeue=False) # → Dead Letter Queue

ch.basic_consume("billing", on_message)
ch.start_consuming()
```

L'**exchange topic** fait le pub/sub (`order.paid.*`), les **queues** font la work queue — les deux patterns dans le même broker. `prefetch_count` = la backpressure du chap. 12.

Implémentation 3 : Celery, la task queue intégrée

Pour le cas le plus courant — « *exécuter cette fonction ailleurs, plus tard* » — Celery industrialise tout :

```
# tasks.py
from celery import Celery

app = Celery("shop", broker="amqp://localhost", backend="redis://localhost")

@app.task(bind=True, max_retries=3, acks_late=True)    # le décorateur-registre (chap. 3 !)
def generate_invoice(self, order_id: int) -> str:
    try:
        pdf = render_invoice(order_id)
    except TransientError as exc:
        raise self.retry(exc=exc, countdown=2 ** self.request.retries)    # backoff
    return upload(pdf)
```

```
# Dans la vue web (FastAPI — chap. 10) : on répond IMMÉDIATEMENT
result = generate_invoice.delay(order_id=42)          # publié au broker → 202 Accepted
result.id                                             # à rendre au client pour suivi

# Plus tard / ailleurs :
result.ready(), result.get(timeout=2)                # état et résultat (via backend)
```

```
celery -A tasks worker --concurrency=8 --loglevel=info    # les workers (chap. 12, clé en main)
celery -A tasks beat                                     # + tâches périodiques (cron distribué)
```

Celery vous donne : retries, routage, périodicité, chaînes/groupes de tâches (`chain`, `group`, `chord`), monitoring (Flower). **Vous gardez à votre charge** : l'idempotence (`acks_late=True` = at-least-once !) et des arguments **sérialisables et compacts** (des ids, pas des objets — chap. 12, même règle que pickle).

Le contrat de messages : sérialisation et versions

Le message est une **API publique** : des services que vous ne connaissez pas le consommeront dans 2 ans.

```
from pydantic import BaseModel, Field
from datetime import datetime, timezone
import uuid

class OrderPaid(BaseModel):
    event_id: uuid.UUID = Field(default_factory=uuid.uuid4) # ① clé d'IDEMPOTENCE
    event_type: str = "order.paid"
    version: int = 2 # ② version EXPLICITE
    occurred_at: datetime # ③ l'heure du FAIT (pas de l'envoi)
    order_id: int
    amount_cents: int # ④ des centimes - jamais de float (chap. 1 !)
    currency: str = "EUR"

payload = OrderPaid(occurred_at=now_utc(), order_id=42, amount_cents=9990)
ch.basic_publish(..., body=payload.model_dump_json())

# Côté consommateur : VALIDER avant de traiter (la frontière - chap. 10)
event = OrderPaid.model_validate_json(body) # message malformé → erreur claire → DLQ
```

Les règles d'évolution (sinon : déploiement simultané de tout, adieu le découplage) :

- **Ajouter** un champ optionnel : OK — les anciens consommateurs l'ignorent
- **Renommer/supprimer/retyper** : interdit — publier `version: 3` et consommer les deux pendant la transition
- JSON par défaut ; gros volumes/typage fort → Avro/Protobuf + *schema registry*
- **⚠ Jamais pickle entre services** (sécurité — chap. 10 — et couplage Python-Python)

Tests de charge : Locust

Un système distribué se valide **sous charge** — les pannes intéressantes n'apparaissent qu'à 200 utilisateurs simultanés :

```
# locustfile.py – des utilisateurs SIMULÉS écrits en Python
from locust import HttpUser, task, between

class Shopper(HttpUser):
    wait_time = between(1, 3)           # pensée entre actions (réalisme)

    @task(10)                           # poids : 10× plus fréquent
    def browse(self):
        self.client.get("/products", name="/products")

    @task(1)
    def order(self):
        with self.client.post("/orders", json={"product_id": 7, "qty": 1},
                               catch_response=True) as r:
            if r.status_code == 202 and r.elapsed.total_seconds() < 0.5:
                r.success()
            else:
                r.failure(f"{r.status_code} en {r.elapsed.total_seconds():.2f}s")
```

```
locust --headless -u 200 -r 20 -t 5m --host http://localhost:8000 --csv report
#      200 users, +20/s, 5 minutes → percentiles, RPS, erreurs (UI web sinon)
```

Lire les résultats : le **p95/p99**, pas la moyenne (la moyenne cache les victimes) · le débit **au plateau** (la courbe de saturation du chap. 12, niveau système) · le taux d'erreur **pendant** la montée. Et testez les **scénarios de panne** : tuez un worker pendant le test — les messages sont-ils repris ? la DLQ se remplit-elle ?

Monitoring : voir ce qui se passe (ou mourir aveugle)

Dans le distribué, le debugger n'existe plus — l'**observabilité** le remplace. Les trois piliers :

```
# ① LOGS STRUCTURÉS – du JSON cherchable, pas des phrases
import structlog
log = structlog.get_logger()
log.info("order_processed", order_id=42, amount_cents=9990,
        duration_ms=87, worker="billing-3")
# {"event": "order_processed", "order_id": 42, ..., "timestamp": "2026-06-04T..."}
```

```
# ② MÉTRIQUES – compteurs et histogrammes agrégés (Prometheus + Grafana)
from prometheus_client import Counter, Histogram
ORDERS = Counter("orders_processed_total", "Commandes traitées", ["status"])
LATENCY = Histogram("order_duration_seconds", "Durée de traitement")

@LATENCY.time()                # vos décorateurs du chap. 3, en prod !
def handle_order(msg):
    ...
    ORDERS.labels(status="ok").inc()
```

```
# ③ TRACING – suivre UNE requête à travers TOUS les services
# Le minimum vital : un correlation_id propagé dans chaque message et chaque log
log.info("payment_started", correlation_id=msg["correlation_id"]) # → reconstituer le fil
# L'industrialisation : OpenTelemetry (traces inter-services automatiques, Jaeger/Tempo)
```

Les alertes qui comptent pour le messaging : profondeur de file (elle monte = consommateurs débordés), âge du plus vieux message, taille de la **DLQ** (> 0 = incident), taux d'erreur par consommateur, lag des consumer groups.

Bonnes pratiques — synthèse du chapitre

Conception :

- Distribuez sous **contrainte prouvée**, pas par mode — un monolithe modulaire (chap. 6/7) est un excellent système distribué en devenir
- Commande (work queue) $\neq$ événement (pub/sub) — poser la distinction tôt structure tout
- Chaque service possède ses données ; les messages sont des **API versionnées** (Pydantic aux frontières)

Messaging — le kit de survie :

- At-least-once partout $\rightarrow$ **idempotence partout** (event_id + marquage atomique — chap. 9)
- Ack après traitement, `prefetch` pour la backpressure, **DLQ** branchée et surveillée
- Des ids dans les messages, pas des objets ; jamais pickle ; montants en centimes

Synchrone quand il le faut :

- Timeout court + retry-avec-jitter sur erreurs transitoires + circuit breaker + dégradation prévue

Valider et observer :

- Locust avant la prod : p95/p99 au plateau, et des **scénarios de panne** (tuer un worker en plein test)
- Logs structurés + métriques + correlation_id dès le **premier jour** — pas après le premier incident
- Alertes sur les files : profondeur, âge, DLQ — c'est le pouls du système

Travaux pratiques 1/2

Environnement fourni : `docker compose up` (Redis, RabbitMQ, l'API FastAPI du TP 10.3 et ses workers).

TP 13.1 — Conception d'architecture distribuée (45 min)

1. Cahier des charges fourni : boutique en ligne (commandes, paiement, facturation PDF, emails, stats) avec contraintes chiffrées (pic à 50 commandes/s, facturation tolérée à +5 min, paiement < 2 s). **Sur papier** : découpez en services, classez chaque interaction *commande/événement* et *sync/async*, dessinez le flux d'une commande
2. Pour chaque flèche du schéma : que se passe-t-il si la cible est **down** ? Notez la réponse (retry ? file ? dégradation ? erreur utilisateur ?) — c'est la colonne la plus importante du tableau
3. Confrontation en groupe (15 min) : comparez deux conceptions, défendez vos choix sync/async

TP 13.2 — Implémentation du message queuing (60 min)

1. **Work queue Redis** : l'endpoint `POST /orders` publie une tâche `generate_invoice` ; 3 workers la consomment (`BRPOP`). Tuez un worker pendant un lot de 100 : combien de messages perdus ? Fiabilisez avec `LMOVE` + liste « processing »
2. **Pub/Sub RabbitMQ** : publiez `order.paid` sur un exchange topic ; trois consommateurs (billing, stock, emails) avec leurs queues durables. Vérifiez : billing arrêté 1 min ne perd **rien** ; un message malformé part en **DLQ** après 3 nacks
3. **Idempotence** : les messages sont publiés en double (script fourni) — implémentez la déduplication par `event_id` (le pattern complet du cours, sur SQLite) et prouvez par un test qu'un rejeu ne crée pas de seconde facture
4. Bonus : remplacez le worker maison par une tâche **Celery** (`acks_late` , retries avec backoff) — comparez les lignes de code

Travaux pratiques 2/2

TP 13.3 — Tests de charge et monitoring (45 min)

1. Instrumentez l'API et les workers : logs structurés (`structlog`) avec `correlation_id` propagé HTTP → message → worker, et métriques Prometheus (commandes traitées, latence, profondeur de file via `LLEN`)
2. Locust : montée à 100 users sur le parcours browse/order — relevez p50/p95/p99 et RPS au plateau ; identifiez le goulot (la courbe de saturation du chap. 12 vous attend)
3. **Chaos** : pendant un tir de 3 min, tuez 2 des 3 workers 30 s puis relancez-les — montrez via vos métriques : la file qui gonfle, le rattrapage, zéro message perdu ; retrouvez UNE commande de bout en bout par son `correlation_id` dans les logs
4. **Synthèse écrite** (5 lignes) : le goulot identifié, la prochaine action d'optimisation, et l'alerte que vous configureriez en premier

Livrables : schéma + tableau des pannes (13.1), code + test d'idempotence (13.2), rapport Locust + captures de métriques + synthèse (13.3).